

Day-3

Browser: Document, Events

JS

- Document
 - Browser environment, specs
 - DOM tree
 - Walking the DOM
 - Searching: getElement*, querySelector*
 - Node properties: type, tag and contents
 - Attributes and properties
 - Modifying the document
 - Styles and classes
 - Element size and scrolling
 - Window sizes and scrolling
 - Coordinates
- Introduction to Events
 - Introduction to browser events
 - Bubbling and capturing
 - Event delegation
 - Browser default actions
 - Dispatching custom events

Learning how to manage the browser page: add elements, manipulate their size and position, dynamically create interfaces and interact with the visitor.

Document

Here we'll learn to manipulate a web-page using JavaScript.

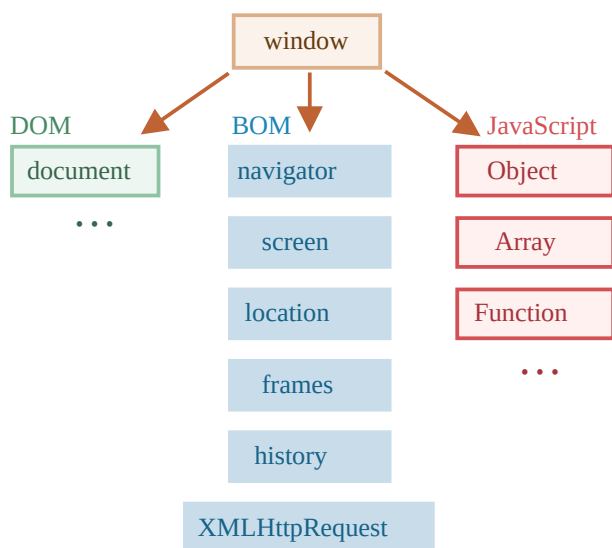
Browser environment, specs

The JavaScript language was initially created for web browsers. Since then, it has evolved into a language with many uses and platforms.

A platform may be a browser, or a web-server or another *host*, or even a “smart” coffee machine if it can run JavaScript. Each of these provides platform-specific functionality. The JavaScript specification calls that a *host environment*.

A host environment provides its own objects and functions in addition to the language core. Web browsers give a means to control web pages. Node.js provides server-side features, and so on.

Here's a bird's-eye view of what we have when JavaScript runs in a web browser:



There's a “root” object called `window`. It has two roles:

1. First, it is a global object for JavaScript code, as described in the chapter [Global object](#).
2. Second, it represents the “browser window” and provides methods to control it.

For instance, we can use it as a global object:

```
function sayHi() {  
  alert("Hello");  
}
```

```
// global functions are methods of the global object:  
window.sayHi();
```

And we can use it as a browser window, to show the window height:

```
alert(window.innerHeight); // inner window height
```

There are more window-specific methods and properties, which we'll cover later.

DOM (Document Object Model)

The Document Object Model, or DOM for short, represents all page content as objects that can be modified.

The `document` object is the main “entry point” to the page. We can change or create anything on the page using it.

For instance:

```
// change the background color to red  
document.body.style.background = "red";  
  
// change it back after 1 second  
setTimeout(() => document.body.style.background = "", 1000);
```

Here, we used `document.body.style`, but there's much, much more. Properties and methods are described in the specification: [DOM Living Standard](#) .

DOM is not only for browsers

The DOM specification explains the structure of a document and provides objects to manipulate it. There are non-browser instruments that use DOM too.

For instance, server-side scripts that download HTML pages and process them can also use the DOM. They may support only a part of the specification though.

CSSOM for styling

There's also a separate specification, [CSS Object Model \(CSSOM\)](#)  for CSS rules and stylesheets, that explains how they are represented as objects, and how to read and write them.

The CSSOM is used together with the DOM when we modify style rules for the document. In practice though, the CSSOM is rarely required, because we rarely need to modify CSS rules from JavaScript (usually we just add/remove CSS classes, not modify their CSS rules), but that's also possible.

BOM (Browser Object Model)

The Browser Object Model (BOM) represents additional objects provided by the browser (host environment) for working with everything except the document.

For instance:

- The [navigator](#)  object provides background information about the browser and the operating system. There are many properties, but the two most widely known are: `navigator.userAgent` – about the current browser, and `navigator.platform` – about the platform (can help to differentiate between Windows/Linux/Mac etc).
- The [location](#)  object allows us to read the current URL and can redirect the browser to a new one.

Here's how we can use the `location` object:

```
alert(location.href); // shows current URL
if (confirm("Go to Wikipedia?")) {
    location.href = "https://wikipedia.org"; // redirect the browser to another URL
}
```

The functions `alert/confirm/prompt` are also a part of the BOM: they are not directly related to the document, but represent pure browser methods for communicating with the user.

Specifications

The BOM is a part of the general [HTML specification](#) .

Yes, you heard that right. The HTML spec at <https://html.spec.whatwg.org>  is not only about the “HTML language” (tags, attributes), but also covers a bunch of objects, methods, and browser-specific DOM extensions. That’s “HTML in broad terms”. Also, some parts have additional specs listed at <https://spec.whatwg.org> .

Summary

Talking about standards, we have:

DOM specification

Describes the document structure, manipulations, and events, see <https://dom.spec.whatwg.org> .

CSSOM specification

Describes stylesheets and style rules, manipulations with them, and their binding to documents, see <https://www.w3.org/TR/cssom-1/> .

HTML specification

Describes the HTML language (e.g. tags) and also the BOM (browser object model) – various browser functions: `setTimeout`, `alert`, `location` and so on, see <https://html.spec.whatwg.org> . It takes the DOM specification and extends it with many additional properties and methods.

Additionally, some classes are described separately at <https://spec.whatwg.org/> .

Please note these links, as there’s so much to learn that it’s impossible to cover everything and remember it all.

When you’d like to read about a property or a method, the Mozilla manual at <https://developer.mozilla.org/en-US/>  is also a nice resource, but the corresponding spec may be better: it’s more complex and longer to read, but will make your fundamental knowledge sound and complete.

To find something, it’s often convenient to use an internet search “WHATWG [term]” or “MDN [term]”, e.g <https://google.com?q=whatwg+localStorage> , <https://google.com?q=mdn+localStorage> .

Now, we’ll get down to learning the DOM, because the document plays the central role in the UI.

DOM tree

The backbone of an HTML document is tags.

According to the Document Object Model (DOM), every HTML tag is an object. Nested tags are “children” of the enclosing one. The text inside a tag is an object as well.

All these objects are accessible using JavaScript, and we can use them to modify the page.

For example, `document.body` is the object representing the `<body>` tag.

Running this code will make the `<body>` red for 3 seconds:

```
document.body.style.background = 'red'; // make the background red

setTimeout(() => document.body.style.background = '', 3000); // return back
```

Here we used `style.background` to change the background color of `document.body`, but there are many other properties, such as:

- `innerHTML` – HTML contents of the node.
- `offsetWidth` – the node width (in pixels)
- ...and so on.

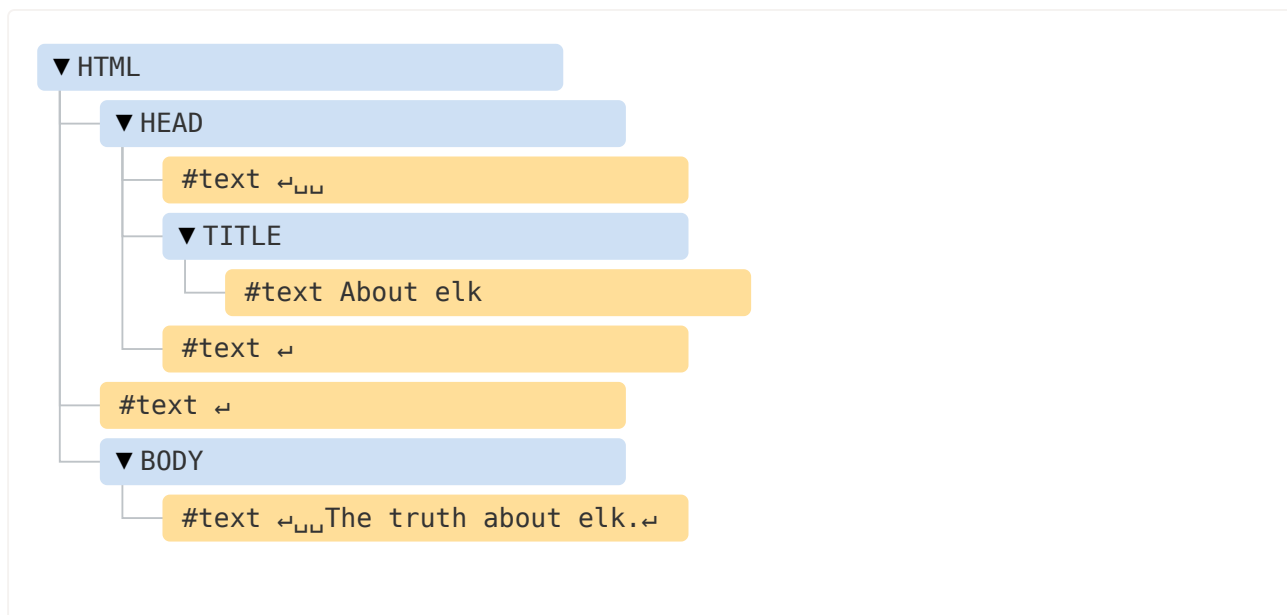
Soon we'll learn more ways to manipulate the DOM, but first we need to know about its structure.

An example of the DOM

Let's start with the following simple document:

```
<!DOCTYPE HTML>
<html>
<head>
  <title>About elk</title>
</head>
<body>
  The truth about elk.
</body>
</html>
```

The DOM represents HTML as a tree structure of tags. Here's how it looks:



Every tree node is an object.

Tags are *element nodes* (or just elements) and form the tree structure: `<html>` is at the root, then `<head>` and `<body>` are its children, etc.

The text inside elements forms *text nodes*, labelled as `#text`. A text node contains only a string. It may not have children and is always a leaf of the tree.

For instance, the `<title>` tag has the text `"About elk"`.

Please note the special characters in text nodes:

- a newline: `\n` (in JavaScript known as `\n`)
- a space:

Spaces and newlines are totally valid characters, like letters and digits. They form text nodes and become a part of the DOM. So, for instance, in the example above the `<head>` tag contains some spaces before `<title>`, and that text becomes a `#text` node (it contains a newline and some spaces only).

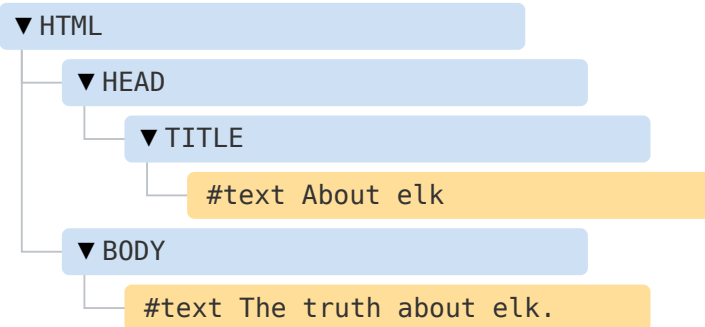
There are only two top-level exclusions:

1. Spaces and newlines before `<head>` are ignored for historical reasons.
2. If we put something after `</body>`, then that is automatically moved inside the `body`, at the end, as the HTML spec requires that all content must be inside `<body>`. So there can't be any spaces after `</body>`.

In other cases everything's straightforward – if there are spaces (just like any character) in the document, then they become text nodes in the DOM, and if we remove them, then there won't be any.

Here are no space-only text nodes:

```
<!DOCTYPE HTML>
<html><head><title>About elk</title></head><body>The truth about elk.</body></html>
```



i Spaces at string start/end and space-only text nodes are usually hidden in tools

Browser tools (to be covered soon) that work with DOM usually do not show spaces at the start/end of the text and empty text nodes (line-breaks) between tags.

Developer tools save screen space this way.

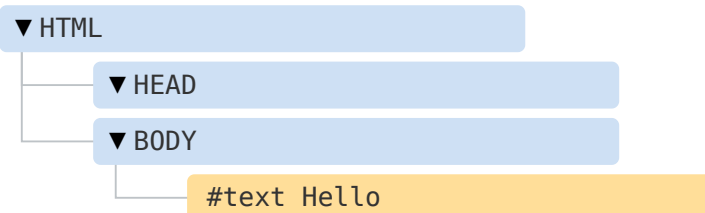
On further DOM pictures we'll sometimes omit them when they are irrelevant. Such spaces usually do not affect how the document is displayed.

Autocorrection

If the browser encounters malformed HTML, it automatically corrects it when making the DOM.

For instance, the top tag is always `<html>`. Even if it doesn't exist in the document, it will exist in the DOM, because the browser will create it. The same goes for `<body>`.

As an example, if the HTML file is the single word `"Hello"`, the browser will wrap it into `<html>` and `<body>`, and add the required `<head>`, and the DOM will be:

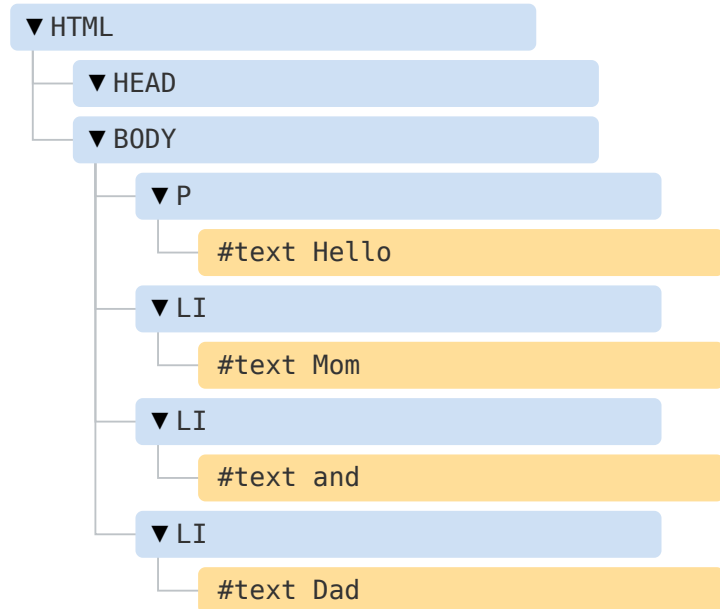


While generating the DOM, browsers automatically process errors in the document, close tags and so on.

A document with unclosed tags:

```
<p>Hello  
<li>Mom  
<li>and  
<li>Dad
```

...will become a normal DOM as the browser reads tags and restores the missing parts:



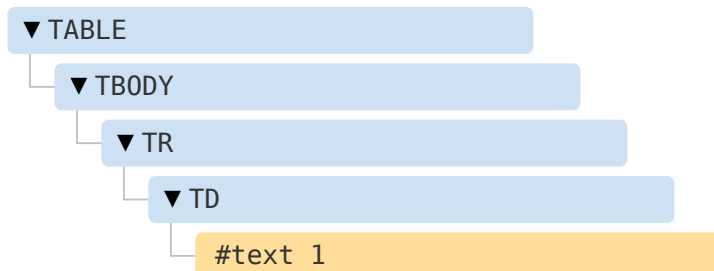
⚠ Tables always have `<tbody>`

An interesting “special case” is tables. By DOM specification they must have `<tbody>` tag, but HTML text may omit it. Then the browser creates `<tbody>` in the DOM automatically.

For the HTML:

```
<table id="table"><tr><td>1</td></tr></table>
```

DOM-structure will be:



You see? The `<tbody>` appeared out of nowhere. We should keep this in mind while working with tables to avoid surprises.

Other node types

There are some other node types besides elements and text nodes.

For example, comments:

```
<!DOCTYPE HTML>
<html>
<body>
  The truth about elk.
  <ol>
    <li>An elk is a smart</li>
    <!-- comment -->
    <li>...and cunning animal!</li>
  </ol>
</body>
</html>
```



We can see here a new tree node type – *comment node*, labeled as `#comment`, between two text nodes.

We may think – why is a comment added to the DOM? It doesn't affect the visual representation in any way. But there's a rule – if something's in HTML, then it also must be in the DOM tree.

Everything in HTML, even comments, becomes a part of the DOM.

Even the `<!DOCTYPE . . .>` directive at the very beginning of HTML is also a DOM node. It's in the DOM tree right before `<html>`. Few people know about that. We are not going to touch that node, we even don't draw it on diagrams, but it's there.

The `document` object that represents the whole document is, formally, a DOM node as well.

There are [12 node types](#). In practice we usually work with 4 of them:

1. `document` – the “entry point” into DOM.
2. element nodes – HTML-tags, the tree building blocks.
3. text nodes – contain text.
4. comments – sometimes we can put information there, it won't be shown, but JS can read it from the DOM.

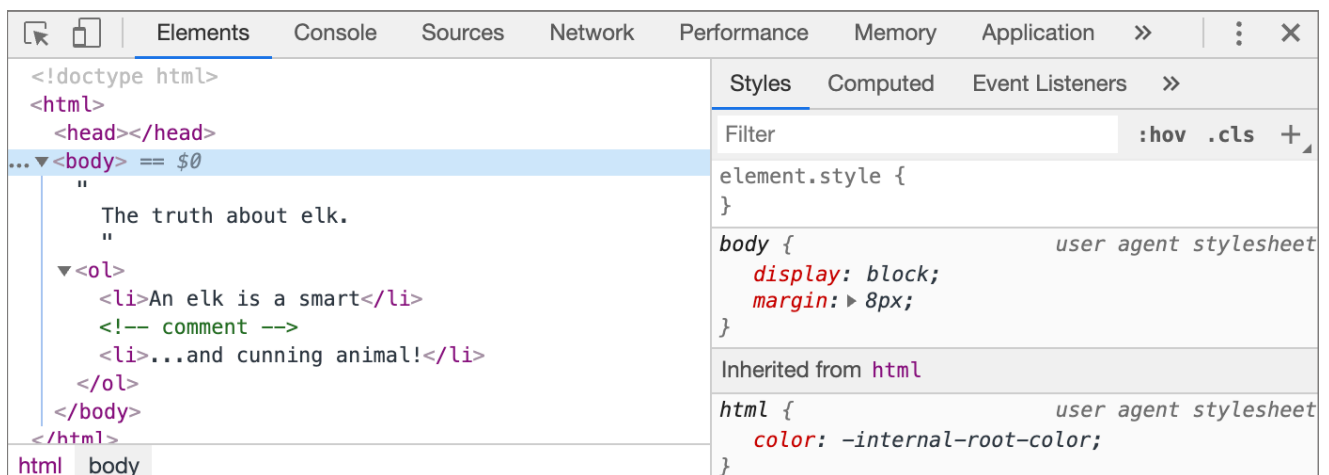
See it for yourself

To see the DOM structure in real-time, try [Live DOM Viewer](#) . Just type in the document, and it will show up as a DOM at an instant.

Another way to explore the DOM is to use the browser developer tools. Actually, that's what we use when developing.

To do so, open the web page [elk.html](#), turn on the browser developer tools and switch to the Elements tab.

It should look like this:

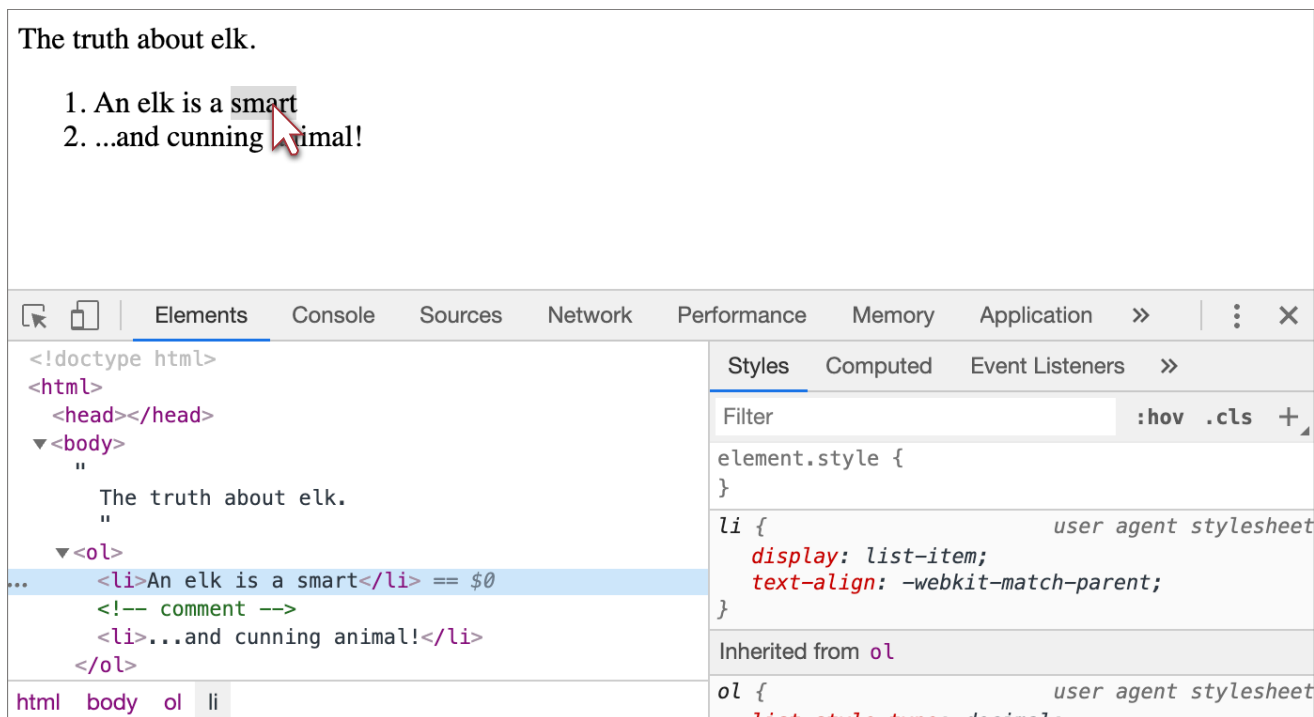


You can see the DOM, click on elements, see their details and so on.

Please note that the DOM structure in developer tools is simplified. Text nodes are shown just as text. And there are no “blank” (space only) text nodes at all. That's fine, because most of the time we are interested in element nodes.

Clicking the  button in the left-upper corner allows us to choose a node from the webpage using a mouse (or other pointer devices) and “inspect” it (scroll to it in the Elements tab). This works great when we have a huge HTML page (and corresponding huge DOM) and would like to see the place of a particular element in it.

Another way to do it would be just right-clicking on a webpage and selecting “Inspect” in the context menu.



At the right part of the tools there are the following subtabs:

- **Styles** – we can see CSS applied to the current element rule by rule, including built-in rules (gray). Almost everything can be edited in-place, including the dimensions/margins/paddings of the box below.
- **Computed** – to see CSS applied to the element by property: for each property we can see a rule that gives it (including CSS inheritance and such).
- **Event Listeners** – to see event listeners attached to DOM elements (we'll cover them in the next part of the tutorial).
- ...and so on.

The best way to study them is to click around. Most values are editable in-place.

Interaction with console

As we work the DOM, we also may want to apply JavaScript to it. Like: get a node and run some code to modify it, to see the result. Here are few tips to travel between the Elements tab and the console.

For the start:

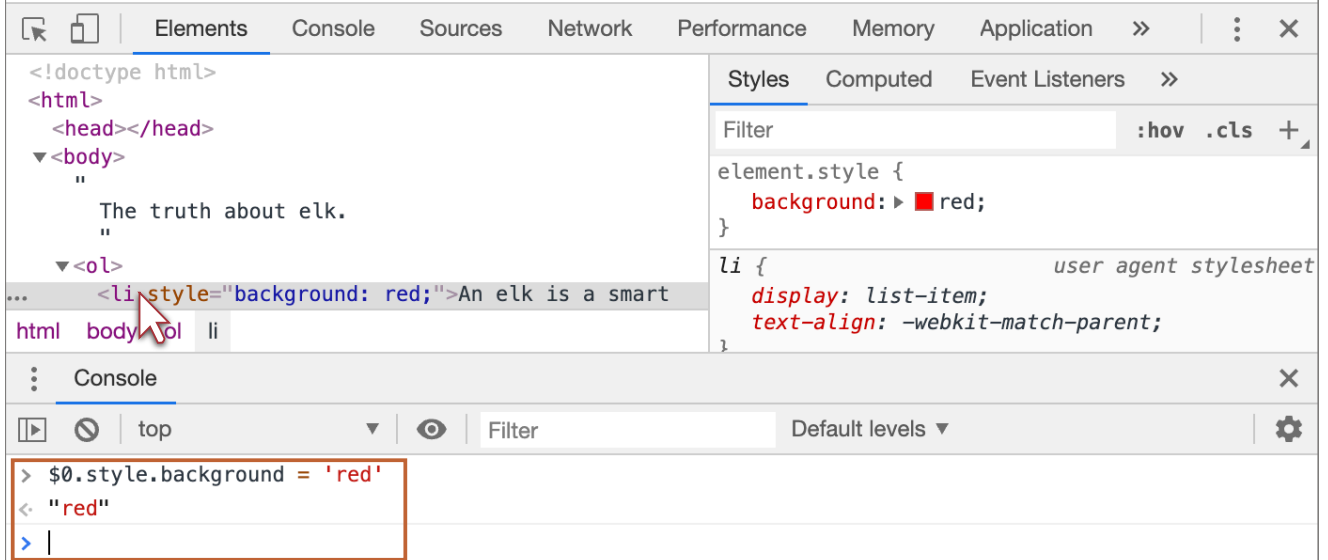
1. Select the first `<li>` in the Elements tab.
2. Press `Esc` – it will open console right below the Elements tab.

Now the last selected element is available as `$0`, the previously selected is `$1` etc.

We can run commands on them. For instance, `$0.style.background = 'red'` makes the selected list item red, like this:

The truth about elk.

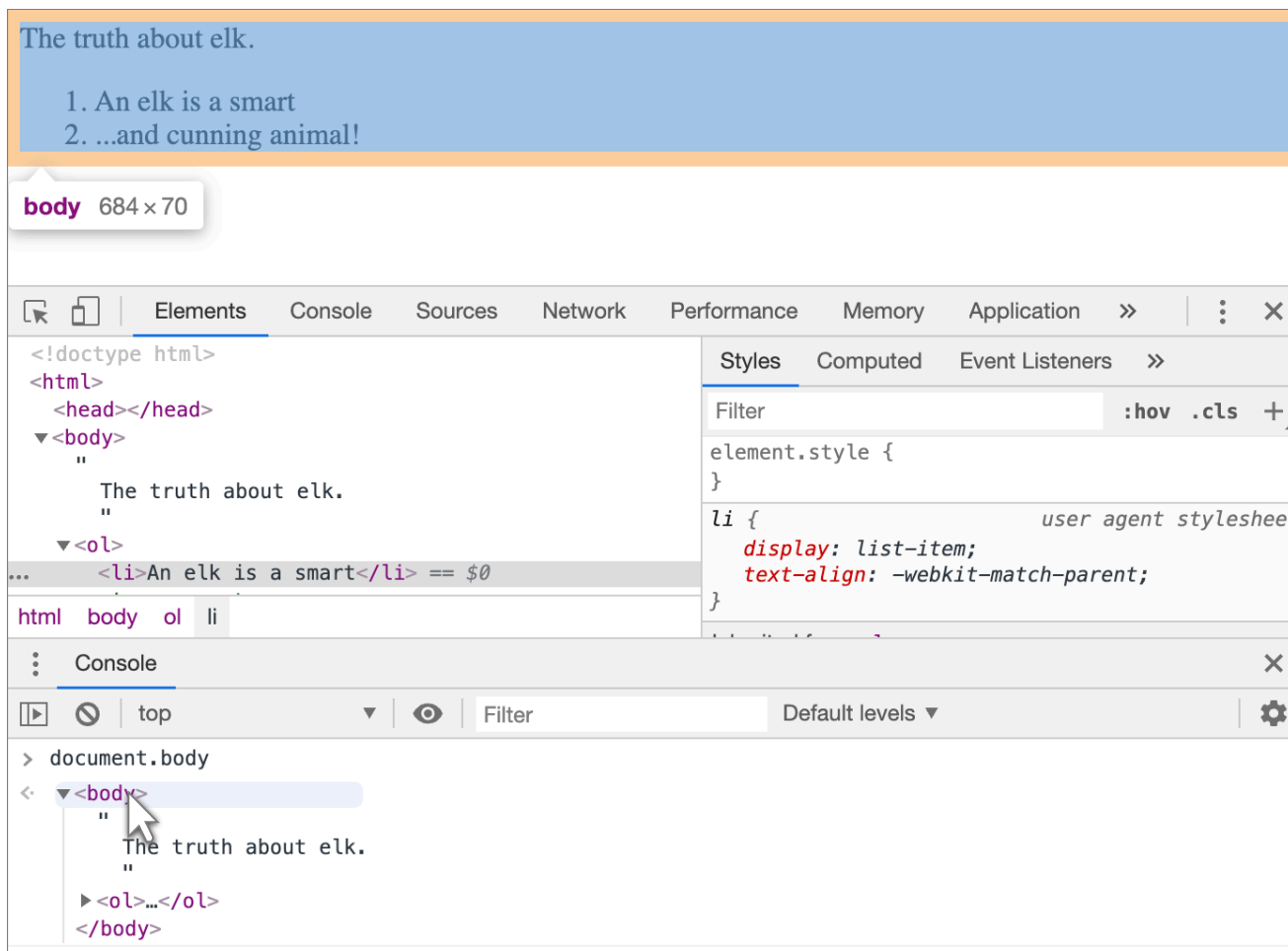
1. An elk is a smart
2. ...and cunning animal!



That's how to get a node from Elements in Console.

There's also a road back. If there's a variable referencing a DOM node, then we can use the command `inspect(node)` in Console to see it in the Elements pane.

Or we can just output the DOM node in the console and explore "in-place", like `document.body` below:



That's for debugging purposes of course. From the next chapter on we'll access and modify DOM using JavaScript.

The browser developer tools are a great help in development: we can explore the DOM, try things and see what goes wrong.

Summary

An HTML/XML document is represented inside the browser as the DOM tree.

- Tags become element nodes and form the structure.
- Text becomes text nodes.
- ...etc, everything in HTML has its place in DOM, even comments.

We can use developer tools to inspect DOM and modify it manually.

Here we covered the basics, the most used and important actions to start with.

There's an extensive documentation about Chrome Developer Tools at <https://developers.google.com/web/tools/chrome-devtools> [↗](#). The best way to learn the tools is to click here and there, read menus: most options are obvious. Later, when you know them in general, read the docs and pick up the rest.

DOM nodes have properties and methods that allow us to travel between them, modify them, move around the page, and more. We'll get down to them in the next

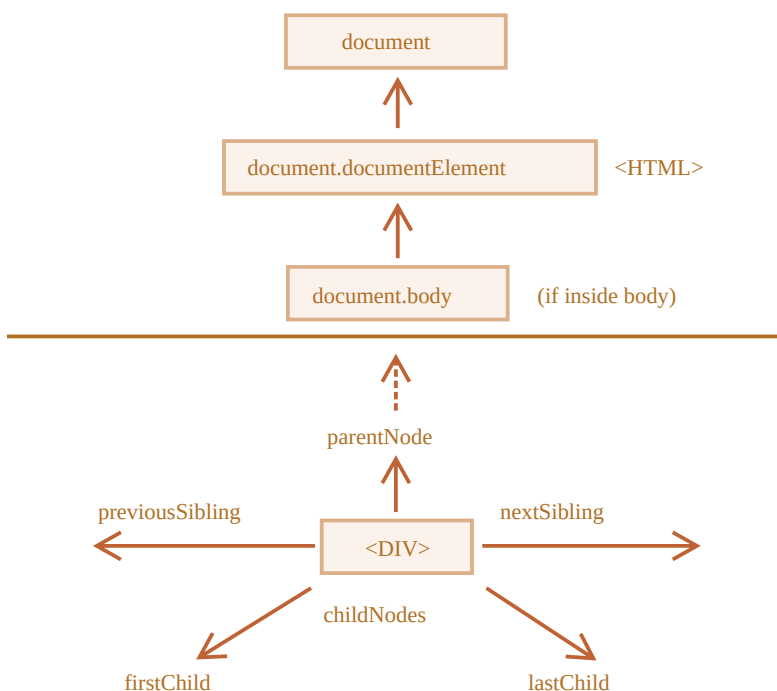
chapters.

Walking the DOM

The DOM allows us to do anything with elements and their contents, but first we need to reach the corresponding DOM object.

All operations on the DOM start with the `document` object. That's the main “entry point” to DOM. From it we can access any node.

Here's a picture of links that allow for travel between DOM nodes:



Let's discuss them in more detail.

On top: documentElement and body

The topmost tree nodes are available directly as `document` properties:

`<html> = document.documentElement`

The topmost document node is `document.documentElement`. That's the DOM node of the `<html>` tag.

`<body> = document.body`

Another widely used DOM node is the `<body>` element – `document.body`.

`<head> = document.head`

The `<head>` tag is available as `document.head`.

 There's a catch: `document.body` can be `null`

A script cannot access an element that doesn't exist at the moment of running.

In particular, if a script is inside `<head>`, then `document.body` is unavailable, because the browser did not read it yet.

So, in the example below the first `alert` shows `null`:

```
<html>

<head>
  <script>
    alert( "From HEAD: " + document.body ); // null, there's no <body> yet
  </script>
</head>

<body>

  <script>
    alert( "From BODY: " + document.body ); // HTMLBodyElement, now it exist
  </script>

</body>
</html>
```

 In the DOM world `null` means “doesn't exist”

In the DOM, the `null` value means “doesn't exist” or “no such node”.

Children: `childNodes`, `firstChild`, `lastChild`

There are two terms that we'll use from now on:

- **Child nodes (or children)** – elements that are direct children. In other words, they are nested exactly in the given one. For instance, `<head>` and `<body>` are children of `<html>` element.
- **Descendants** – all elements that are nested in the given one, including children, their children and so on.

For instance, here `<body>` has children `<div>` and `<ul>` (and few blank text nodes):

```
<html>
<body>
  <div>Begin</div>
```

```

<ul>
  <li>
    <b>Information</b>
  </li>
</ul>
</body>
</html>

```

...And descendants of `<body>` are not only direct children `<div>`, `<ul>` but also more deeply nested elements, such as `<li>` (a child of `<ul>`) and `<b>` (a child of `<li>`) – the entire subtree.

The **`childNodes`** collection lists all child nodes, including text nodes.

The example below shows children of `document.body`:

```

<html>
<body>
  <div>Begin</div>

  <ul>
    <li>Information</li>
  </ul>

  <div>End</div>

  <script>
    for (let i = 0; i < document.body.childNodes.length; i++) {
      alert( document.body.childNodes[i] ); // Text, DIV, Text, UL, ..., SCRIPT
    }
  </script>
  ...more stuff...
</body>
</html>

```

Please note an interesting detail here. If we run the example above, the last element shown is `<script>`. In fact, the document has more stuff below, but at the moment of the script execution the browser did not read it yet, so the script doesn't see it.

Properties **`firstChild`** and **`lastChild`** give fast access to the first and last children.

They are just shorthands. If there exist child nodes, then the following is always true:

```

elem.childNodes[0] === elem.firstChild
elem.childNodes[elem.childNodes.length - 1] === elem.lastChild

```

There's also a special function `elem.hasChildNodes()` to check whether there are any child nodes.

DOM collections

As we can see, `childNodes` looks like an array. But actually it's not an array, but rather a *collection* – a special array-like iterable object.

There are two important consequences:

1. We can use `for..of` to iterate over it:

```
for (let node of document.body.childNodes) {  
  alert(node); // shows all nodes from the collection  
}
```

That's because it's iterable (provides the `Symbol.iterator` property, as required).

2. Array methods won't work, because it's not an array:

```
alert(document.body.childNodes.filter); // undefined (there's no filter method!)
```

The first thing is nice. The second is tolerable, because we can use `Array.from` to create a “real” array from the collection, if we want array methods:

```
alert( Array.from(document.body.childNodes).filter ); // function
```

DOM collections are read-only

DOM collections, and even more – *all* navigation properties listed in this chapter are read-only.

We can't replace a child by something else by assigning `childNodes[i] = ...`.

Changing DOM needs other methods. We will see them in the next chapter.

DOM collections are live

Almost all DOM collections with minor exceptions are *live*. In other words, they reflect the current state of DOM.

If we keep a reference to `elem.childNodes`, and add/remove nodes into DOM, then they appear in the collection automatically.

⚠ Don't use `for..in` to loop over collections

Collections are iterable using `for..of`. Sometimes people try to use `for..in` for that.

Please, don't. The `for..in` loop iterates over all enumerable properties. And collections have some “extra” rarely used properties that we usually do not want to get:

```
<body>
<script>
  // shows 0, 1, length, item, values and more.
  for (let prop in document.body.childNodes) alert(prop);
</script>
</body>
```

Siblings and the parent

Siblings are nodes that are children of the same parent.

For instance, here `<head>` and `<body>` are siblings:

```
<html>
  <head>...</head><body>...</body>
</html>
```

- `<body>` is said to be the “next” or “right” sibling of `<head>`,
- `<head>` is said to be the “previous” or “left” sibling of `<body>`.

The next sibling is in `nextSibling` property, and the previous one – in `previousSibling`.

The parent is available as `parentNode`.

For example:

```
// parent of <body> is <html>
alert( document.body.parentNode === document.documentElement ); // true

// after <head> goes <body>
alert( document.head.nextSibling ); // HTMLBodyElement

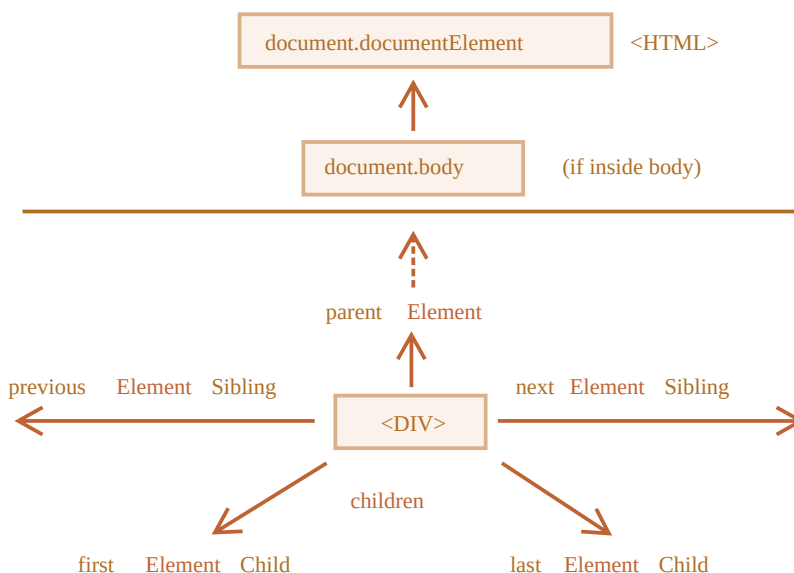
// before <body> goes <head>
alert( document.body.previousSibling ); // HTMLHeadElement
```

Element-only navigation

Navigation properties listed above refer to *all* nodes. For instance, in `childNodes` we can see both text nodes, element nodes, and even comment nodes if they exist.

But for many tasks we don't want text or comment nodes. We want to manipulate element nodes that represent tags and form the structure of the page.

So let's see more navigation links that only take *element nodes* into account:



The links are similar to those given above, just with `Element` word inside:

- `children` – only those children that are element nodes.
- `firstElementChild`, `lastElementChild` – first and last element children.
- `previousElementSibling`, `nextElementSibling` – neighbor elements.
- `parentElement` – parent element.

i Why `parentElement` ? Can the parent be *not* an element?

The `parentElement` property returns the “element” parent, while `parentNode` returns “any node” parent. These properties are usually the same: they both get the parent.

With the one exception of `document.documentElement` :

```
alert( document.documentElement.parentNode ); // document
alert( document.documentElement.parentElement ); // null
```

The reason is that the root node `document.documentElement` (`<html>`) has `document` as its parent. But `document` is not an element node, so `parentNode` returns it and `parentElement` does not.

This detail may be useful when we want to travel up from an arbitrary element `elem` to `<html>`, but not to the `document` :

```
while(elem = elem.parentElement) { // go up till <html>
  alert( elem );
}
```

Let’s modify one of the examples above: replace `childNodes` with `children`.
Now it shows only elements:

```
<html>
<body>
  <div>Begin</div>

  <ul>
    <li>Information</li>
  </ul>

  <div>End</div>

  <script>
    for (let elem of document.body.children) {
      alert(elem); // DIV, UL, DIV, SCRIPT
    }
  </script>
  ...
</body>
</html>
```

More links: tables

Till now we described the basic navigation properties.

Certain types of DOM elements may provide additional properties, specific to their type, for convenience.

Tables are a great example of that, and represent a particularly important case:

The `<table>` element supports (in addition to the given above) these properties:

- `table.rows` – the collection of `<tr>` elements of the table.
- `table.caption/tHead/tFoot` – references to elements `<caption>`, `<thead>`, `<tfoot>`.
- `table.tBodies` – the collection of `<tbody>` elements (can be many according to the standard, but there will always be at least one – even if it is not in the source HTML, the browser will put it in the DOM).

`<thead>`, `<tfoot>`, `<tbody>` elements provide the `rows` property:

- `tbody.rows` – the collection of `<tr>` inside.

`<tr>`:

- `tr.cells` – the collection of `<td>` and `<th>` cells inside the given `<tr>`.
- `tr.sectionRowIndex` – the position (index) of the given `<tr>` inside the enclosing `<thead>/<tbody>/<tfoot>`.
- `tr.rowIndex` – the number of the `<tr>` in the table as a whole (including all table rows).

`<td>` and `<th>`:

- `td.cellIndex` – the number of the cell inside the enclosing `<tr>`.

An example of usage:

```
<table id="table">
  <tr>
    <td>one</td><td>two</td>
  </tr>
  <tr>
    <td>three</td><td>four</td>
  </tr>
</table>

<script>
  // get td with "two" (first row, second column)
  let td = table.rows[0].cells[1];
```

```
td.style.backgroundColor = "red"; // highlight it
</script>
```

The specification: [tabular data](#) .

There are also additional navigation properties for HTML forms. We'll look at them later when we start working with forms.

Summary

Given a DOM node, we can go to its immediate neighbors using navigation properties.

There are two main sets of them:

- For all nodes: `parentNode`, `childNodes`, `firstChild`, `lastChild`, `previousSibling`, `nextSibling`.
- For element nodes only: `parentElement`, `children`, `firstElementChild`, `lastElementChild`, `previousElementSibling`, `nextElementSibling`.

Some types of DOM elements, e.g. tables, provide additional properties and collections to access their content.

✓ Tasks

DOM children

importance: 5

Look at this page:

```
<html>
<body>
  <div>Users:</div>
  <ul>
    <li>John</li>
    <li>Pete</li>
  </ul>
</body>
</html>
```

For each of the following, give at least one way of how to access them:

- The `<div>` DOM node?
- The `<ul>` DOM node?
- The second `<li>` (with Pete)?

[To solution](#)

The sibling question

importance: 5

If `elem` – is an arbitrary DOM element node...

- Is it true that `elem.lastChild.nextSibling` is always `null`?
- Is it true that `elem.children[0].previousSibling` is always `null`?

[To solution](#)

Select all diagonal cells

importance: 5

Write the code to paint all diagonal table cells in red.

You'll need to get all diagonal `<td>` from the `<table>` and paint them using the code:

```
// td should be the reference to the table cell
td.style.backgroundColor = 'red';
```

The result should be:

1:1	2:1	3:1	4:1	5:1
1:2	2:2	3:2	4:2	5:2
1:3	2:3	3:3	4:3	5:3
1:4	2:4	3:4	4:4	5:4
1:5	2:5	3:5	4:5	5:5

[Open a sandbox for the task.](#) ↗

[To solution](#)

Searching: `getElement*`, `querySelector*`

DOM navigation properties are great when elements are close to each other. What if they are not? How to get an arbitrary element of the page?

There are additional searching methods for that.

document.getElementById or just id

If an element has the `id` attribute, we can get the element using the method `document.getElementById(id)`, no matter where it is.

For instance:

```
<div id="elem">
  <div id="elem-content">Element</div>
</div>

<script>
  // get the element
  let elem = document.getElementById('elem');

  // make its background red
  elem.style.background = 'red';
</script>
```

Also, there's a global variable named by `id` that references the element:

```
<div id="elem">
  <div id="elem-content">Element</div>
</div>

<script>
  // elem is a reference to DOM-element with id="elem"
  elem.style.background = 'red';

  // id="elem-content" has a hyphen inside, so it can't be a variable name
  // ...but we can access it using square brackets: window['elem-content']
</script>
```

...That's unless we declare a JavaScript variable with the same name, then it takes precedence:

```
<div id="elem"></div>

<script>
  let elem = 5; // now elem is 5, not a reference to <div id="elem">

  alert(elem); // 5
</script>
```

 Please don't use id-named global variables to access elements

This behavior is described [in the specification](#) , but it is supported mainly for compatibility.

The browser tries to help us by mixing namespaces of JS and DOM. That's fine for simple scripts, inlined into HTML, but generally isn't a good thing. There may be naming conflicts. Also, when one reads JS code and doesn't have HTML in view, it's not obvious where the variable comes from.

Here in the tutorial we use `id` to directly reference an element for brevity, when it's obvious where the element comes from.

In real life `document.getElementById` is the preferred method.

 The `id` must be unique

The `id` must be unique. There can be only one element in the document with the given `id`.

If there are multiple elements with the same `id`, then the behavior of methods that use it is unpredictable, e.g. `document.getElementById` may return any of such elements at random. So please stick to the rule and keep `id` unique.

 Only `document.getElementById`, not `anyElem.getElementById`

The method `getElementById` can be called only on `document` object. It looks for the given `id` in the whole document.

querySelectorAll

By far, the most versatile method, `elem.querySelectorAll(css)` returns all elements inside `elem` matching the given CSS selector.

Here we look for all `<li>` elements that are last children:

```
<ul>
  <li>The</li>
  <li>test</li>
</ul>
<ul>
  <li>has</li>
  <li>passed</li>
</ul>
<script>
  let elements = document.querySelectorAll('ul > li:last-child');

  for (let elem of elements) {
```

```
    alert(elem.innerHTML); // "test", "passed"
  }
</script>
```

This method is indeed powerful, because any CSS selector can be used.

i Can use pseudo-classes as well

Pseudo-classes in the CSS selector like `:hover` and `:active` are also supported. For instance, `document.querySelectorAll(':hover')` will return the collection with elements that the pointer is over now (in nesting order: from the outermost `<html>` to the most nested one).

querySelector

The call to `elem.querySelector(css)` returns the first element for the given CSS selector.

In other words, the result is the same as `elem.querySelectorAll(css)[0]`, but the latter is looking for *all* elements and picking one, while `elem.querySelector` just looks for one. So it's faster and also shorter to write.

matches

Previous methods were searching the DOM.

The `elem.matches(css)` [↗](#) does not look for anything, it merely checks if `elem` matches the given CSS-selector. It returns `true` or `false`.

The method comes in handy when we are iterating over elements (like in an array or something) and trying to filter out those that interest us.

For instance:

```
<a href="http://example.com/file.zip">...</a>
<a href="http://ya.ru">...</a>

<script>
  // can be any collection instead of document.body.children
  for (let elem of document.body.children) {
    if (elem.matches('a[href$="zip"]')) {
      alert("The archive reference: " + elem.href );
    }
  }
</script>
```

closest

Ancestors of an element are: parent, the parent of parent, its parent and so on. The ancestors together form the chain of parents from the element to the top.

The method `elem.closest(css)` looks for the nearest ancestor that matches the CSS-selector. The `elem` itself is also included in the search.

In other words, the method `closest` goes up from the element and checks each of parents. If it matches the selector, then the search stops, and the ancestor is returned.

For instance:

```
<h1>Contents</h1>

<div class="contents">
  <ul class="book">
    <li class="chapter">Chapter 1</li>
    <li class="chapter">Chapter 2</li>
  </ul>
</div>

<script>
  let chapter = document.querySelector('.chapter'); // LI

  alert(chapter.closest('.book')); // UL
  alert(chapter.closest('.contents')); // DIV

  alert(chapter.closest('h1')); // null (because h1 is not an ancestor)
</script>
```

getElementsBy*

There are also other methods to look for nodes by a tag, class, etc.

Today, they are mostly history, as `querySelector` is more powerful and shorter to write.

So here we cover them mainly for completeness, while you can still find them in the old scripts.

- `elem.getElementsByTagName(tag)` looks for elements with the given tag and returns the collection of them. The `tag` parameter can also be a star `"*"` for “any tags”.
- `elem.getElementsByClassName(className)` returns elements that have the given CSS class.
- `document.getElementsByName(name)` returns elements with the given `name` attribute, document-wide. Very rarely used.

For instance:

```
// get all divs in the document
let divs = document.getElementsByTagName('div');
```

Let's find all `input` tags inside the table:

```
<table id="table">
  <tr>
    <td>Your age:</td>

    <td>
      <label>
        <input type="radio" name="age" value="young" checked> less than 18
      </label>
      <label>
        <input type="radio" name="age" value="mature"> from 18 to 50
      </label>
      <label>
        <input type="radio" name="age" value="senior"> more than 60
      </label>
    </td>
  </tr>
</table>

<script>
  let inputs = table.getElementsByTagName('input');

  for (let input of inputs) {
    alert( input.value + ': ' + input.checked );
  }
</script>
```

 Don't forget the "s" letter!

Novice developers sometimes forget the letter "s". That is, they try to call `getElementByTagName` instead of `getElementsByTagName`.

The "s" letter is absent in `getElementById`, because it returns a single element. But `getElementsByTagName` returns a collection of elements, so there's "s" inside.

⚠ It returns a collection, not an element!

Another widespread novice mistake is to write:

```
// doesn't work
document.getElementsByTagName('input').value = 5;
```

That won't work, because it takes a *collection* of inputs and assigns the value to it rather than to elements inside it.

We should either iterate over the collection or get an element by its index, and then assign, like this:

```
// should work (if there's an input)
document.getElementsByTagName('input')[0].value = 5;
```

Looking for `.article` elements:

```
<form name="my-form">
  <div class="article">Article</div>
  <div class="long article">Long article</div>
</form>

<script>
  // find by name attribute
  let form = document.getElementsByTagName('my-form')[0];

  // find by class inside the form
  let articles = form.getElementsByClassName('article');
  alert(articles.length); // 2, found two elements with class "article"
</script>
```

Live collections

All methods `"getElementsBy*"` return a *live* collection. Such collections always reflect the current state of the document and “auto-update” when it changes.

In the example below, there are two scripts.

1. The first one creates a reference to the collection of `<div>`. As of now, its length is `1`.
2. The second script runs after the browser meets one more `<div>`, so its length is `2`.

```

<div>First div</div>

<script>
  let divs = document.getElementsByTagName('div');
  alert(divs.length); // 1
</script>

<div>Second div</div>

<script>
  alert(divs.length); // 2
</script>

```

In contrast, `querySelectorAll` returns a *static* collection. It's like a fixed array of elements.

If we use it instead, then both scripts output `1`:

```

<div>First div</div>

<script>
  let divs = document.querySelectorAll('div');
  alert(divs.length); // 1
</script>

<div>Second div</div>

<script>
  alert(divs.length); // 1
</script>

```

Now we can easily see the difference. The static collection did not increase after the appearance of a new `div` in the document.

Summary

There are 6 main methods to search for nodes in DOM:

Method	Searches by...	Can call on an element?	Live?
<code>querySelector</code>	CSS-selector	✓	-
<code>querySelectorAll</code>	CSS-selector	✓	-
<code>getElementById</code>	<code>id</code>	-	-
<code>getElementsByName</code>	name	-	✓
<code>getElementsByTagName</code>	tag or <code>'*'</code>	✓	✓
<code>getElementsByClassName</code>	class	✓	✓

By far the most used are `querySelector` and `querySelectorAll`, but `getElement(s)By*` can be sporadically helpful or found in the old scripts.

Besides that:

- There is `elem.matches(css)` to check if `elem` matches the given CSS selector.
- There is `elem.closest(css)` to look for the nearest ancestor that matches the given CSS-selector. The `elem` itself is also checked.

And let's mention one more method here to check for the child-parent relationship, as it's sometimes useful:

- `elemA.contains(elemB)` returns true if `elemB` is inside `elemA` (a descendant of `elemA`) or when `elemA==elemB`.

✓ Tasks

Search for elements

importance: 4

Here's the document with the table and form.

How to find?...

1. The table with `id="age-table"`.
2. All `label` elements inside that table (there should be 3 of them).
3. The first `td` in that table (with the word "Age").
4. The `form` with `name="search"`.
5. The first `input` in that form.
6. The last `input` in that form.

Open the page [table.html](#) in a separate window and make use of browser tools for that.

[To solution](#)

Node properties: type, tag and contents

Let's get a more in-depth look at DOM nodes.

In this chapter we'll see more into what they are and learn their most used properties.

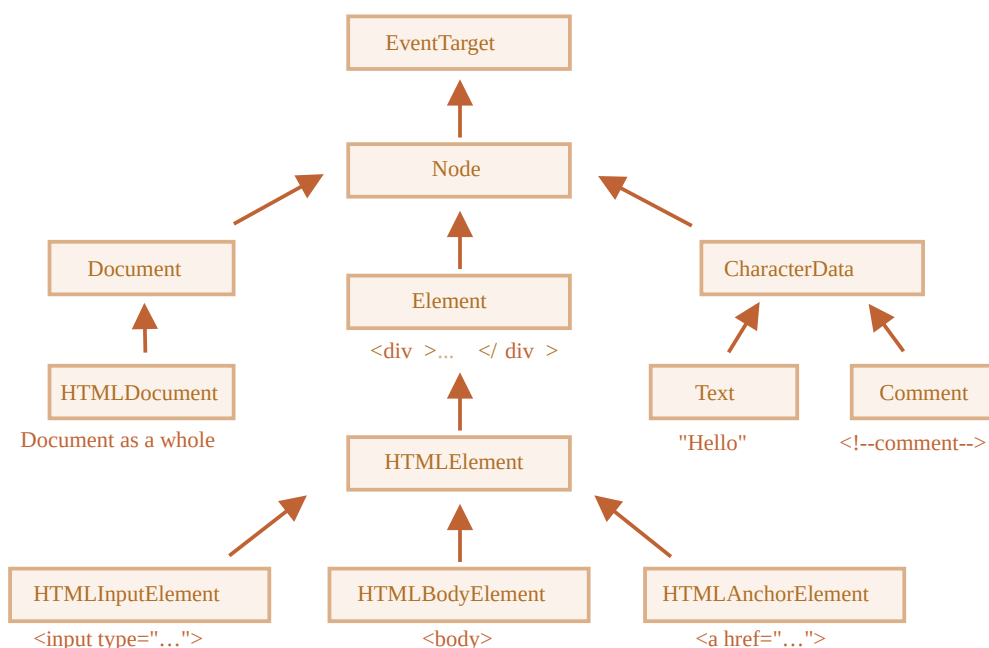
DOM node classes

Different DOM nodes may have different properties. For instance, an element node corresponding to tag `<a>` has link-related properties, and the one corresponding to `<input>` has input-related properties and so on. Text nodes are not the same as element nodes. But there are also common properties and methods between all of them, because all classes of DOM nodes form a single hierarchy.

Each DOM node belongs to the corresponding built-in class.

The root of the hierarchy is [EventTarget](#), that is inherited by [Node](#), and other DOM nodes inherit from it.

Here's the picture, explanations to follow:



The classes are:

- [EventTarget](#) – is the root “abstract” class for everything.

Objects of that class are never created. It serves as a base, so that all DOM nodes support so-called “events”, we’ll study them later.

- [Node](#) – is also an “abstract” class, serving as a base for DOM nodes.

It provides the core tree functionality: `parentNode`, `nextSibling`, `childNodes` and so on (they are getters). Objects of `Node` class are never created. But there are other classes that inherit from it (and so inherit the `Node` functionality).

- [Document](#), for historical reasons often inherited by `HTMLDocument` (though the latest spec doesn’t dictate it) – is a document as a whole.

The `document` global object belongs exactly to this class. It serves as an entry point to the DOM.

- [CharacterData](#) – an “abstract” class, inherited by:
 - [Text](#) – the class corresponding to a text inside elements, e.g. `Hello` in `<p>Hello</p>`.
 - [Comment](#) – the class for comments. They are not shown, but each comment becomes a member of DOM.
- [Element](#) – is the base class for DOM elements.

It provides element-level navigation like `nextElementSibling`, `children` and searching methods like `getElementsByTagName`, `querySelector`.

A browser supports not only HTML, but also XML and SVG. So the `Element` class serves as a base for more specific classes: `SVGElement`, `XMLElement` (we don't need them here) and `HTMLElement`.

- Finally, [HTMLElement](#) is the basic class for all HTML elements. We'll work with it most of the time.

It is inherited by concrete HTML elements:

- [HTMLInputElement](#) – the class for `<input>` elements,
- [HTMLBodyElement](#) – the class for `<body>` elements,
- [HTMLAnchorElement](#) – the class for `<a>` elements,
- ...and so on.

There are many other tags with their own classes that may have specific properties and methods, while some elements, such as `<span>`, `<section>`, `<article>` do not have any specific properties, so they are instances of `HTMLElement` class.

So, the full set of properties and methods of a given node comes as the result of the chain of inheritance.

For example, let's consider the DOM object for an `<input>` element. It belongs to [HTMLInputElement](#) class.

It gets properties and methods as a superposition of (listed in inheritance order):

- `HTMLInputElement` – this class provides input-specific properties,
- `HTMLElement` – it provides common HTML element methods (and getters/setters),
- `Element` – provides generic element methods,
- `Node` – provides common DOM node properties,
- `EventTarget` – gives the support for events (to be covered),
- ...and finally it inherits from `Object`, so “plain object” methods like `hasOwnProperty` are also available.

To see the DOM node class name, we can recall that an object usually has the `constructor` property. It references the class constructor, and `constructor.name` is its name:

```
alert( document.body.constructor.name ); // HTMLBodyElement
```

...Or we can just `toString` it:

```
alert( document.body ); // [object HTMLBodyElement]
```

We also can use `instanceof` to check the inheritance:

```
alert( document.body instanceof HTMLBodyElement ); // true
alert( document.body instanceof HTMLElement ); // true
alert( document.body instanceof Element ); // true
alert( document.body instanceof Node ); // true
alert( document.body instanceof EventTarget ); // true
```

As we can see, DOM nodes are regular JavaScript objects. They use prototype-based classes for inheritance.

That's also easy to see by outputting an element with `console.dir(elem)` in a browser. There in the console you can see `HTMLElement.prototype`, `Element.prototype` and so on.

`console.dir(elem)` versus `console.log(elem)`

Most browsers support two commands in their developer tools: `console.log` and `console.dir`. They output their arguments to the console. For JavaScript objects these commands usually do the same.

But for DOM elements they are different:

- `console.log(elem)` shows the element DOM tree.
- `console.dir(elem)` shows the element as a DOM object, good to explore its properties.

Try it on `document.body`.

IDL in the spec

In the specification, DOM classes aren't described by using JavaScript, but a special [Interface description language](#) (IDL), that is usually easy to understand.

In IDL all properties are prepended with their types. For instance, `DOMString`, `boolean` and so on.

Here's an excerpt from it, with comments:

```
// Define HTMLInputElement
// The colon ":" means that HTMLInputElement inherits from HTMLElement
interface HTMLInputElement: HTMLElement {
  // here go properties and methods of <input> elements

  // "DOMString" means that the value of a property is a string
  attribute DOMString accept;
  attribute DOMString alt;
  attribute DOMString autocomplete;
  attribute DOMString value;

  // boolean value property (true/false)
  attribute boolean autofocus;
  ...
  // now the method: "void" means that the method returns no value
  void select();
  ...
}
```

The “nodeType” property

The `nodeType` property provides one more, “old-fashioned” way to get the “type” of a DOM node.

It has a numeric value:

- `elem.nodeType == 1` for element nodes,
- `elem.nodeType == 3` for text nodes,
- `elem.nodeType == 9` for the document object,
- there are few other values in [the specification](#) .

For instance:

```
<body>
<script>
let elem = document.body;
```

```
// let's examine: what type of node is in elem?
alert(elem.nodeType); // 1 => element

// and its first child is...
alert(elem.firstChild.nodeType); // 3 => text

// for the document object, the type is 9
alert( document.nodeType ); // 9
</script>
</body>
```

In modern scripts, we can use `instanceof` and other class-based tests to see the node type, but sometimes `nodeType` may be simpler. We can only read `nodeType`, not change it.

Tag: nodeName and tagName

Given a DOM node, we can read its tag name from `nodeName` or `tagName` properties:

For instance:

```
alert( document.body.nodeName ); // BODY
alert( document.body.tagName ); // BODY
```

Is there any difference between `tagName` and `nodeName`?

Sure, the difference is reflected in their names, but is indeed a bit subtle.

- The `tagName` property exists only for `Element` nodes.
- The `nodeName` is defined for any `Node`:
 - for elements it means the same as `tagName`.
 - for other node types (text, comment, etc.) it has a string with the node type.

In other words, `tagName` is only supported by element nodes (as it originates from `Element` class), while `nodeName` can say something about other node types.

For instance, let's compare `tagName` and `nodeName` for the `document` and a comment node:

```
<body><!-- comment -->

<script>
  // for comment
  alert( document.body.firstChild.tagName ); // undefined (not an element)
  alert( document.body.firstChild.nodeName ); // #comment
```

```
// for document
alert( document.tagName ); // undefined (not an element)
alert( document.nodeName ); // #document
</script>
</body>
```

If we only deal with elements, then we can use both `tagName` and `nodeName` – there's no difference.

i The tag name is always uppercase except in XML mode

The browser has two modes of processing documents: HTML and XML. Usually the HTML-mode is used for webpages. XML-mode is enabled when the browser receives an XML-document with the header: `Content-Type: application/xml+xhtml`.

In HTML mode `tagName/nodeName` is always uppercased: it's `BODY` either for `<body>` or `<BoDy>`.

In XML mode the case is kept "as is". Nowadays XML mode is rarely used.

innerHTML: the contents

The `innerHTML` [↗](#) property allows to get the HTML inside the element as a string.

We can also modify it. So it's one of the most powerful ways to change the page.

The example shows the contents of `document.body` and then replaces it completely:

```
<body>
  <p>A paragraph</p>
  <div>A div</div>

  <script>
    alert( document.body.innerHTML ); // read the current contents
    document.body.innerHTML = 'The new BODY!'; // replace it
  </script>

</body>
```

We can try to insert invalid HTML, the browser will fix our errors:

```
<body>

  <script>
    document.body.innerHTML = '<b>test'; // forgot to close the tag
    alert( document.body.innerHTML ); // <b>test</b> (fixed)
```

```
</script>

</body>
```

i Scripts don't execute

If `innerHTML` inserts a `<script>` tag into the document – it becomes a part of HTML, but doesn't execute.

Beware: “innerHTML+=” does a full overwrite

We can append HTML to an element by using `elem.innerHTML+="more html"`.

Like this:

```
chatDiv.innerHTML += "<div>Hello<img src='smile.gif'/> !</div>";
chatDiv.innerHTML += "How goes?";
```

But we should be very careful about doing it, because what's going on is *not* an addition, but a full overwrite.

Technically, these two lines do the same:

```
elem.innerHTML += "...";
// is a shorter way to write:
elem.innerHTML = elem.innerHTML + "..."
```

In other words, `innerHTML+=` does this:

1. The old contents is removed.
2. The new `innerHTML` is written instead (a concatenation of the old and the new one).

As the content is “zeroed-out” and rewritten from the scratch, all images and other resources will be reloaded.

In the `chatDiv` example above the line `chatDiv.innerHTML+="How goes?"` re-creates the HTML content and reloads `smile.gif` (hope it's cached). If `chatDiv` has a lot of other text and images, then the reload becomes clearly visible.

There are other side-effects as well. For instance, if the existing text was selected with the mouse, then most browsers will remove the selection upon rewriting `innerHTML`. And if there was an `<input>` with a text entered by the visitor, then the text will be removed. And so on.

Luckily, there are other ways to add HTML besides `innerHTML`, and we'll study them soon.

outerHTML: full HTML of the element

The `outerHTML` property contains the full HTML of the element. That's like `innerHTML` plus the element itself.

Here's an example:

```
<div id="elem">Hello <b>World</b></div>

<script>
  alert(elem.outerHTML); // <div id="elem">Hello <b>World</b></div>
</script>
```

Beware: unlike `innerHTML`, writing to `outerHTML` does not change the element. Instead, it replaces it in the DOM.

Yeah, sounds strange, and strange it is, that's why we make a separate note about it here. Take a look.

Consider the example:

```
<div>Hello, world!</div>

<script>
  let div = document.querySelector('div');

  // replace div.outerHTML with <p>...</p>
  div.outerHTML = '<p>A new element</p>'; // (*)

  // Wow! 'div' is still the same!
  alert(div.outerHTML); // <div>Hello, world!</div> (**)
</script>
```

Looks really odd, right?

In the line `(*)` we replaced `div` with `<p>A new element</p>`. In the outer document (the DOM) we can see the new content instead of the `<div>`. But, as we can see in line `(**)`, the value of the old `div` variable hasn't changed!

The `outerHTML` assignment does not modify the DOM element (the object referenced by, in this case, the variable 'div'), but removes it from the DOM and inserts the new HTML in its place.

So what happened in `div.outerHTML=...` is:

- `div` was removed from the document.
- Another piece of HTML `<p>A new element</p>` was inserted in its place.
- `div` still has its old value. The new HTML wasn't saved to any variable.

It's so easy to make an error here: modify `div.outerHTML` and then continue to work with `div` as if it had the new content in it. But it doesn't. Such thing is correct for `innerHTML`, but not for `outerHTML`.

We can write to `elem.outerHTML`, but should keep in mind that it doesn't change the element we're writing to ('elem'). It puts the new HTML in its place instead. We can get references to the new elements by querying the DOM.

nodeValue/data: text node content

The `innerHTML` property is only valid for element nodes.

Other node types, such as text nodes, have their counterpart: `nodeValue` and `data` properties. These two are almost the same for practical use, there are only minor specification differences. So we'll use `data`, because it's shorter.

An example of reading the content of a text node and a comment:

```
<body>
  Hello
  <!-- Comment -->
  <script>
    let text = document.body.firstChild;
    alert(text.data); // Hello

    let comment = text.nextSibling;
    alert(comment.data); // Comment
  </script>
</body>
```

For text nodes we can imagine a reason to read or modify them, but why comments?

Sometimes developers embed information or template instructions into HTML in them, like this:

```
<!-- if isAdmin -->
  <div>Welcome, Admin!</div>
<!-- /if -->
```

...Then JavaScript can read it from `data` property and process embedded instructions.

textContent: pure text

The `textContent` provides access to the *text* inside the element: only text, minus all `<tags>`.

For instance:

```
<div id="news">
  <h1>Headline!</h1>
  <p>Martians attack people!</p>
</div>

<script>
  // Headline! Martians attack people!
  alert(news.textContent);
</script>
```

As we can see, only text is returned, as if all `<tags>` were cut out, but the text in them remained.

In practice, reading such text is rarely needed.

Writing to `textContent` is much more useful, because it allows to write text the “safe way”.

Let's say we have an arbitrary string, for instance entered by a user, and want to show it.

- With `innerHTML` we'll have it inserted “as HTML”, with all HTML tags.
- With `textContent` we'll have it inserted “as text”, all symbols are treated literally.

Compare the two:

```
<div id="elem1"></div>
<div id="elem2"></div>

<script>
  let name = prompt("What's your name?", "<b>Winnie-the-Pooh!</b>");

  elem1.innerHTML = name;
  elem2.textContent = name;
</script>
```

1. The first `<div>` gets the name “as HTML”: all tags become tags, so we see the bold name.
2. The second `<div>` gets the name “as text”, so we literally see `<b>Winnie-the-Pooh!</b>`.

In most cases, we expect the text from a user, and want to treat it as text. We don't want unexpected HTML in our site. An assignment to `textContent` does exactly that.

The “hidden” property

The “hidden” attribute and the DOM property specifies whether the element is visible or not.

We can use it in HTML or assign it using JavaScript, like this:

```
<div>Both divs below are hidden</div>

<div hidden>With the attribute "hidden"</div>

<div id="elem">JavaScript assigned the property "hidden"</div>

<script>
  elem.hidden = true;
</script>
```

Technically, `hidden` works the same as `style="display:none"`. But it's shorter to write.

Here's a blinking element:

```
<div id="elem">A blinking element</div>

<script>
  setInterval(() => elem.hidden = !elem.hidden, 1000);
</script>
```

More properties

DOM elements also have additional properties, in particular those that depend on the class:

- `value` – the value for `<input>`, `<select>` and `<textarea>` (`HTMLInputElement`, `HTMLSelectElement` ...).
- `href` – the “href” for `<a href="...">` (`HTMLAnchorElement`).
- `id` – the value of “id” attribute, for all elements (`HTMLElement`).
- ...and much more...

For instance:

```
<input type="text" id="elem" value="value">

<script>
  alert(elem.type); // "text"
  alert(elem.id); // "elem"
```

```
    alert(elem.value); // value
</script>
```

Most standard HTML attributes have the corresponding DOM property, and we can access it like that.

If we want to know the full list of supported properties for a given class, we can find them in the specification. For instance, `HTMLInputElement` is documented at <https://html.spec.whatwg.org/#htmlinputelement> .

Or if we'd like to get them fast or are interested in a concrete browser specification – we can always output the element using `console.dir(elem)` and read the properties. Or explore “DOM properties” in the Elements tab of the browser developer tools.

Summary

Each DOM node belongs to a certain class. The classes form a hierarchy. The full set of properties and methods come as the result of inheritance.

Main DOM node properties are:

`nodeType`

We can use it to see if a node is a text or an element node. It has a numeric value: `1` for elements, `3` for text nodes, and a few others for other node types. Read-only.

`nodeName/tagName`

For elements, tag name (uppercased unless XML-mode). For non-element nodes `nodeName` describes what it is. Read-only.

`innerHTML`

The HTML content of the element. Can be modified.

`outerHTML`

The full HTML of the element. A write operation into `elem.outerHTML` does not touch `elem` itself. Instead it gets replaced with the new HTML in the outer context.

`nodeValue/data`

The content of a non-element node (text, comment). These two are almost the same, usually we use `data` . Can be modified.

`textContent`

The text inside the element: HTML minus all `<tags>` . Writing into it puts the text inside the element, with all special characters and tags treated exactly as text. Can safely insert user-generated text and protect from unwanted HTML insertions.

hidden

When set to `true`, does the same as CSS `display:none`.

DOM nodes also have other properties depending on their class. For instance, `<input>` elements (`HTMLInputElement`) support `value`, `type`, while `<a>` elements (`HTMLAnchorElement`) support `href` etc. Most standard HTML attributes have a corresponding DOM property.

However, HTML attributes and DOM properties are not always the same, as we'll see in the next chapter.

✓ Tasks

Count descendants

importance: 5

There's a tree structured as nested `ul/li`.

Write the code that for each `<li>` shows:

1. What's the text inside it (without the subtree)
2. The number of nested `<li>` – all descendants, including the deeply nested ones.

[Demo in new window](#) ↗

[Open a sandbox for the task.](#) ↗

[To solution](#)

What's in the nodeType?

importance: 5

What does the script show?

```
<html>

<body>
  <script>
    alert(document.body.lastChild.nodeType);
  </script>
</body>

</html>
```

[To solution](#)

Tag in comment

importance: 3

What does this code show?

```
<script>
  let body = document.body;

  body.innerHTML = "<!--" + body.tagName + "-->";

  alert( body.firstChild.data ); // what's here?
</script>
```

[To solution](#)

Where's the "document" in the hierarchy?

importance: 4

Which class does the `document` belong to?

What's its place in the DOM hierarchy?

Does it inherit from `Node` or `Element`, or maybe `HTMLElement`?

[To solution](#)

Attributes and properties

When the browser loads the page, it “reads” (another word: “parses”) the HTML and generates DOM objects from it. For element nodes, most standard HTML attributes automatically become properties of DOM objects.

For instance, if the tag is `<body id="page">`, then the DOM object has `body.id="page"`.

But the attribute-property mapping is not one-to-one! In this chapter we'll pay attention to separate these two notions, to see how to work with them, when they are the same, and when they are different.

DOM properties

We've already seen built-in DOM properties. There are a lot. But technically no one limits us, and if there aren't enough, we can add our own.

DOM nodes are regular JavaScript objects. We can alter them.

For instance, let's create a new property in `document.body`:

```
document.body.myData = {
  name: 'Caesar',
  title: 'Imperator'
};

alert(document.body.myData.title); // Imperator
```

We can add a method as well:

```
document.body.sayTagName = function() {
  alert(this.tagName);
};

document.body.sayTagName(); // BODY (the value of "this" in the method is document.body)
```

We can also modify built-in prototypes like `Element.prototype` and add new methods to all elements:

```
Element.prototype.sayHi = function() {
  alert(`Hello, I'm ${this.tagName}`);
};

document.documentElement.sayHi(); // Hello, I'm HTML
document.body.sayHi(); // Hello, I'm BODY
```

So, DOM properties and methods behave just like those of regular JavaScript objects:

- They can have any value.
- They are case-sensitive (write `elem.nodeType`, not `elem.NoDeTyPe`).

HTML attributes

In HTML, tags may have attributes. When the browser parses the HTML to create DOM objects for tags, it recognizes *standard* attributes and creates DOM properties from them.

So when an element has `id` or another *standard* attribute, the corresponding property gets created. But that doesn't happen if the attribute is non-standard.

For instance:

```
<body id="test" something="non-standard">
  <script>
    alert(document.body.id); // test
    // non-standard attribute does not yield a property
    alert(document.body.something); // undefined
  </script>
</body>
```

Please note that a standard attribute for one element can be unknown for another one. For instance, `"type"` is standard for `<input>` ([HTMLInputElement](#)), but not for `<body>` ([HTMLBodyElement](#)). Standard attributes are described in the specification for the corresponding element class.

Here we can see it:

```
<body id="body" type="...">
  <input id="input" type="text">
  <script>
    alert(input.type); // text
    alert(body.type); // undefined: DOM property not created, because it's non-s
  </script>
</body>
```

So, if an attribute is non-standard, there won't be a DOM-property for it. Is there a way to access such attributes?

Sure. All attributes are accessible by using the following methods:

- `elem.hasAttribute(name)` – checks for existence.
- `elem.getAttribute(name)` – gets the value.
- `elem.setAttribute(name, value)` – sets the value.
- `elem.removeAttribute(name)` – removes the attribute.

These methods operate exactly with what's written in HTML.

Also one can read all attributes using `elem.attributes`: a collection of objects that belong to a built-in [Attr](#) class, with `name` and `value` properties.

Here's a demo of reading a non-standard property:

```
<body something="non-standard">
  <script>
```

```
    alert(document.body.getAttribute('something')); // non-standard
  </script>
</body>
```

HTML attributes have the following features:

- Their name is case-insensitive (`id` is same as `ID`).
- Their values are always strings.

Here's an extended demo of working with attributes:

```
<body>
  <div id="elem" about="Elephant"></div>

  <script>
    alert( elem.getAttribute('About') ); // (1) 'Elephant', reading

    elem.setAttribute('Test', 123); // (2), writing

    alert( elem.outerHTML ); // (3), see if the attribute is in HTML (yes)

    for (let attr of elem.attributes) { // (4) list all
      alert( `${attr.name} = ${attr.value}` );
    }
  </script>
</body>
```

Please note:

1. `getAttribute('About')` – the first letter is uppercase here, and in HTML it's all lowercase. But that doesn't matter: attribute names are case-insensitive.
2. We can assign anything to an attribute, but it becomes a string. So here we have `"123"` as the value.
3. All attributes including ones that we set are visible in `outerHTML`.
4. The `attributes` collection is iterable and has all the attributes of the element (standard and non-standard) as objects with `name` and `value` properties.

Property-attribute synchronization

When a standard attribute changes, the corresponding property is auto-updated, and (with some exceptions) vice versa.

In the example below `id` is modified as an attribute, and we can see the property changed too. And then the same backwards:

```
<input>
```

```

<script>
  let input = document.querySelector('input');

  // attribute => property
  input.setAttribute('id', 'id');
  alert(input.id); // id (updated)

  // property => attribute
  input.id = 'newId';
  alert(input.getAttribute('id')); // newId (updated)
</script>

```

But there are exclusions, for instance `input.value` synchronizes only from attribute → property, but not back:

```

<input>

<script>
  let input = document.querySelector('input');

  // attribute => property
  input.setAttribute('value', 'text');
  alert(input.value); // text

  // NOT property => attribute
  input.value = 'newValue';
  alert(input.getAttribute('value')); // text (not updated!)
</script>

```

In the example above:

- Changing the attribute `value` updates the property.
- But the property change does not affect the attribute.

That “feature” may actually come in handy, because the user actions may lead to `value` changes, and then after them, if we want to recover the “original” value from HTML, it’s in the attribute.

DOM properties are typed

DOM properties are not always strings. For instance, the `input.checked` property (for checkboxes) is a boolean:

```

<input id="input" type="checkbox" checked> checkbox

<script>
  alert(input.getAttribute('checked')); // the attribute value is: empty string

```

```
    alert(input.checked); // the property value is: true
</script>
```

There are other examples. The `style` attribute is a string, but the `style` property is an object:

```
<div id="div" style="color:red;font-size:120%">Hello</div>

<script>
    // string
    alert(div.getAttribute('style')); // color:red;font-size:120%

    // object
    alert(div.style); // [object CSSStyleDeclaration]
    alert(div.style.color); // red
</script>
```

Most properties are strings though.

Quite rarely, even if a DOM property type is a string, it may differ from the attribute. For instance, the `href` DOM property is always a *full* URL, even if the attribute contains a relative URL or just a `#hash`.

Here's an example:

```
<a id="a" href="#hello">link</a>
<script>
    // attribute
    alert(a.getAttribute('href')); // #hello

    // property
    alert(a.href ); // full URL in the form http://site.com/page#hello
</script>
```

If we need the value of `href` or any other attribute exactly as written in the HTML, we can use `getAttribute`.

Non-standard attributes, dataset

When writing HTML, we use a lot of standard attributes. But what about non-standard, custom ones? First, let's see whether they are useful or not? What for?

Sometimes non-standard attributes are used to pass custom data from HTML to JavaScript, or to “mark” HTML-elements for JavaScript.

Like this:

```

<!-- mark the div to show "name" here -->
<div show-info="name"></div>
<!-- and age here -->
<div show-info="age"></div>

<script>
  // the code finds an element with the mark and shows what's requested
  let user = {
    name: "Pete",
    age: 25
  };

  for(let div of document.querySelectorAll('[show-info]')) {
    // insert the corresponding info into the field
    let field = div.getAttribute('show-info');
    div.innerHTML = user[field]; // first Pete into "name", then 25 into "age"
  }
</script>

```

Also they can be used to style an element.

For instance, here for the order state the attribute `order-state` is used:

```

<style>
  /* styles rely on the custom attribute "order-state" */
  .order[order-state="new"] {
    color: green;
  }

  .order[order-state="pending"] {
    color: blue;
  }

  .order[order-state="canceled"] {
    color: red;
  }
</style>

<div class="order" order-state="new">
  A new order.
</div>

<div class="order" order-state="pending">
  A pending order.
</div>

<div class="order" order-state="canceled">
  A canceled order.
</div>

```

Why would using an attribute be preferable to having classes like `.order-state-new`, `.order-state-pending`, `.order-state-canceled`?

Because an attribute is more convenient to manage. The state can be changed as easy as:

```
// a bit simpler than removing old/adding a new class
div.setAttribute('order-state', 'canceled');
```

But there may be a possible problem with custom attributes. What if we use a non-standard attribute for our purposes and later the standard introduces it and makes it do something? The HTML language is alive, it grows, and more attributes appear to suit the needs of developers. There may be unexpected effects in such case.

To avoid conflicts, there exist `data-*` [↗](#) attributes.

All attributes starting with “data-” are reserved for programmers’ use. They are available in the `dataset` property.

For instance, if an `elem` has an attribute named `"data-about"`, it’s available as `elem.dataset.about`.

Like this:

```
<body data-about="Elephants">
<script>
  alert(document.body.dataset.about); // Elephants
</script>
```

Multiword attributes like `data-order-state` become camel-cased: `dataset.orderState`.

Here’s a rewritten “order state” example:

```
<style>
  .order[data-order-state="new"] {
    color: green;
  }

  .order[data-order-state="pending"] {
    color: blue;
  }

  .order[data-order-state="canceled"] {
    color: red;
  }
</style>

<div id="order" class="order" data-order-state="new">
```

```

A new order.
</div>

<script>
  // read
  alert(order.dataset.orderState); // new

  // modify
  order.dataset.orderState = "pending"; // (*)
</script>

```

Using `data-*` attributes is a valid, safe way to pass custom data.

Please note that we can not only read, but also modify data-attributes. Then CSS updates the view accordingly: in the example above the last line `(*)` changes the color to blue.

Summary

- Attributes – is what’s written in HTML.
- Properties – is what’s in DOM objects.

A small comparison:

	Properties	Attributes
Type	Any value, standard properties have types described in the spec	A string
Name	Name is case-sensitive	Name is not case-sensitive

Methods to work with attributes are:

- `elem.hasAttribute(name)` – to check for existence.
- `elem.getAttribute(name)` – to get the value.
- `elem.setAttribute(name, value)` – to set the value.
- `elem.removeAttribute(name)` – to remove the attribute.
- `elem.attributes` is a collection of all attributes.

For most situations using DOM properties is preferable. We should refer to attributes only when DOM properties do not suit us, when we need exactly attributes, for instance:

- We need a non-standard attribute. But if it starts with `data-`, then we should use `dataset`.
- We want to read the value “as written” in HTML. The value of the DOM property may be different, for instance the `href` property is always a full URL, and we may want to get the “original” value.

✓ Tasks

Get the attribute

importance: 5

Write the code to select the element with `data-widget-name` attribute from the document and to read its value.

```
<!DOCTYPE html>
<html>
<body>

  <div data-widget-name="menu">Choose the genre</div>

  <script>
    /* your code */
  </script>
</body>
</html>
```

To solution

Make external links orange

importance: 3

Make all external links orange by altering their `style` property.

A link is external if:

- Its `href` has `://` in it
- But doesn't start with `http://internal.com`.

Example:

```
<a name="list">the list</a>
<ul>
  <li><a href="http://google.com">http://google.com</a></li>
  <li><a href="/tutorial">/tutorial.html</a></li>
  <li><a href="local/path">local/path</a></li>
  <li><a href="ftp://ftp.com/my.zip">ftp://ftp.com/my.zip</a></li>
  <li><a href="http://nodejs.org">http://nodejs.org</a></li>
  <li><a href="http://internal.com/test">http://internal.com/test</a></li>
</ul>

<script>
  // setting style for a single link
  let link = document.querySelector('a');
```

```
link.style.color = 'orange';  
</script>
```

The result should be:

The list:

- <http://google.com>
- </tutorial.html>
- <local/path>
- <ftp://ftp.com/my.zip>
- <http://nodejs.org>
- <http://internal.com/test>

[Open a sandbox for the task.](#) ↗

[To solution](#)

Modifying the document

DOM modification is the key to creating “live” pages.

Here we’ll see how to create new elements “on the fly” and modify the existing page content.

Example: show a message

Let’s demonstrate using an example. We’ll add a message on the page that looks nicer than `alert`.

Here’s how it will look:

```
<style>  
.alert {  
  padding: 15px;  
  border: 1px solid #d6e9c6;  
  border-radius: 4px;  
  color: #3c763d;  
  background-color: #dff0d8;  
}  
</style>
```

```
<div class="alert">  
  <strong>Hi there!</strong> You've read an important message.  
</div>
```

Hi there! You've read an important message.

That was the HTML example. Now let's create the same `div` with JavaScript (assuming that the styles are in the HTML/CSS already).

Creating an element

To create DOM nodes, there are two methods:

`document.createElement(tag)`

Creates a new *element node* with the given tag:

```
let div = document.createElement('div');
```

`document.createTextNode(text)`

Creates a new *text node* with the given text:

```
let textNode = document.createTextNode('Here I am');
```

Most of the time we need to create element nodes, such as the `div` for the message.

Creating the message

Creating the message div takes 3 steps:

```
// 1. Create <div> element
let div = document.createElement('div');

// 2. Set its class to "alert"
div.className = "alert";

// 3. Fill it with the content
div.innerHTML = "<strong>Hi there!</strong> You've read an important message.";
```

We've created the element. But as of now it's only in a variable named `div`, not in the page yet. So we can't see it.

Insertion methods

To make the `div` show up, we need to insert it somewhere into `document`. For instance, into `<body>` element, referenced by `document.body`.

There's a special method `append` for that: `document.body.append(div)`.

Here's the full code:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>Hi there!</strong> You've read an important message."

  document.body.append(div);
</script>
```

Here we called `append` on `document.body`, but we can call `append` method on any other element, to put another element into it. For instance, we can append something to `<div>` by calling `div.append(anotherElement)`.

Here are more insertion methods, they specify different places where to insert:

- `node.append(...nodes or strings)` – append nodes or strings *at the end* of `node`,
- `node.prepend(...nodes or strings)` – insert nodes or strings *at the beginning* of `node`,
- `node.before(...nodes or strings)` – insert nodes or strings *before* `node`,
- `node.after(...nodes or strings)` – insert nodes or strings *after* `node`,
- `node.replaceWith(...nodes or strings)` – replaces `node` with the given nodes or strings.

Arguments of these methods are an arbitrary list of DOM nodes to insert, or text strings (that become text nodes automatically).

Let's see them in action.

Here's an example of using these methods to add items to a list and the text before/after it:

```
<ol id="ol">
  <li>0</li>
```

```

<li>1</li>
<li>2</li>
</ol>

<script>
  ol.before('before'); // insert string "before" before <ol>
  ol.after('after'); // insert string "after" after <ol>

  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // insert liFirst at the beginning of <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // insert liLast at the end of <ol>
</script>

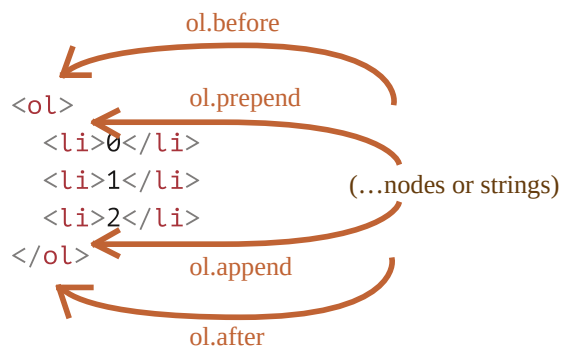
```

before

1. prepend
2. 0
3. 1
4. 2
5. append

after

Here's a visual picture of what the methods do:



So the final list will be:

```

before
<ol id="ol">
  <li>prepend</li>
  <li>0</li>
  <li>1</li>
  <li>2</li>
  <li>append</li>
</ol>
after

```

As said, these methods can insert multiple nodes and text pieces in a single call.

For instance, here a string and an element are inserted:

```
<div id="div"></div>
<script>
  div.before('<p>Hello</p>', document.createElement('hr'));
</script>
```

Please note: the text is inserted “as text”, not “as HTML”, with proper escaping of characters such as `<`, `>`.

So the final HTML is:

```
&lt;p&gt;Hello&lt;/p&gt;
<hr>
<div id="div"></div>
```

In other words, strings are inserted in a safe way, like `elem.textContent` does it.

So, these methods can only be used to insert DOM nodes or text pieces.

But what if we'd like to insert an HTML string “as html”, with all tags and stuff working, in the same manner as `elem.innerHTML` does it?

insertAdjacentHTML/Text/Element

For that we can use another, pretty versatile method:

```
elem.insertAdjacentHTML(where, html).
```

The first parameter is a code word, specifying where to insert relative to `elem`. Must be one of the following:

- `"beforebegin"` – insert `html` immediately before `elem`,
- `"afterbegin"` – insert `html` into `elem`, at the beginning,
- `"beforeend"` – insert `html` into `elem`, at the end,
- `"afterend"` – insert `html` immediately after `elem`.

The second parameter is an HTML string, that is inserted “as HTML”.

For instance:

```
<div id="div"></div>
<script>
  div.insertAdjacentHTML('beforebegin', '<p>Hello</p>');
```

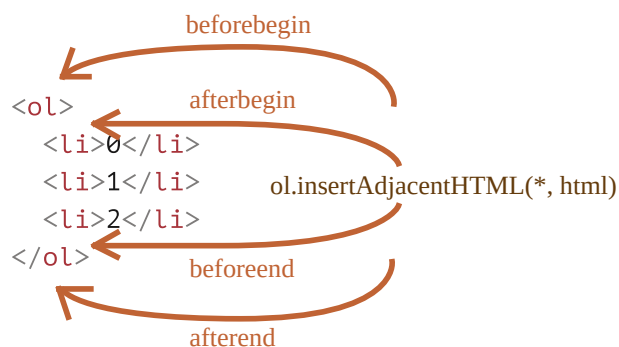
```
div.insertAdjacentHTML('afterend', '<p>Bye</p>');
</script>
```

...Would lead to:

```
<p>Hello</p>
<div id="div"></div>
<p>Bye</p>
```

That's how we can append arbitrary HTML to the page.

Here's the picture of insertion variants:



We can easily notice similarities between this and the previous picture. The insertion points are actually the same, but this method inserts HTML.

The method has two brothers:

- `elem.insertAdjacentText(where, text)` – the same syntax, but a string of `text` is inserted “as text” instead of HTML,
- `elem.insertAdjacentElement(where, elem)` – the same syntax, but inserts an element.

They exist mainly to make the syntax “uniform”. In practice, only `insertAdjacentHTML` is used most of the time. Because for elements and text, we have methods `append/prepend/before/after` – they are shorter to write and can insert nodes/text pieces.

So here's an alternative variant of showing a message:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
```

```

</style>

<script>
  document.body.insertAdjacentHTML("afterbegin", `<div class="alert">
    <strong>Hi there!</strong> You've read an important message.
  </div>`);
</script>

```

Node removal

To remove a node, there's a method `node.remove()`.

Let's make our message disappear after a second:

```

<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>Hi there!</strong> You've read an important message."

  document.body.append(div);
  setTimeout(() => div.remove(), 1000);
</script>

```

Please note: if we want to *move* an element to another place – there's no need to remove it from the old one.

All insertion methods automatically remove the node from the old place.

For instance, let's swap elements:

```

<div id="first">First</div>
<div id="second">Second</div>
<script>
  // no need to call remove
  second.after(first); // take #second and after it insert #first
</script>

```

Cloning nodes: cloneNode

How to insert one more similar message?

We could make a function and put the code there. But the alternative way would be to *clone* the existing `div` and modify the text inside it (if needed).

Sometimes when we have a big element, that may be faster and simpler.

- The call `elem.cloneNode(true)` creates a “deep” clone of the element – with all attributes and subelements. If we call `elem.cloneNode(false)`, then the clone is made without child elements.

An example of copying the message:

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<div class="alert" id="div">
  <strong>Hi there!</strong> You've read an important message.
</div>

<script>
  let div2 = div.cloneNode(true); // clone the message
  div2.querySelector('strong').innerHTML = 'Bye there!'; // change the clone

  div.after(div2); // show the clone after the existing div
</script>
```

DocumentFragment

`DocumentFragment` is a special DOM node that serves as a wrapper to pass around lists of nodes.

We can append other nodes to it, but when we insert it somewhere, then its content is inserted instead.

For example, `getListContent` below generates a fragment with `<li>` items, that are later inserted into `<ul>`:

```
<ul id="ul"></ul>
```

```

<script>
function getListContent() {
  let fragment = new DocumentFragment();

  for(let i=1; i<=3; i++) {
    let li = document.createElement('li');
    li.append(i);
    fragment.append(li);
  }

  return fragment;
}

ul.append(getListContent()); // (*)
</script>

```

Please note, at the last line (*) we append `DocumentFragment`, but it “blends in”, so the resulting structure will be:

```

<ul>
  <li>1</li>
  <li>2</li>
  <li>3</li>
</ul>

```

`DocumentFragment` is rarely used explicitly. Why append to a special kind of node, if we can return an array of nodes instead? Rewritten example:

```

<ul id="ul"></ul>

<script>
function getListContent() {
  let result = [];

  for(let i=1; i<=3; i++) {
    let li = document.createElement('li');
    li.append(i);
    result.push(li);
  }

  return result;
}

ul.append(...getListContent()); // append + "..." operator = friends!
</script>

```

We mention `DocumentFragment` mainly because there are some concepts on top of it, like `template` element, that we’ll cover much later.

Old-school insert/remove methods



Old school

This information helps to understand old scripts, but not needed for new development.

There are also “old school” DOM manipulation methods, existing for historical reasons.

These methods come from really ancient times. Nowadays, there’s no reason to use them, as modern methods, such as `append`, `prepend`, `before`, `after`, `remove`, `replaceWith`, are more flexible.

The only reason we list these methods here is that you can find them in many old scripts:

`parentElem.appendChild(node)`

Appends `node` as the last child of `parentElem`.

The following example adds a new `<li>` to the end of `<ol>`:

```
<ol id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  let newLi = document.createElement('li');
  newLi.innerHTML = 'Hello, world!';

  list.appendChild(newLi);
</script>
```

`parentElem.insertBefore(node, nextSibling)`

Inserts `node` before `nextSibling` into `parentElem`.

The following code inserts a new list item before the second `<li>`:

```
<ol id="list">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>
<script>
  let newLi = document.createElement('li');
  newLi.innerHTML = 'Hello, world!';
```

```
list.insertBefore(newLi, list.children[1]);  
</script>
```

To insert `newLi` as the first element, we can do it like this:

```
list.insertBefore(newLi, list.firstChild);
```

`parentElem.replaceChild(node, oldChild)`

Replaces `oldChild` with `node` among children of `parentElem`.

`parentElem.removeChild(node)`

Removes `node` from `parentElem` (assuming `node` is its child).

The following example removes first `<li>` from `<ol>`:

```
<ol id="list">  
  <li>0</li>  
  <li>1</li>  
  <li>2</li>  
</ol>  
  
<script>  
  let li = list.firstChild;  
  list.removeChild(li);  
</script>
```

All these methods return the inserted/removed node. In other words, `parentElem.appendChild(node)` returns `node`. But usually the returned value is not used, we just run the method.

A word about “`document.write`”

There's one more, very ancient method of adding something to a web-page: `document.write`.

The syntax:

```
<p>Somewhere in the page...</p>  
<script>  
  document.write('<b>Hello from JS</b>');  
</script>  
<p>The end</p>
```

The call to `document.write(html)` writes the `html` into page “right here and now”. The `html` string can be dynamically generated, so it’s kind of flexible. We can use JavaScript to create a full-fledged webpage and write it.

The method comes from times when there was no DOM, no standards... Really old times. It still lives, because there are scripts using it.

In modern scripts we can rarely see it, because of the following important limitation:

The call to `document.write` only works while the page is loading.

If we call it afterwards, the existing document content is erased.

For instance:

```
<p>After one second the contents of this page will be replaced...</p>
<script>
  // document.write after 1 second
  // that's after the page loaded, so it erases the existing content
  setTimeout(() => document.write('<b>...By this.</b>'), 1000);
</script>
```

So it’s kind of unusable at “after loaded” stage, unlike other DOM methods we covered above.

That’s the downside.

There’s an upside also. Technically, when `document.write` is called while the browser is reading (“parsing”) incoming HTML, and it writes something, the browser consumes it just as if it were initially there, in the HTML text.

So it works blazingly fast, because there’s *no DOM modification* involved. It writes directly into the page text, while the DOM is not yet built.

So if we need to add a lot of text into HTML dynamically, and we’re at page loading phase, and the speed matters, it may help. But in practice these requirements rarely come together. And usually we can see this method in scripts just because they are old.

Summary

- Methods to create new nodes:
 - `document.createElement(tag)` – creates an element with the given tag,
 - `document.createTextNode(value)` – creates a text node (rarely used),
 - `elem.cloneNode(deep)` – clones the element, if `deep==true` then with all descendants.
- Insertion and removal:
 - `node.append(...nodes or strings)` – insert into `node`, at the end,

- `node.prepend(...nodes or strings)` – insert into `node`, at the beginning,
- `node.before(...nodes or strings)` – insert right before `node`,
- `node.after(...nodes or strings)` – insert right after `node`,
- `node.replaceWith(...nodes or strings)` – replace `node`.
- `node.remove()` – remove the `node`.

Text strings are inserted “as text”.

- There are also “old school” methods:
 - `parent.appendChild(node)`
 - `parent.insertBefore(node, nextSibling)`
 - `parent.removeChild(node)`
 - `parent.replaceChild(newElem, node)`

All these methods return `node`.

- Given some HTML in `html`, `elem.insertAdjacentHTML(where, html)` inserts it depending on the value of `where`:
 - `"beforebegin"` – insert `html` right before `elem`,
 - `"afterbegin"` – insert `html` into `elem`, at the beginning,
 - `"beforeend"` – insert `html` into `elem`, at the end,
 - `"afterend"` – insert `html` right after `elem`.

Also there are similar methods, `elem.insertAdjacentText` and `elem.insertAdjacentElement`, that insert text strings and elements, but they are rarely used.

- To append HTML to the page before it has finished loading:
 - `document.write(html)`

After the page is loaded such a call erases the document. Mostly seen in old scripts.

✓ Tasks

createTextNode vs innerHTML vs textContent

importance: 5

We have an empty DOM element `elem` and a string `text`.

Which of these 3 commands will do exactly the same?

1. `elem.append(document.createTextNode(text))`

2. `elem.innerHTML = text`
3. `elem.textContent = text`

[To solution](#)

Clear the element

importance: 5

Create a function `clear(elem)` that removes everything from the element.

```
<ol id="elem">
  <li>Hello</li>
  <li>World</li>
</ol>

<script>
  function clear(elem) { /* your code */ }

  clear(elem); // clears the list
</script>
```

[To solution](#)

Why does "aaa" remain?

importance: 1

In the example below, the call `table.remove()` removes the table from the document.

But if you run it, you can see that the text `"aaa"` is still visible.

Why does that happen?

```
<table id="table">
  aaa
  <tr>
    <td>Test</td>
  </tr>
</table>

<script>
  alert(table); // the table, as it should be

  table.remove();
  // why there's still "aaa" in the document?
</script>
```

[To solution](#)

Create a list

importance: 4

Write an interface to create a list from user input.

For every list item:

1. Ask a user about its content using `prompt`.
2. Create the `<li>` with it and add it to `<ul>`.
3. Continue until the user cancels the input (by pressing `Esc` or via an empty entry).

All elements should be created dynamically.

If a user types HTML-tags, they should be treated like a text.

[Demo in new window](#) ↗

[To solution](#)

Create a tree from the object

importance: 5

Write a function `createTree` that creates a nested `ul/li` list from the nested object.

For instance:

```
let data = {
  "Fish": {
    "trout": {},
    "salmon": {}
  },

  "Tree": {
    "Huge": {
      "sequoia": {},
      "oak": {}
    },
    "Flowering": {
      "apple tree": {},
      "magnolia": {}
    }
  }
}
```

```
}  
};
```

The syntax:

```
let container = document.getElementById('container');  
createTree(container, data); // creates the tree in the container
```

The result (tree) should look like this:

- Fish
 - trout
 - salmon
- Tree
 - Huge
 - sequoia
 - oak
 - Flowering
 - apple tree
 - magnolia

Choose one of two ways of solving this task:

1. Create the HTML for the tree and then assign to `container.innerHTML`.
2. Create tree nodes and append with DOM methods.

Would be great if you could do both.

P.S. The tree should not have “extra” elements like empty `<ul></ul>` for the leaves.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Show descendants in a tree

importance: 5

There's a tree organized as nested `ul/li`.

Write the code that adds to each `<li>` the number of its descendants. Skip leaves (nodes without children).

The result:

- Animals [9]
 - Mammals [4]
 - Cows
 - Donkeys
 - Dogs
 - Tigers
 - Other [3]
 - Snakes
 - Birds
 - Lizards
- Fishes [5]
 - Aquarium [2]
 - Guppy
 - Angelfish
 - Sea [1]
 - Sea trout

[Open a sandbox for the task.](#)

[To solution](#)

Create a calendar

importance: 4

Write a function `createCalendar(elem, year, month)`.

The call should create a calendar for the given year/month and put it inside `elem`.

The calendar should be a table, where a week is `<tr>`, and a day is `<td>`. The table top should be `<th>` with weekday names: the first day should be Monday, and so on till Sunday.

For instance, `createCalendar(cal, 2012, 9)` should generate in element `cal` the following calendar:

MO	TU	WE	TH	FR	SA	SU
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30

P.S. For this task it's enough to generate the calendar, should not yet be clickable.

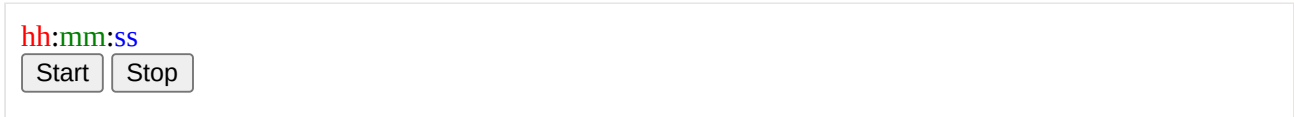
[Open a sandbox for the task.](#)

[To solution](#)

Colored clock with setInterval

importance: 4

Create a colored clock like here:



Use HTML/CSS for the styling, JavaScript only updates time in elements.

[Open a sandbox for the task.](#) [↗](#)

[To solution](#)

Insert the HTML in the list

importance: 5

Write the code to insert `<li>2</li><li>3</li>` between two `<li>` here:

```
<ul id="ul">
  <li id="one">1</li>
  <li id="two">4</li>
</ul>
```

[To solution](#)

Sort the table

importance: 5

There's a table:

```
<table>
<thead>
  <tr>
    <th>Name</th><th>Surname</th><th>Age</th>
  </tr>
</thead>
<tbody>
  <tr>
    <td>John</td><td>Smith</td><td>10</td>
  </tr>
  <tr>
    <td>Pete</td><td>Brown</td><td>15</td>
```

```
</tr>
<tr>
  <td>Ann</td><td>Lee</td><td>5</td>
</tr>
<tr>
  <td>...</td><td>...</td><td>...</td>
</tr>
</tbody>
</table>
```

There may be more rows in it.

Write the code to sort it by the "name" column.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Styles and classes

Before we get into JavaScript's ways of dealing with styles and classes – here's an important rule. Hopefully it's obvious enough, but we still have to mention it.

There are generally two ways to style an element:

1. Create a class in CSS and add it: `<div class="...">`
2. Write properties directly into `style`: `<div style="...">.`

JavaScript can modify both classes and `style` properties.

We should always prefer CSS classes to `style`. The latter should only be used if classes “can't handle it”.

For example, `style` is acceptable if we calculate coordinates of an element dynamically and want to set them from JavaScript, like this:

```
let top = /* complex calculations */;
let left = /* complex calculations */;

elem.style.left = left; // e.g '123px', calculated at run-time
elem.style.top = top; // e.g '456px'
```

For other cases, like making the text red, adding a background icon – describe that in CSS and then add the class (JavaScript can do that). That's more flexible and easier to support.

className and classList

Changing a class is one of the most often used actions in scripts.

In the ancient time, there was a limitation in JavaScript: a reserved word like `"class"` could not be an object property. That limitation does not exist now, but at that time it was impossible to have a `"class"` property, like `elem.class`.

So for classes the similar-looking property `"className"` was introduced: the `elem.className` corresponds to the `"class"` attribute.

For instance:

```
<body class="main page">
  <script>
    alert(document.body.className); // main page
  </script>
</body>
```

If we assign something to `elem.className`, it replaces the whole string of classes. Sometimes that's what we need, but often we want to add/remove a single class.

There's another property for that: `elem.classList`.

The `elem.classList` is a special object with methods to `add/remove/toggle` a single class.

For instance:

```
<body class="main page">
  <script>
    // add a class
    document.body.classList.add('article');

    alert(document.body.className); // main page article
  </script>
</body>
```

So we can operate both on the full class string using `className` or on individual classes using `classList`. What we choose depends on our needs.

Methods of `classList`:

- `elem.classList.add/remove("class")` – adds/removes the class.
- `elem.classList.toggle("class")` – adds the class if it doesn't exist, otherwise removes it.
- `elem.classList.contains("class")` – checks for the given class, returns `true/false`.

Besides, `classList` is iterable, so we can list all classes with `for...of`, like this:

```
<body class="main page">
  <script>
    for (let name of document.body.classList) {
      alert(name); // main, and then page
    }
  </script>
</body>
```

Element style

The property `elem.style` is an object that corresponds to what's written in the `"style"` attribute. Setting `elem.style.width="100px"` works the same as if we had in the attribute `style` a string `width:100px`.

For multi-word property the camelCase is used:

```
background-color  => elem.style.backgroundColor
z-index           => elem.style.zIndex
border-left-width => elem.style.borderLeftWidth
```

For instance:

```
document.body.style.backgroundColor = prompt('background color?', 'green');
```

Prefixed properties

Browser-prefixed properties like `-moz-border-radius`, `-webkit-border-radius` also follow the same rule: a dash means upper case.

For instance:

```
button.style.MozBorderRadius = '5px';
button.style.WebkitBorderRadius = '5px';
```

Resetting the style property

Sometimes we want to assign a style property, and later remove it.

For instance, to hide an element, we can set `elem.style.display = "none"`.

Then later we may want to remove the `style.display` as if it were not set. Instead of `delete elem.style.display` we should assign an empty string to it:
`elem.style.display = ""`.

```
// if we run this code, the <body> will blink
document.body.style.display = "none"; // hide

setTimeout(() => document.body.style.display = "", 1000); // back to normal
```

If we set `style.display` to an empty string, then the browser applies CSS classes and its built-in styles normally, as if there were no such `style.display` property at all.

Also there is a special method for that, `elem.style.removeProperty('style property')`. So, We can remove a property like this:

```
document.body.style.background = 'red'; //set background to red

setTimeout(() => document.body.style.removeProperty('background'), 1000); // remove
```

i Full rewrite with `style.cssText`

Normally, we use `style.*` to assign individual style properties. We can't set the full style like `div.style="color: red; width: 100px"`, because `div.style` is an object, and it's read-only.

To set the full style as a string, there's a special property `style.cssText`:

```
<div id="div">Button</div>

<script>
  // we can set special style flags like "important" here
  div.style.cssText=`color: red !important;
    background-color: yellow;
    width: 100px;
    text-align: center;
  `;

  alert(div.style.cssText);
</script>
```

This property is rarely used, because such assignment removes all existing styles: it does not add, but replaces them. May occasionally delete something needed. But we can safely use it for new elements, when we know we won't delete an existing style.

The same can be accomplished by setting an attribute:

```
div.setAttribute('style', 'color: red...').
```

Mind the units

Don't forget to add CSS units to values.

For instance, we should not set `elem.style.top` to `10`, but rather to `10px`. Otherwise it wouldn't work:

```
<body>
<script>
  // doesn't work!
  document.body.style.margin = 20;
  alert(document.body.style.margin); // '' (empty string, the assignment is ignored)

  // now add the CSS unit (px) - and it works
  document.body.style.margin = '20px';
  alert(document.body.style.margin); // 20px

  alert(document.body.style.marginTop); // 20px
  alert(document.body.style.marginLeft); // 20px
```

```
</script>
</body>
```

Please note: the browser “unpacks” the property `style.margin` in the last lines and infers `style.marginLeft` and `style.marginTop` from it.

Computed styles: `getComputedStyle`

So, modifying a style is easy. But how to *read* it?

For instance, we want to know the size, margins, the color of an element. How to do it?

The `style` property operates only on the value of the `"style"` attribute, without any CSS cascade.

So we can't read anything that comes from CSS classes using `elem.style`.

For instance, here `style` doesn't see the margin:

```
<head>
  <style> body { color: red; margin: 5px } </style>
</head>
<body>

  The red text
  <script>
    alert(document.body.style.color); // empty
    alert(document.body.style.marginTop); // empty
  </script>
</body>
```

...But what if we need, say, to increase the margin by `20px`? We would want the current value of it.

There's another method for that: `getComputedStyle`.

The syntax is:

```
getComputedStyle(element, [pseudo])
```

element

Element to read the value for.

pseudo

A pseudo-element if required, for instance `::before`. An empty string or no argument means the element itself.

The result is an object with styles, like `elem.style`, but now with respect to all CSS classes.

For instance:

```
<head>
  <style> body { color: red; margin: 5px } </style>
</head>
<body>

  <script>
    let computedStyle = getComputedStyle(document.body);

    // now we can read the margin and the color from it

    alert( computedStyle.marginTop ); // 5px
    alert( computedStyle.color ); // rgb(255, 0, 0)
  </script>

</body>
```

i Computed and resolved values

There are two concepts in [CSS](#) ↗ :

1. A *computed* style value is the value after all CSS rules and CSS inheritance is applied, as the result of the CSS cascade. It can look like `height:1em` or `font-size:125%`.
2. A *resolved* style value is the one finally applied to the element. Values like `1em` or `125%` are relative. The browser takes the computed value and makes all units fixed and absolute, for instance: `height:20px` or `font-size:16px`. For geometry properties resolved values may have a floating point, like `width:50.5px`.

A long time ago `getComputedStyle` was created to get computed values, but it turned out that resolved values are much more convenient, and the standard changed.

So nowadays `getComputedStyle` actually returns the resolved value of the property, usually in `px` for geometry.

 `getComputedStyle` requires the full property name

We should always ask for the exact property that we want, like `paddingLeft` or `marginTop` or `borderTopWidth`. Otherwise the correct result is not guaranteed.

For instance, if there are properties `paddingLeft/paddingTop`, then what should we get for `getComputedStyle(elem).padding`? Nothing, or maybe a “generated” value from known paddings? There’s no standard rule here.

 Styles applied to `:visited` links are hidden!

Visited links may be colored using `:visited` CSS pseudoclass.

But `getComputedStyle` does not give access to that color, because otherwise an arbitrary page could find out whether the user visited a link by creating it on the page and checking the styles.

JavaScript may not see the styles applied by `:visited`. And also, there’s a limitation in CSS that forbids applying geometry-changing styles in `:visited`. That’s to guarantee that there’s no side way for an evil page to test if a link was visited and hence to break the privacy.

Summary

To manage classes, there are two DOM properties:

- `className` – the string value, good to manage the whole set of classes.
- `classList` – the object with methods `add/remove/toggle/contains`, good for individual classes.

To change the styles:

- The `style` property is an object with camelCased styles. Reading and writing to it has the same meaning as modifying individual properties in the `"style"` attribute. To see how to apply `important` and other rare stuff – there’s a list of methods at [MDN](#) .
- The `style.cssText` property corresponds to the whole `"style"` attribute, the full string of styles.

To read the resolved styles (with respect to all classes, after all CSS is applied and final values are calculated):

- The `getComputedStyle(elem, [pseudo])` returns the style-like object with them. Read-only.

✓ Tasks

Create a notification

importance: 5

Write a function `showNotification(options)` that creates a notification: `<div class="notification">` with the given content. The notification should automatically disappear after 1.5 seconds.

The options are:

```
// shows an element with the text "Hello" near the right-top of the window
showNotification({
  top: 10, // 10px from the top of the window (by default 0px)
  right: 10, // 10px from the right edge of the window (by default 0px)
  html: "Hello!", // the HTML of notification
  className: "welcome" // an additional class for the div (optional)
});
```

[Demo in new window](#) ↗

Use CSS positioning to show the element at given top/right coordinates. The source document has the necessary styles.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Element size and scrolling

There are many JavaScript properties that allow us to read information about element width, height and other geometry features.

We often need them when moving or positioning elements in JavaScript.

Sample element

As a sample element to demonstrate properties we'll use the one given below:

```
<div id="example">
  ...Text...
</div>
<style>
  #example {
```

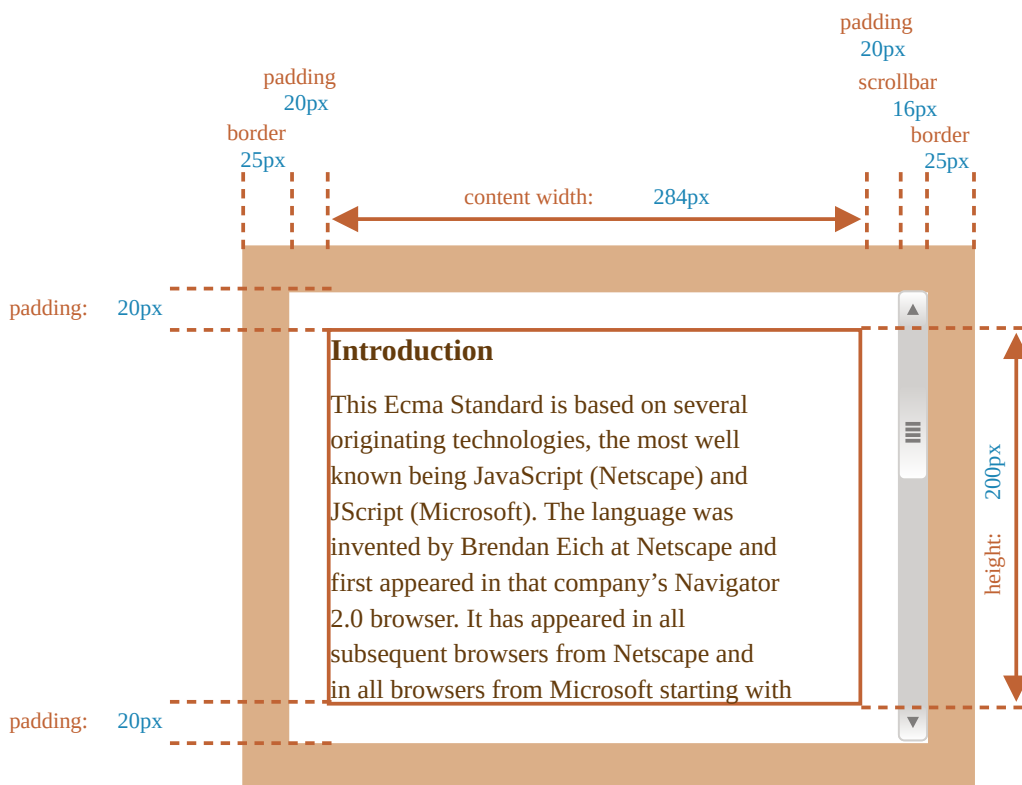
```

width: 300px;
height: 200px;
border: 25px solid #E8C48F;
padding: 20px;
overflow: auto;
}
</style>

```

It has the border, padding and scrolling. The full set of features. There are no margins, as they are not the part of the element itself, and there are no special properties for them.

The element looks like this:



You can [open the document in the sandbox](#) .

i Mind the scrollbar

The picture above demonstrates the most complex case when the element has a scrollbar. Some browsers (not all) reserve the space for it by taking it from the content (labeled as “content width” above).

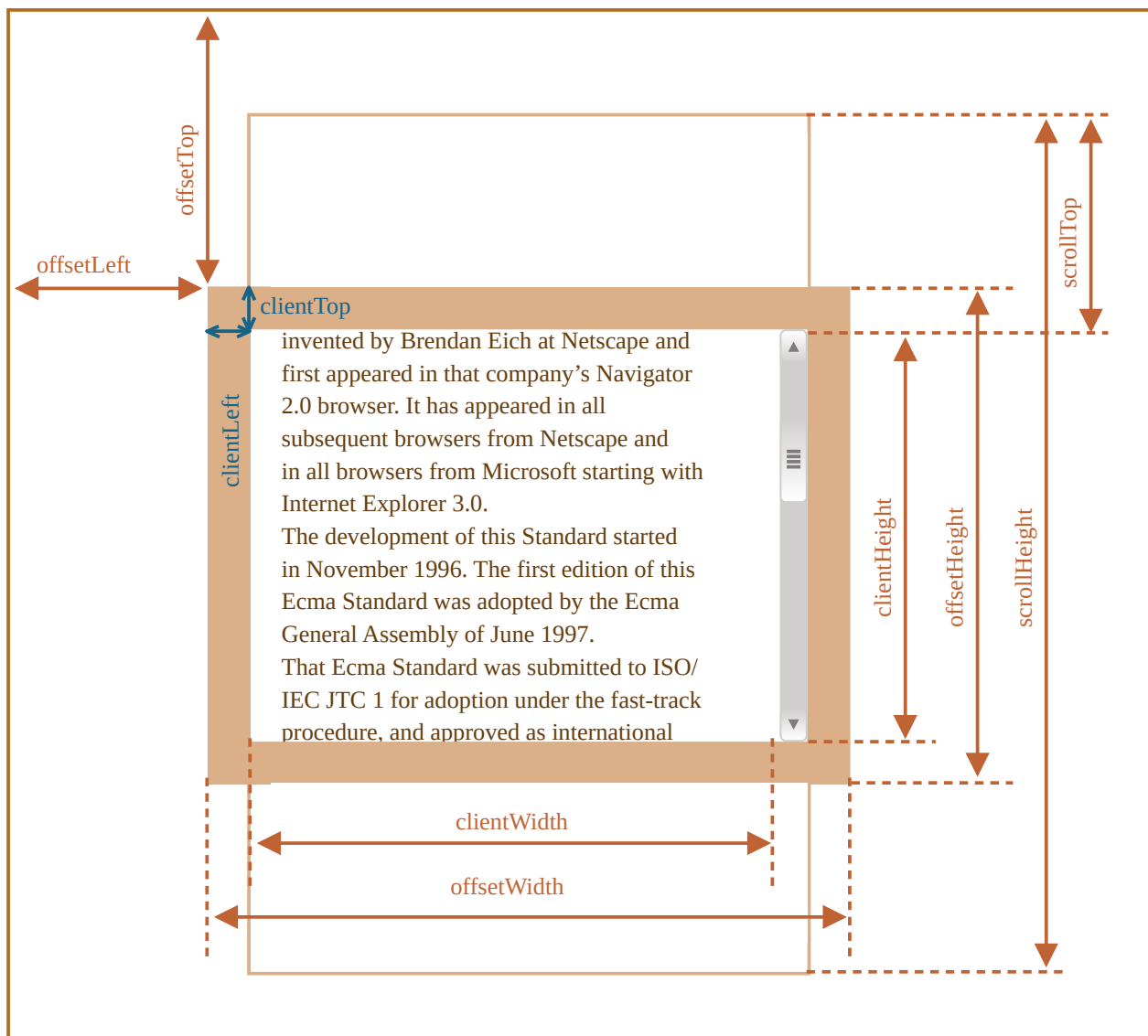
So, without scrollbar the content width would be `300px`, but if the scrollbar is `16px` wide (the width may vary between devices and browsers) then only `300 - 16 = 284px` remains, and we should take it into account. That’s why examples from this chapter assume that there’s a scrollbar. Without it, some calculations are simpler.

i The `padding-bottom` area may be filled with text

Usually paddings are shown empty on our illustrations, but if there’s a lot of text in the element and it overflows, then browsers show the “overflowing” text at `padding-bottom`, that’s normal.

Geometry

Here’s the overall picture with geometry properties:



Values of these properties are technically numbers, but these numbers are “of pixels”, so these are pixel measurements.

Let's start exploring the properties starting from the outside of the element.

offsetParent, offsetLeft/Top

These properties are rarely needed, but still they are the “most outer” geometry properties, so we'll start with them.

The `offsetParent` is the nearest ancestor that the browser uses for calculating coordinates during rendering.

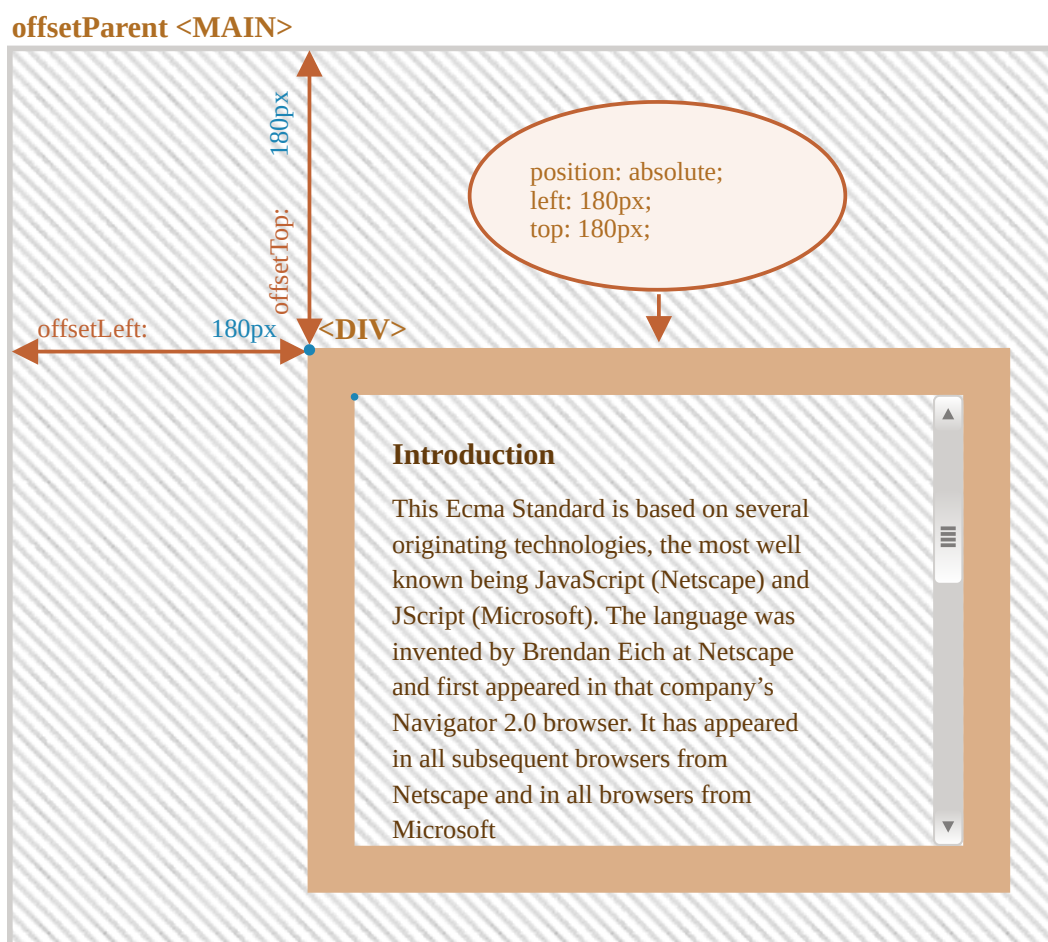
That's the nearest ancestor that is one of the following:

1. CSS-positioned (`position` is `absolute`, `relative`, `fixed` or `sticky`), or
2. `<td>`, `<th>`, or `<table>`, or
3. `<body>`.

Properties `offsetLeft/offsetTop` provide x/y coordinates relative to `offsetParent` upper-left corner.

In the example below the inner `<div>` has `<main>` as `offsetParent` and `offsetLeft/offsetTop` shifts from its upper-left corner (180):

```
<main style="position: relative" id="main">
  <article>
    <div id="example" style="position: absolute; left: 180px; top: 180px">...</div>
  </article>
</main>
<script>
  alert(example.offsetParent.id); // main
  alert(example.offsetLeft); // 180 (note: a number, not a string "180px")
  alert(example.offsetTop); // 180
</script>
```



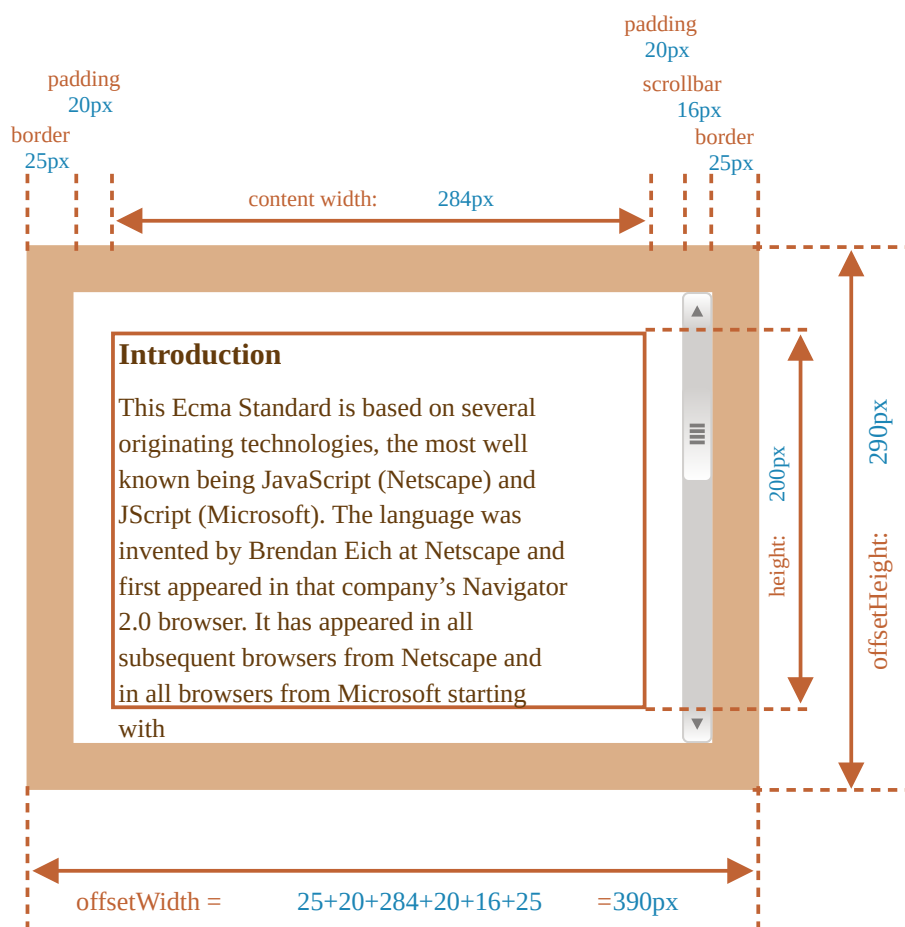
There are several occasions when `offsetParent` is `null`:

1. For not shown elements (`display:none` or not in the document).
2. For `<body>` and `<html>`.
3. For elements with `position:fixed`.

offsetWidth/Height

Now let's move on to the element itself.

These two properties are the simplest ones. They provide the “outer” width/height of the element. Or, in other words, its full size including borders.



For our sample element:

- `offsetWidth = 390` – the outer width, can be calculated as inner CSS-width (300px) plus paddings (2 * 20px) and borders (2 * 25px).
- `offsetHeight = 290` – the outer height.

i Geometry properties are zero/null for elements that are not displayed

Geometry properties are calculated only for displayed elements.

If an element (or any of its ancestors) has `display:none` or is not in the document, then all geometry properties are zero (or `null` for `offsetParent`).

For example, `offsetParent` is `null`, and `offsetWidth`, `offsetHeight` are `0` when we created an element, but haven't inserted it into the document yet, or it (or its ancestor) has `display:none`.

We can use this to check if an element is hidden, like this:

```
function isHidden(elem) {  
    return !elem.offsetWidth && !elem.offsetHeight;  
}
```

Please note that such `isHidden` returns `true` for elements that are on-screen, but have zero sizes.

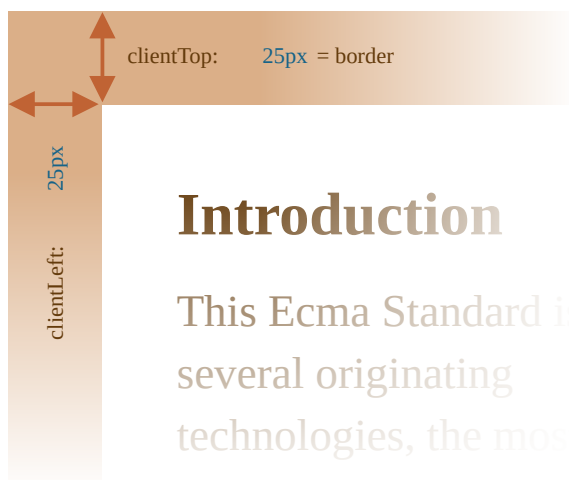
clientTop/Left

Inside the element we have the borders.

To measure them, there are properties `clientTop` and `clientLeft`.

In our example:

- `clientLeft = 25` – left border width
- `clientTop = 25` – top border width



Introduction

This Ecma Standard is
several originating
technologies, the most

1. JavaScript 2. CSS 3. DOM 4. XMLHttpRequest

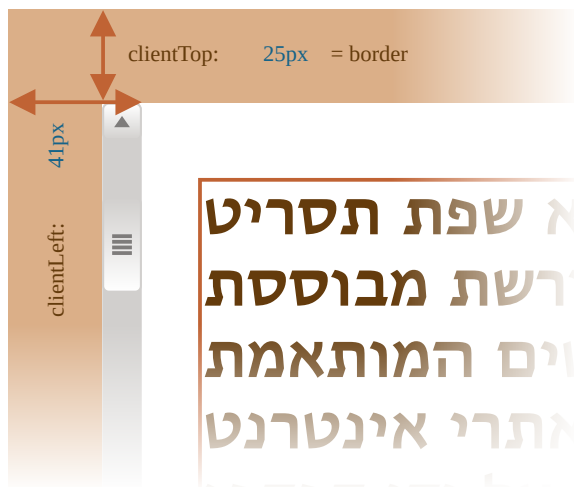
...But to be precise – these properties are not border width/height, but rather relative coordinates of the inner side from the outer side.

What's the difference?

It becomes visible when the document is right-to-left (the operating system is in Arabic or Hebrew languages). The scrollbar is then not on the right, but on the left, and then `clientLeft` also includes the scrollbar width.

In that case, `clientLeft` would be not `25`, but with the scrollbar width `25 + 16 = 41`.

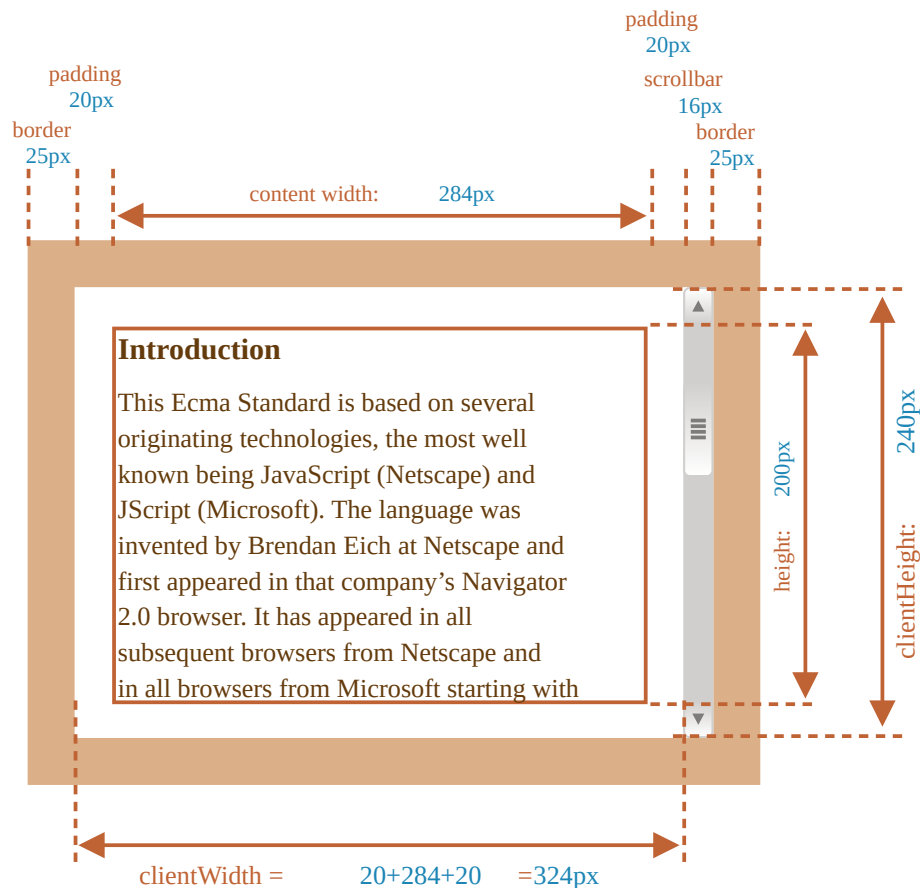
Here's the example in hebrew:



clientWidth/Height

These properties provide the size of the area inside the element borders.

They include the content width together with paddings, but without the scrollbar:

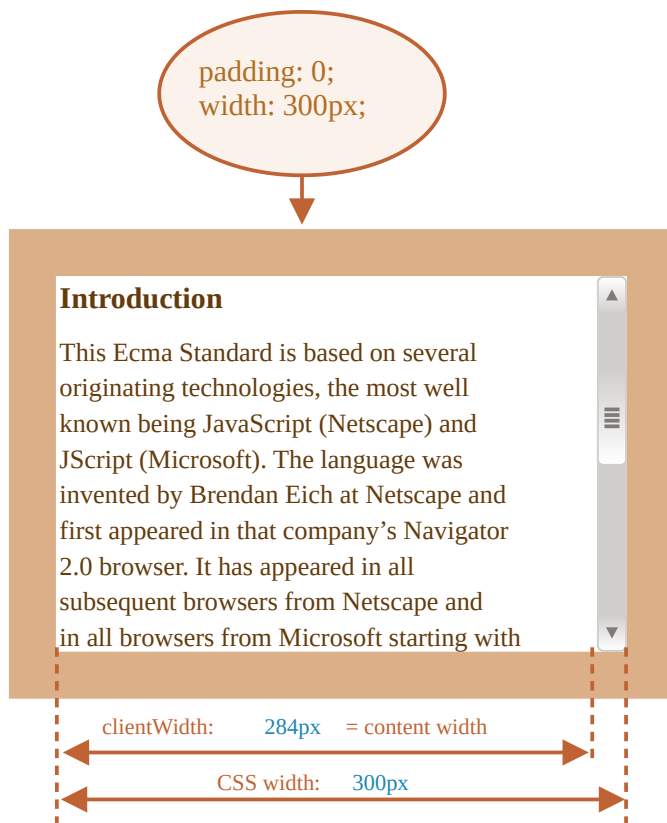


On the picture above let's first consider `clientHeight`.

There's no horizontal scrollbar, so it's exactly the sum of what's inside the borders: CSS-height `200px` plus top and bottom paddings ($2 * 20px$) total `240px`.

Now `clientWidth` – here the content width is not `300px`, but `284px`, because `16px` are occupied by the scrollbar. So the sum is `284px` plus left and right paddings, total `324px`.

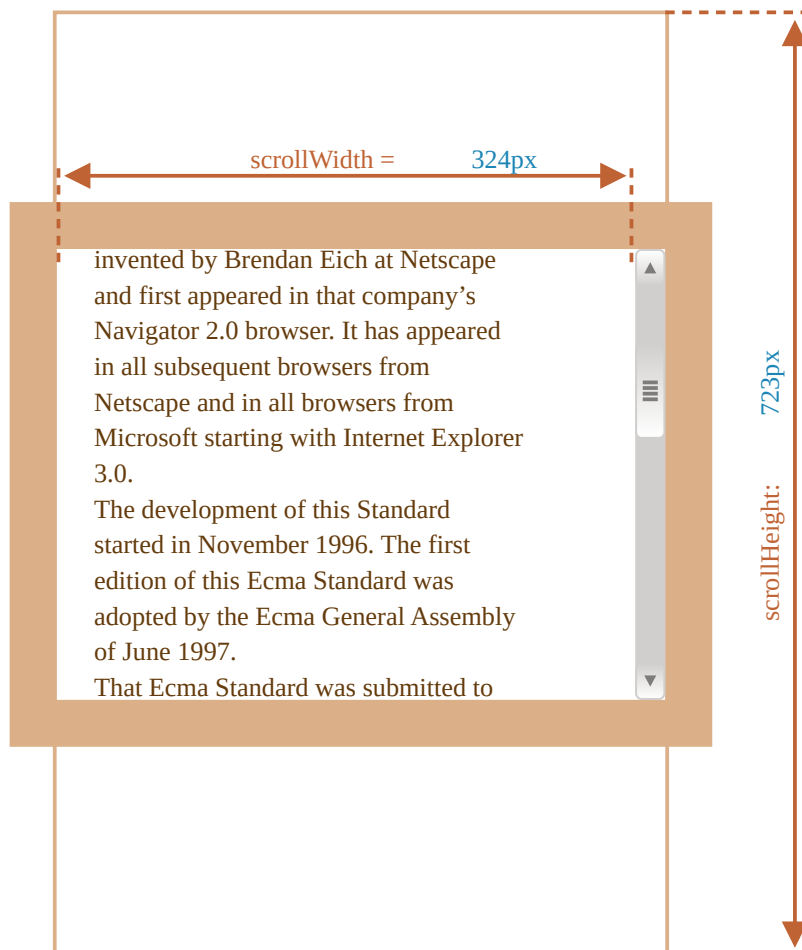
If there are no paddings, then `clientWidth/Height` is exactly the content area, inside the borders and the scrollbar (if any).



So when there's no padding we can use `clientWidth/clientHeight` to get the content area size.

scrollWidth/Height

These properties are like `clientWidth/clientHeight`, but they also include the scrolled out (hidden) parts:



On the picture above:

- `scrollHeight = 723` – is the full inner height of the content area including the scrolled out parts.
- `scrollWidth = 324` – is the full inner width, here we have no horizontal scroll, so it equals `clientWidth`.

We can use these properties to expand the element wide to its full width/height.

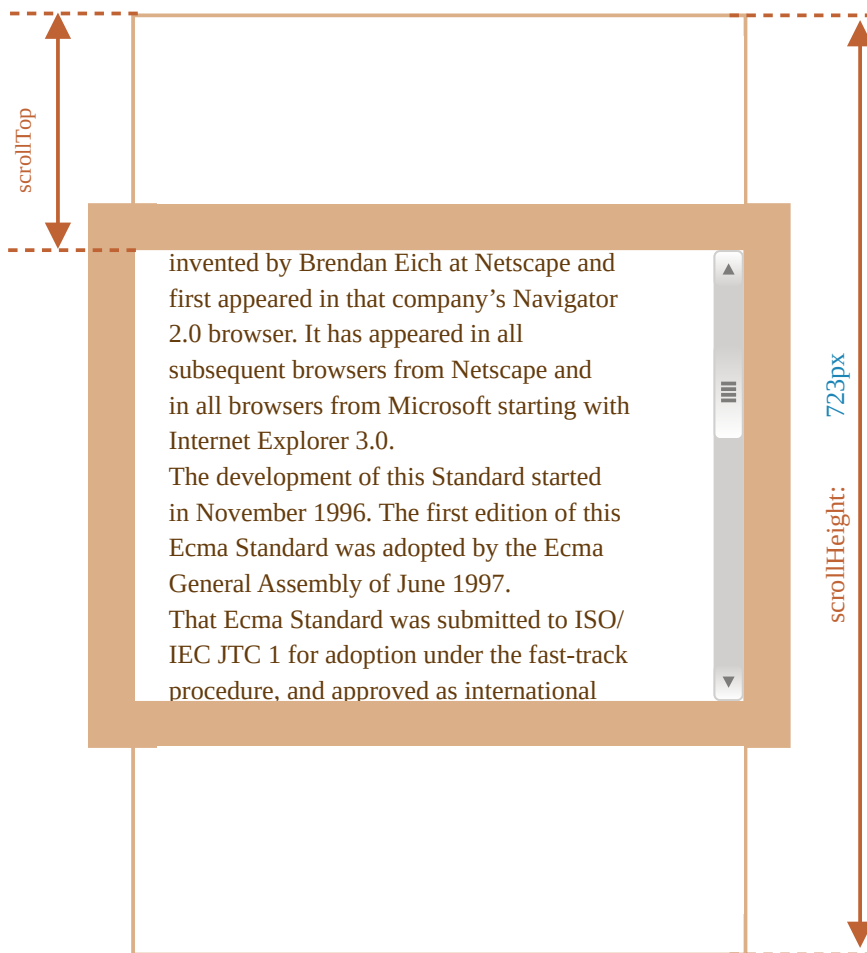
Like this:

```
// expand the element to the full content height
element.style.height = `${element.scrollHeight}px`;
```

scrollLeft/scrollTop

Properties `scrollLeft/scrollTop` are the width/height of the hidden, scrolled out part of the element.

On the picture below we can see `scrollHeight` and `scrollTop` for a block with a vertical scroll.



In other words, `scrollTop` is “how much is scrolled up”.

i `scrollLeft/scrollTop` can be modified

Most of the geometry properties here are read-only, but `scrollLeft/scrollTop` can be changed, and the browser will scroll the element.

Setting `scrollTop` to `0` or a big value, such as `1e9` will make the element scroll to the very top/bottom respectively.

Don't take width/height from CSS

We've just covered geometry properties of DOM elements, that can be used to get widths, heights and calculate distances.

But as we know from the chapter [Styles and classes](#), we can read CSS-height and width using `getComputedStyle`.

So why not to read the width of an element with `getComputedStyle`, like this?

```
let elem = document.body;

alert( getComputedStyle(elem).width ); // show CSS width for elem
```

Why should we use geometry properties instead? There are two reasons:

1. First, CSS `width/height` depend on another property: `box-sizing` that defines “what is” CSS width and height. A change in `box-sizing` for CSS purposes may break such JavaScript.
2. Second, CSS `width/height` may be `auto`, for instance for an inline element:

```
<span id="elem">Hello!</span>

<script>
  alert( getComputedStyle(elem).width ); // auto
</script>
```

From the CSS standpoint, `width:auto` is perfectly normal, but in JavaScript we need an exact size in `px` that we can use in calculations. So here CSS width is useless.

And there’s one more reason: a scrollbar. Sometimes the code that works fine without a scrollbar becomes buggy with it, because a scrollbar takes the space from the content in some browsers. So the real width available for the content is *less* than CSS width. And `clientWidth/clientHeight` take that into account.

...But with `getComputedStyle(elem).width` the situation is different. Some browsers (e.g. Chrome) return the real inner width, minus the scrollbar, and some of them (e.g. Firefox) – CSS width (ignore the scrollbar). Such cross-browser differences is the reason not to use `getComputedStyle`, but rather rely on geometry properties.

Please note that the described difference is only about reading `getComputedStyle(...).width` from JavaScript, visually everything is correct.

Summary

Elements have the following geometry properties:

- `offsetParent` – is the nearest positioned ancestor or `td`, `th`, `table`, `body`.
- `offsetLeft/offsetTop` – coordinates relative to the upper-left edge of `offsetParent`.
- `offsetWidth/offsetHeight` – “outer” width/height of an element including borders.

- `clientLeft/clientTop` – the distances from the upper-left outer corner to the upper-left inner (content + padding) corner. For left-to-right OS they are always the widths of left/top borders. For right-to-left OS the vertical scrollbar is on the left so `clientLeft` includes its width too.
- `clientWidth/clientHeight` – the width/height of the content including paddings, but without the scrollbar.
- `scrollWidth/scrollHeight` – the width/height of the content, just like `clientWidth/clientHeight`, but also include scrolled-out, invisible part of the element.
- `scrollLeft/scrollTop` – width/height of the scrolled out upper part of the element, starting from its upper-left corner.

All properties are read-only except `scrollLeft/scrollTop` that make the browser scroll the element if changed.

✓ Tasks

What's the scroll from the bottom?

importance: 5

The `elem.scrollTop` property is the size of the scrolled out part from the top. How to get the size of the bottom scroll (let's call it `scrollBottom`)?

Write the code that works for an arbitrary `elem`.

P.S. Please check your code: if there's no scroll or the element is fully scrolled down, then it should return `0`.

[To solution](#)

What is the scrollbar width?

importance: 3

Write the code that returns the width of a standard scrollbar.

For Windows it usually varies between `12px` and `20px`. If the browser doesn't reserve any space for it (the scrollbar is half-translucent over the text, also happens), then it may be `0px`.

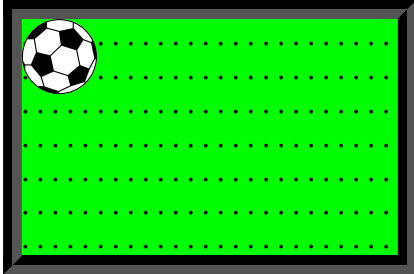
P.S. The code should work for any HTML document, do not depend on its content.

[To solution](#)

Place the ball in the field center

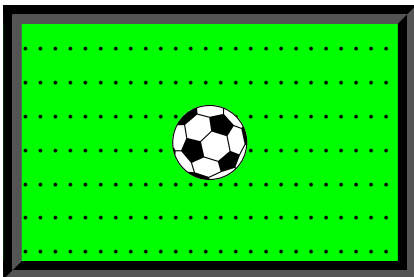
importance: 5

Here's how the source document looks:



What are coordinates of the field center?

Calculate them and use to place the ball into the center of the green field:



- The element should be moved by JavaScript, not CSS.
- The code should work with any ball size (`10` , `20` , `30` pixels) and any field size, not be bound to the given values.

P.S. Sure, centering could be done with CSS, but here we want exactly JavaScript. Further we'll meet other topics and more complex situations when JavaScript must be used. Here we do a “warm-up”.

[Open a sandbox for the task.](#) [↗](#)

[To solution](#)

The difference: CSS width versus clientWidth

importance: 5

What's the difference between `getComputedStyle(elem).width` and `elem.clientWidth`?

Give at least 3 differences. The more the better.

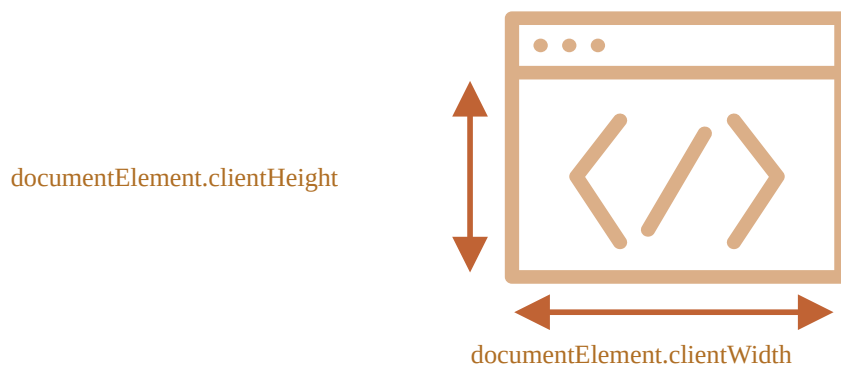
Window sizes and scrolling

How do we find the width and height of the browser window? How do we get the full width and height of the document, including the scrolled out part? How do we scroll the page using JavaScript?

For this type of information, we can use the root document element `document.documentElement`, that corresponds to the `<html>` tag. But there are additional methods and peculiarities to consider.

Width/height of the window

To get window width and height, we can use the `clientWidth/clientHeight` of `document.documentElement`:



⚠ Not `window.innerWidth/innerHeight`

Browsers also support properties like `window.innerWidth/innerHeight`. They look like what we want, so why not to use them instead?

If there exists a scrollbar, and it occupies some space, `clientWidth/clientHeight` provide the width/height without it (subtract it). In other words, they return the width/height of the visible part of the document, available for the content.

`window.innerWidth/innerHeight` includes the scrollbar.

If there's a scrollbar, and it occupies some space, then these two lines show different values:

```
alert( window.innerWidth ); // full window width
alert( document.documentElement.clientWidth ); // window width minus the scr
```

In most cases, we need the *available* window width in order to draw or position something within scrollbars (if there are any), so we should use `documentElement.clientWidth`.

⚠ DOCTYPE is important

Please note: top-level geometry properties may work a little bit differently when there's no `<!DOCTYPE HTML>` in HTML. Odd things are possible.

In modern HTML we should always write `DOCTYPE`.

Width/height of the document

Theoretically, as the root document element is `document.documentElement`, and it encloses all the content, we could measure the document's full size as `document.documentElement.scrollHeight`.

But on that element, for the whole page, these properties do not work as intended. In Chrome/Safari/Opera, if there's no scroll, then `documentElement.scrollHeight` may be even less than `documentElement.clientHeight`! Weird, right?

To reliably obtain the full document height, we should take the maximum of these properties:

```
let scrollHeight = Math.max(
  document.body.scrollHeight, document.documentElement.scrollHeight,
  document.body.offsetHeight, document.documentElement.offsetHeight,
  document.body.clientHeight, document.documentElement.clientHeight
);
```

```
alert('Full document height, with scrolled out part: ' + scrollHeight);
```

Why so? Better don't ask. These inconsistencies come from ancient times, not a "smart" logic.

Get the current scroll

DOM elements have their current scroll state in their `scrollLeft/scrollTop` properties.

For document scroll, `document.documentElement.scrollLeft/scrollTop` works in most browsers, except older WebKit-based ones, like Safari (bug [5991](#)), where we should use `document.body` instead of `document.documentElement`.

Luckily, we don't have to remember these peculiarities at all, because the scroll is available in the special properties, `window.pageXOffset/pageYOffset`:

```
alert('Current scroll from the top: ' + window.pageYOffset);
alert('Current scroll from the left: ' + window.pageXOffset);
```

These properties are read-only.

i Also available as `window` properties `scrollX` and `scrollY`

For historical reasons, both properties exist, but they are the same:

- `window.pageXOffset` is an alias of `window.scrollX`.
- `window.pageYOffset` is an alias of `window.scrollY`.

Scrolling: `scrollTo`, `scrollBy`, `scrollIntoView`

Important:

To scroll the page with JavaScript, its DOM must be fully built.

For instance, if we try to scroll the page with a script in `<head>`, it won't work.

Regular elements can be scrolled by changing `scrollTop/scrollLeft`.

We can do the same for the page using

`document.documentElement.scrollTop/scrollLeft` (except Safari, where `document.body.scrollTop/Left` should be used instead).

Alternatively, there's a simpler, universal solution: special methods `window.scrollBy(x,y)` and `window.scrollTo(pageX,pageY)`.

- The method `scrollBy(x,y)` scrolls the page *relative to its current position*. For instance, `scrollBy(0,10)` scrolls the page `10px` down.
- The method `scrollTo(pageX,pageY)` scrolls the page *to absolute coordinates*, so that the top-left corner of the visible part has coordinates `(pageX, pageY)` relative to the document's top-left corner. It's like setting `scrollLeft/scrollTop`.

To scroll to the very beginning, we can use `scrollTo(0,0)`.

These methods work for all browsers the same way.

scrollIntoView

For completeness, let's cover one more method: [elem.scrollIntoView\(top\)](#) .

The call to `elem.scrollIntoView(top)` scrolls the page to make `elem` visible. It has one argument:

- If `top=true` (that's the default), then the page will be scrolled to make `elem` appear on the top of the window. The upper edge of the element will be aligned with the window top.
- If `top=false`, then the page scrolls to make `elem` appear at the bottom. The bottom edge of the element will be aligned with the window bottom.

Forbid the scrolling

Sometimes we need to make the document “unscrollable”. For instance, when we need to cover the page with a large message requiring immediate attention, and we want the visitor to interact with that message, not with the document.

To make the document unscrollable, it's enough to set `document.body.style.overflow = "hidden"`. The page will “freeze” at its current scroll position.

We can use the same technique to freeze the scroll for other elements, not just for `document.body`.

The drawback of the method is that the scrollbar disappears. If it occupied some space, then that space is now free and the content “jumps” to fill it.

That looks a bit odd, but can be worked around if we compare `clientWidth` before and after the freeze. If it increased (the scrollbar disappeared), then add `padding` to `document.body` in place of the scrollbar to keep the content width the same.

Summary

Geometry:

- Width/height of the visible part of the document (content area width/height):
`document.documentElement.clientWidth/clientHeight`
- Width/height of the whole document, with the scrolled out part:

```
let scrollHeight = Math.max(
  document.body.scrollHeight, document.documentElement.scrollHeight,
  document.body.offsetHeight, document.documentElement.offsetHeight,
  document.body.clientHeight, document.documentElement.clientHeight
);
```

Scrolling:

- Read the current scroll: `window.pageYOffset/pageXOffset`.
- Change the current scroll:
 - `window.scrollTo(pageX, pageY)` – absolute coordinates,
 - `window.scrollBy(x, y)` – scroll relative the current place,
 - `elem.scrollIntoView(top)` – scroll to make `elem` visible (align with the top/bottom of the window).

Coordinates

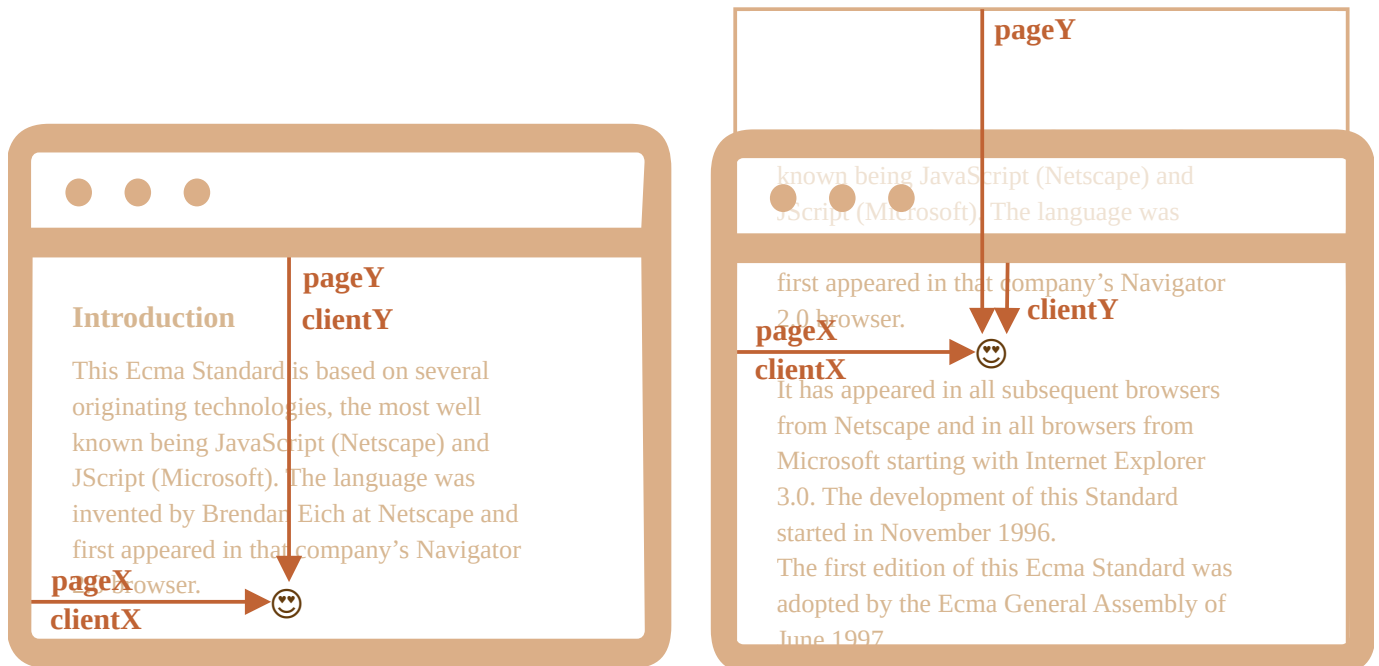
To move elements around we should be familiar with coordinates.

Most JavaScript methods deal with one of two coordinate systems:

1. **Relative to the window** – similar to `position:fixed`, calculated from the window top/left edge.
 - we'll denote these coordinates as `clientX/clientY`, the reasoning for such name will become clear later, when we study event properties.
2. **Relative to the document** – similar to `position:absolute` in the document root, calculated from the document top/left edge.
 - we'll denote them `pageX/pageY`.

When the page is scrolled to the very beginning, so that the top/left corner of the window is exactly the document top/left corner, these coordinates equal each other. But after the document shifts, window-relative coordinates of elements change, as elements move across the window, while document-relative coordinates remain the same.

On this picture we take a point in the document and demonstrate its coordinates before the scroll (left) and after it (right):



When the document scrolled:

- **pageY** – document-relative coordinate stayed the same, it's counted from the document top (now scrolled out).
- **clientY** – window-relative coordinate did change (the arrow became shorter), as the same point became closer to window top.

Element coordinates: `getBoundingClientRect`

The method `elem.getBoundingClientRect()` returns window coordinates for a minimal rectangle that encloses `elem` as an object of built-in `DOMRect` class.

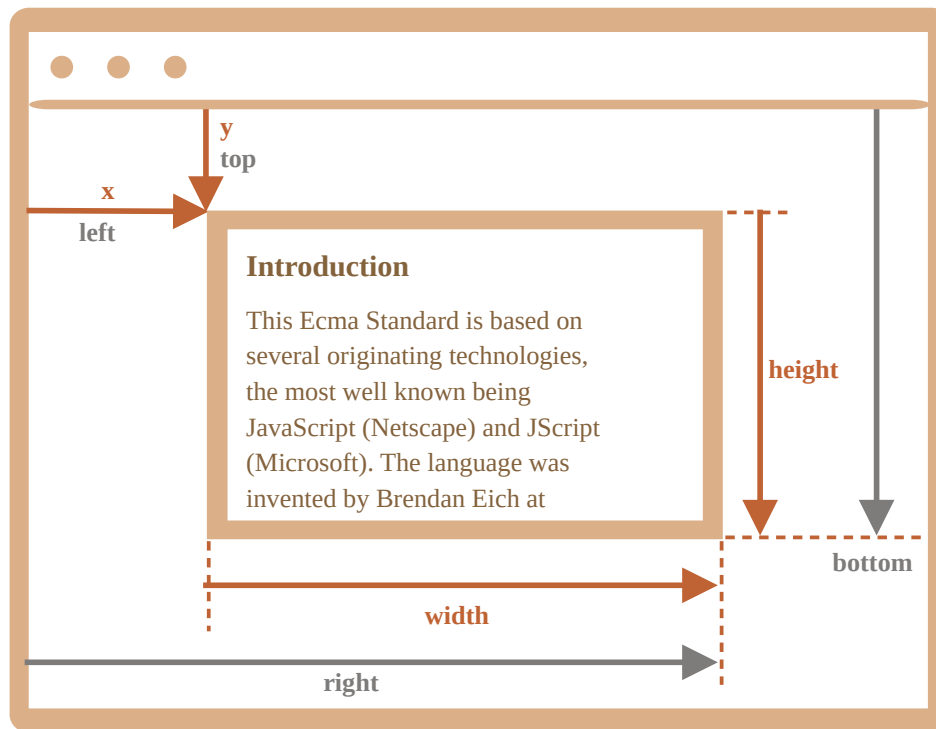
Main `DOMRect` properties:

- **x/y** – X/Y-coordinates of the rectangle origin relative to window,
- **width/height** – width/height of the rectangle (can be negative).

Additionally, there are derived properties:

- **top/bottom** – Y-coordinate for the top/bottom rectangle edge,
- **left/right** – X-coordinate for the left/right rectangle edge.

Here's the picture of `elem.getBoundingClientRect()` output:



As you can see, `x/y` and `width/height` fully describe the rectangle. Derived properties can be easily calculated from them:

- `left = x`
- `top = y`
- `right = x + width`
- `bottom = y + height`

Please note:

- Coordinates may be decimal fractions, such as `10.5`. That's normal, internally browser uses fractions in calculations. We don't have to round them when setting to `style.left/top`.
- Coordinates may be negative. For instance, if the page is scrolled so that `elem` is now above the window, then `elem.getBoundingClientRect().top` is negative.

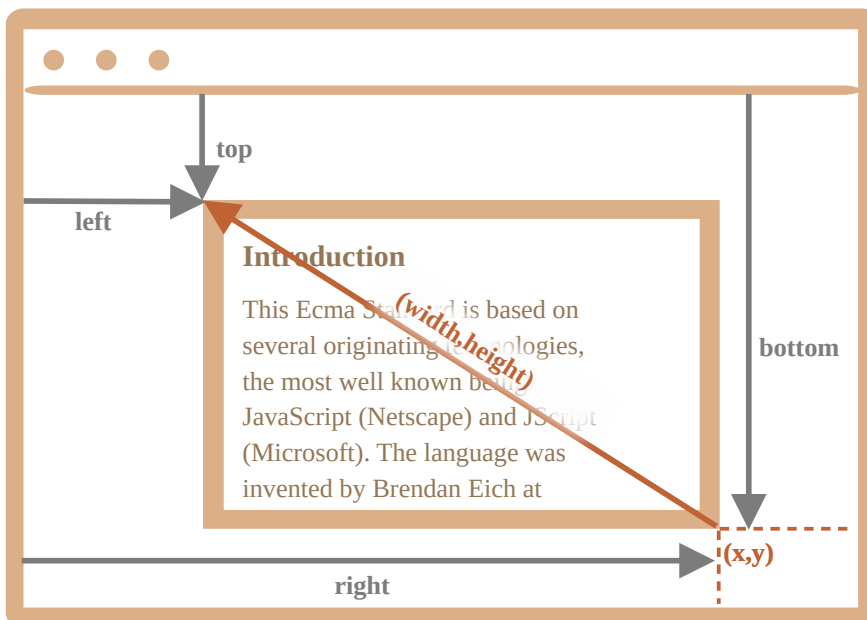
i Why derived properties are needed? Why does `top/left` exist if there's `x/y` ?

Mathematically, a rectangle is uniquely defined with its starting point `(x,y)` and the direction vector `(width,height)`. So the additional derived properties are for convenience.

Technically it's possible for `width/height` to be negative, that allows for "directed" rectangle, e.g. to represent mouse selection with properly marked start and end.

Negative `width/height` values mean that the rectangle starts at its bottom-right corner and then "grows" left-upwards.

Here's a rectangle with negative `width` and `height` (e.g. `width=-200`, `height=-100`):



As you can see, `left/top` do not equal `x/y` in such case.

In practice though, `elem.getBoundingClientRect()` always returns positive `width/height`, here we mention negative `width/height` only for you to understand why these seemingly duplicate properties are not actually duplicates.

⚠ Internet Explorer: no support for `x/y`

Internet Explorer doesn't support `x/y` properties for historical reasons.

So we can either make a polyfill (add getters in `DomRect.prototype`) or just use `top/left`, as they are always the same as `x/y` for positive `width/height`, in particular in the result of `elem.getBoundingClientRect()`.

⚠ Coordinates right/bottom are different from CSS position properties

There are obvious similarities between window-relative coordinates and CSS `position:fixed`.

But in CSS positioning, `right` property means the distance from the right edge, and `bottom` property means the distance from the bottom edge.

If we just look at the picture above, we can see that in JavaScript it is not so. All window coordinates are counted from the top-left corner, including these ones.

elementFromPoint(x, y)

The call to `document.elementFromPoint(x, y)` returns the most nested element at window coordinates `(x, y)`.

The syntax is:

```
let elem = document.elementFromPoint(x, y);
```

For instance, the code below highlights and outputs the tag of the element that is now in the middle of the window:

```
let centerX = document.documentElement.clientWidth / 2;
let centerY = document.documentElement.clientHeight / 2;

let elem = document.elementFromPoint(centerX, centerY);

elem.style.background = "red";
alert(elem.tagName);
```

As it uses window coordinates, the element may be different depending on the current scroll position.

 **For out-of-window coordinates the `elementFromPoint` returns `null`**

The method `document.elementFromPoint(x,y)` only works if `(x,y)` are inside the visible area.

If any of the coordinates is negative or exceeds the window width/height, then it returns `null`.

Here's a typical error that may occur if we don't check for it:

```
let elem = document.elementFromPoint(x, y);  
// if the coordinates happen to be out of the window, then elem = null  
elem.style.background = ''; // Error!
```

Using for “fixed” positioning

Most of time we need coordinates in order to position something.

To show something near an element, we can use `getBoundingClientRect` to get its coordinates, and then CSS `position` together with `left/top` (or `right/bottom`).

For instance, the function `createMessageUnder(elem, html)` below shows the message under `elem`:

```
let elem = document.getElementById("coords-show-mark");  
  
function createMessageUnder(elem, html) {  
  // create message element  
  let message = document.createElement('div');  
  // better to use a css class for the style here  
  message.style.cssText = "position:fixed; color: red";  
  
  // assign coordinates, don't forget "px"!  
  let coords = elem.getBoundingClientRect();  
  
  message.style.left = coords.left + "px";  
  message.style.top = coords.bottom + "px";  
  
  message.innerHTML = html;  
  
  return message;  
}  
  
// Usage:  
// add it for 5 seconds in the document  
let message = createMessageUnder(elem, 'Hello, world!');
```

```
document.body.append(message);  
setTimeout(() => message.remove(), 5000);
```

The code can be modified to show the message at the left, right, below, apply CSS animations to “fade it in” and so on. That’s easy, as we have all the coordinates and sizes of the element.

But note the important detail: when the page is scrolled, the message flows away from the button.

The reason is obvious: the message element relies on `position:fixed`, so it remains at the same place of the window while the page scrolls away.

To change that, we need to use document-based coordinates and `position:absolute`.

Document coordinates

Document-relative coordinates start from the upper-left corner of the document, not the window.

In CSS, window coordinates correspond to `position:fixed`, while document coordinates are similar to `position:absolute` on top.

We can use `position:absolute` and `top/left` to put something at a certain place of the document, so that it remains there during a page scroll. But we need the right coordinates first.

There’s no standard method to get the document coordinates of an element. But it’s easy to write it.

The two coordinate systems are connected by the formula:

- `pageY` = `clientY` + height of the scrolled-out vertical part of the document.
- `pageX` = `clientX` + width of the scrolled-out horizontal part of the document.

The function `getCoords(elem)` will take window coordinates from `elem.getBoundingClientRect()` and add the current scroll to them:

```
// get document coordinates of the element  
function getCoords(elem) {  
  let box = elem.getBoundingClientRect();  
  
  return {  
    top: box.top + window.pageYOffset,  
    right: box.right + window.pageXOffset,  
    bottom: box.bottom + window.pageYOffset,  
    left: box.left + window.pageXOffset  
  }  
}
```

```
};  
}
```

If in the example above we used it with `position: absolute`, then the message would stay near the element on scroll.

The modified `createMessageUnder` function:

```
function createMessageUnder(elem, html) {  
  let message = document.createElement('div');  
  message.style.cssText = "position: absolute; color: red";  
  
  let coords = getCoords(elem);  
  
  message.style.left = coords.left + "px";  
  message.style.top = coords.bottom + "px";  
  
  message.innerHTML = html;  
  
  return message;  
}
```

Summary

Any point on the page has coordinates:

1. Relative to the window – `elem.getBoundingClientRect()`.
2. Relative to the document – `elem.getBoundingClientRect()` plus the current page scroll.

Window coordinates are great to use with `position: fixed`, and document coordinates do well with `position: absolute`.

Both coordinate systems have their pros and cons; there are times we need one or the other one, just like CSS `position absolute` and `fixed`.

✓ Tasks

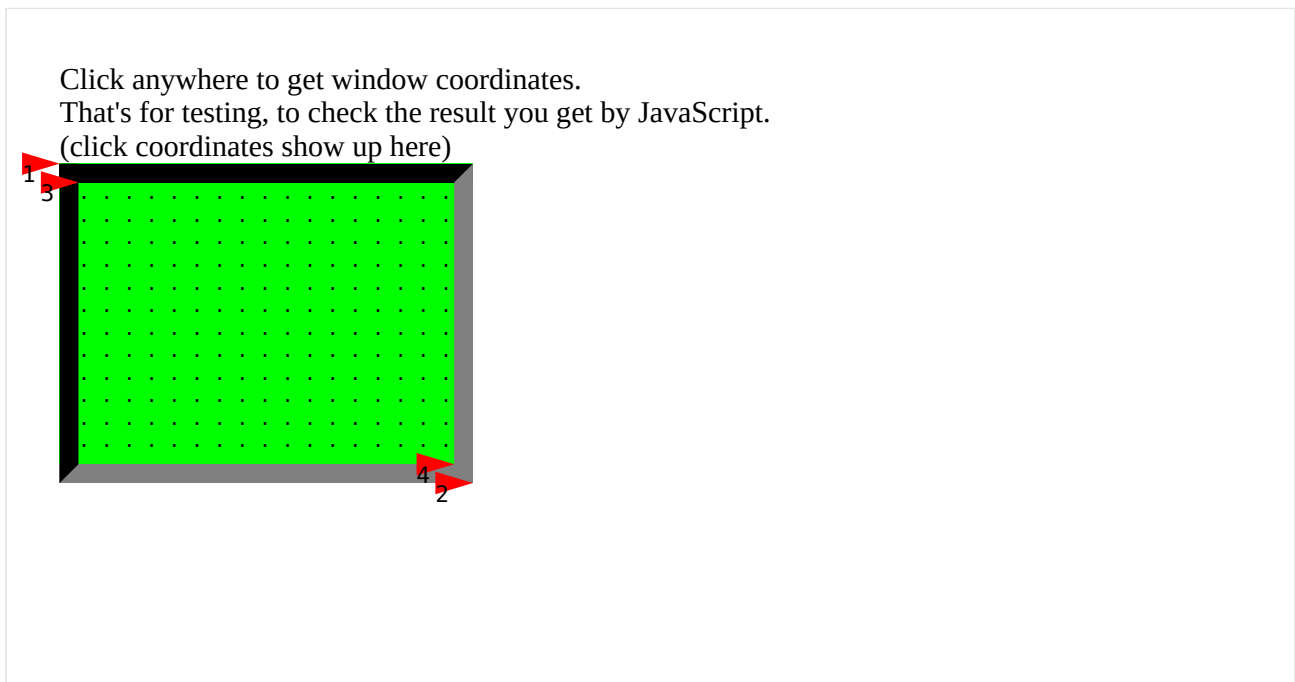
Find window coordinates of the field

importance: 5

In the iframe below you can see a document with the green “field”.

Use JavaScript to find window coordinates of corners pointed by with arrows.

There's a small feature implemented in the document for convenience. A click at any place shows coordinates there.



Your code should use DOM to get window coordinates of:

1. Upper-left, outer corner (that's simple).
2. Bottom-right, outer corner (simple too).
3. Upper-left, inner corner (a bit harder).
4. Bottom-right, inner corner (there are several ways, choose one).

The coordinates that you calculate should be the same as those returned by the mouse click.

P.S. The code should also work if the element has another size or border, not bound to any fixed values.

[Open a sandbox for the task.](#) [↗](#)

[To solution](#)

Show a note near the element

importance: 5

Create a function `positionAt(anchor, position, elem)` that positions `elem`, depending on `position` near `anchor` element.

The `position` must be a string with any one of 3 values:

- `"top"` – position `elem` right above `anchor`

- "right" – position `elem` immediately at the right of `anchor`
- "bottom" – position `elem` right below `anchor`

It's used inside function `showNote(anchor, position, html)`, provided in the task source code, that creates a "note" element with given `html` and shows it at the given `position` near the `anchor`.

Here's the demo of notes:

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit sint atque dolorum fuga ad incidunt voluptatum error fugiat animi amet! Odio temporibus nulla id unde quaerat dignissimos enim nisi rem provident molestias sit tempore omnis recusandae esse sequi officia sapiente.

note above

“

Teacher: Why are you late?
Student: There was a man who lost a hundred dollar bill.
Teacher: That's nice. Were you helping him look for it?
Student: No. I was standing on it.

note at the right

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit sint atque dolorum fuga ad incidunt voluptatum error fugiat animi amet! Odio temporibus nulla id unde quaerat dignissimos enim nisi rem provident molestias sit tempore omnis recusandae esse sequi officia sapiente.

note below

[Open a sandbox for the task.](#)

[To solution](#)

Show a note near the element (absolute)

importance: 5

Modify the solution of the [previous task](#) so that the note uses `position: absolute` instead of `position: fixed`.

That will prevent its "runaway" from the element when the page scrolls.

Take the solution of that task as a starting point. To test the scroll, add the style `<body style="height: 2000px">`.

[To solution](#)

Position the note inside (absolute)

importance: 5

Extend the previous task [Show a note near the element \(absolute\)](#): teach the function `positionAt(anchor, position, elem)` to insert `elem` inside the `anchor`.

New values for `position`:

- `top-out`, `right-out`, `bottom-out` – work the same as before, they insert the `elem` over/right/under `anchor`.
- `top-in`, `right-in`, `bottom-in` – insert `elem` inside the `anchor`: stick it to the upper/right/bottom edge.

For instance:

```
// shows the note above blockquote
positionAt(blockquote, "top-out", note);

// shows the note inside blockquote, at the top
positionAt(blockquote, "top-in", note);
```

The result:

>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit sint atque dolorum fuga ad
incididunt voluptatum error fugiat animi amet! Odio temporibus nulla id unde quaerat dignissimos enim
nisi rem provident molestias sit tempore omnis recusandae esse sequi officia sapiente.

note top-out

note top-in

note right-out

Teacher: Why are you late?
Student: There was a man who lost a hundred dollar bill.
Teacher: That's nice. Were you helping him look for it?
Student: No. I was standing on it.

note bottom-in

>Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit sint atque dolorum fuga ad
incididunt voluptatum error fugiat animi amet! Odio temporibus nulla id unde quaerat dignissimos enim
nisi rem provident molestias sit tempore omnis recusandae esse sequi officia sapiente.

As the source code, take the solution of the task [Show a note near the element \(absolute\)](#).

[To solution](#)

Introduction to Events

An introduction to browser events, event properties and handling patterns.

Introduction to browser events

An *event* is a signal that something has happened. All DOM nodes generate such signals (but events are not limited to DOM).

Here's a list of the most useful DOM events, just to take a look at:

Mouse events:

- `click` – when the mouse clicks on an element (touchscreen devices generate it on a tap).
- `contextmenu` – when the mouse right-clicks on an element.
- `mouseover` / `mouseout` – when the mouse cursor comes over / leaves an element.
- `mousedown` / `mouseup` – when the mouse button is pressed / released over an element.
- `mousemove` – when the mouse is moved.

Keyboard events:

- `keydown` and `keyup` – when a keyboard key is pressed and released.

Form element events:

- `submit` – when the visitor submits a `<form>`.
- `focus` – when the visitor focuses on an element, e.g. on an `<input>`.

Document events:

- `DOMContentLoaded` – when the HTML is loaded and processed, DOM is fully built.

CSS events:

- `transitionend` – when a CSS-animation finishes.

There are many other events. We'll get into more details of particular events in upcoming chapters.

Event handlers

To react on events we can assign a *handler* – a function that runs in case of an event.

Handlers are a way to run JavaScript code in case of user actions.

There are several ways to assign a handler. Let's see them, starting from the simplest one.

HTML-attribute

A handler can be set in HTML with an attribute named `on<event>`.

For instance, to assign a `click` handler for an `input`, we can use `onclick`, like here:

```
<input value="Click me" onclick="alert('Click!')" type="button">
```

On mouse click, the code inside `onclick` runs.

Please note that inside `onclick` we use single quotes, because the attribute itself is in double quotes. If we forget that the code is inside the attribute and use double quotes inside, like this: `onclick="alert("Click!")"`, then it won't work right.

An HTML-attribute is not a convenient place to write a lot of code, so we'd better create a JavaScript function and call it there.

Here a click runs the function `countRabbits()`:

```
<script>
  function countRabbits() {
    for(let i=1; i<=3; i++) {
      alert("Rabbit number " + i);
    }
  }
</script>

<input type="button" onclick="countRabbits()" value="Count rabbits!">
```

Count rabbits!

As we know, HTML attribute names are not case-sensitive, so `ONCLICK` works as well as `onClick` and `onCLICK` ... But usually attributes are lowercased: `onclick`.

DOM property

We can assign a handler using a DOM property `on<event>`.

For instance, `elem.onclick`:

```
<input id="elem" type="button" value="Click me">
<script>
  elem.onclick = function() {
    alert('Thank you');
  };
</script>
```

Click me

If the handler is assigned using an HTML-attribute then the browser reads it, creates a new function from the attribute content and writes it to the DOM property.

So this way is actually the same as the previous one.

These two code pieces work the same:

1. Only HTML:

```
<input type="button" onclick="alert('Click!')" value="Button">
```

Button

2. HTML + JS:

```
<input type="button" id="button" value="Button">
<script>
  button.onclick = function() {
    alert('Click!');
  };
</script>
```

Button

In the first example, the HTML attribute is used to initialize the `button.onclick`, while in the second example – the script, that's all the difference.

As there's only one `onclick` property, we can't assign more than one event handler.

In the example below adding a handler with JavaScript overwrites the existing handler:

```
<input type="button" id="elem" onclick="alert('Before')" value="Click me">
<script>
  elem.onclick = function() { // overwrites the existing handler
    alert('After'); // only this will be shown
  };
</script>
```

Click me

To remove a handler – assign `elem.onclick = null`.

Accessing the element: this

The value of `this` inside a handler is the element. The one which has the handler on it.

In the code below `button` shows its contents using `this.innerHTML`:

```
<button onclick="alert(this.innerHTML)">Click me</button>
```

Click me

Possible mistakes

If you're starting to work with events – please note some subtleties.

We can set an existing function as a handler:

```
function sayThanks() {  
    alert('Thanks!');  
}  
  
elem.onclick = sayThanks;
```

But be careful: the function should be assigned as `sayThanks`, not `sayThanks()`.

```
// right  
button.onclick = sayThanks;  
  
// wrong  
button.onclick = sayThanks();
```

If we add parentheses, then `sayThanks()` becomes a function call. So the last line actually takes the *result* of the function execution, that is `undefined` (as the function returns nothing), and assigns it to `onclick`. That doesn't work.

...On the other hand, in the markup we do need the parentheses:

```
<input type="button" id="button" onclick="sayThanks()">
```

The difference is easy to explain. When the browser reads the attribute, it creates a handler function with body from the attribute content.

So the markup generates this property:

```
button.onclick = function() {  
    sayThanks(); // <-- the attribute content goes here  
};
```

Don't use `setAttribute` for handlers.

Such a call won't work:

```
// a click on <body> will generate errors,  
// because attributes are always strings, function becomes a string  
document.body.setAttribute('onclick', function() { alert(1) });
```

DOM-property case matters.

Assign a handler to `elem.onclick`, not `elem.ONCLICK`, because DOM properties are case-sensitive.

`addEventListener`

The fundamental problem of the aforementioned ways to assign handlers is that we *can't assign multiple handlers to one event*.

Let's say, one part of our code wants to highlight a button on click, and another one wants to show a message on the same click.

We'd like to assign two event handlers for that. But a new DOM property will overwrite the existing one:

```
input.onclick = function() { alert(1); }  
// ...  
input.onclick = function() { alert(2); } // replaces the previous handler
```

Developers of web standards understood that long ago and suggested an alternative way of managing handlers using the special methods `addEventListener` and `removeEventListener` which aren't bound by such constraint.

The syntax to add a handler:

```
element.addEventListener(event, handler, [options]);
```

`event`

Event name, e.g. `"click"`.

`handler`

The handler function.

options

An additional optional object with properties:

- `once`: if `true`, then the listener is automatically removed after it triggers.
- `capture`: the phase where to handle the event, to be covered later in the chapter [Bubbling and capturing](#). For historical reasons, `options` can also be `false/true`, that's the same as `{capture: false/true}`.
- `passive`: if `true`, then the handler will not call `preventDefault()`, we'll explain that later in [Browser default actions](#).

To remove the handler, use `removeEventListener`:

```
element.removeEventListener(event, handler, [options]);
```

⚠ Removal requires the same function

To remove a handler we should pass exactly the same function as was assigned.

This doesn't work:

```
elem.addEventListener( "click" , () => alert('Thanks!'));  
// ....  
elem.removeEventListener( "click", () => alert('Thanks!'));
```

The handler won't be removed, because `removeEventListener` gets another function – with the same code, but that doesn't matter, as it's a different function object.

Here's the right way:

```
function handler() {  
  alert( 'Thanks!' );  
}  
  
input.addEventListener("click", handler);  
// ....  
input.removeEventListener("click", handler);
```

Please note – if we don't store the function in a variable, then we can't remove it. There's no way to “read back” handlers assigned by `addEventListener`.

Multiple calls to `addEventListener` allow it to add multiple handlers, like this:

```

<input id="elem" type="button" value="Click me"/>

<script>
  function handler1() {
    alert('Thanks!');
  };

  function handler2() {
    alert('Thanks again!');
  }

  elem.onclick = () => alert("Hello");
  elem.addEventListener("click", handler1); // Thanks!
  elem.addEventListener("click", handler2); // Thanks again!
</script>

```

As we can see in the example above, we can set handlers *both* using a DOM-property and `addEventListener`. But generally we use only one of these ways.

For some events, handlers only work with `addEventListener`

There exist events that can't be assigned via a DOM-property. Only with `addEventListener`.

For instance, the `DOMContentLoaded` event, that triggers when the document is loaded and the DOM has been built.

```

// will never run
document.onDOMContentLoaded = function() {
  alert("DOM built");
};

```

```

// this way it works
document.addEventListener("DOMContentLoaded", function() {
  alert("DOM built");
});

```

So `addEventListener` is more universal. Although, such events are an exception rather than the rule.

Event object

To properly handle an event we'd want to know more about what's happened. Not just a "click" or a "keydown", but what were the pointer coordinates? Which key was pressed? And so on.

When an event happens, the browser creates an *event object*, puts details into it and passes it as an argument to the handler.

Here's an example of getting pointer coordinates from the event object:

```
<input type="button" value="Click me" id="elem">

<script>
  elem.onclick = function(event) {
    // show event type, element and coordinates of the click
    alert(event.type + " at " + event.currentTarget);
    alert("Coordinates: " + event.clientX + ":" + event.clientY);
  };
</script>
```

Some properties of `event` object:

`event.type`

Event type, here it's `"click"`.

`event.currentTarget`

Element that handled the event. That's exactly the same as `this`, unless the handler is an arrow function, or its `this` is bound to something else, then we can get the element from `event.currentTarget`.

`event.clientX` / `event.clientY`

Window-relative coordinates of the cursor, for pointer events.

There are more properties. Many of them depend on the event type: keyboard events have one set of properties, pointer events – another one, we'll study them later when as we move on to the details of different events.

The event object is also available in HTML handlers

If we assign a handler in HTML, we can also use the `event` object, like this:

```
<input type="button" onclick="alert(event.type)" value="Event type">
```

Event type

That's possible because when the browser reads the attribute, it creates a handler like this: `function(event) { alert(event.type) }`. That is: its first argument is called `"event"`, and the body is taken from the attribute.

Object handlers: `handleEvent`

We can assign not just a function, but an object as an event handler using `addEventListener`. When an event occurs, its `handleEvent` method is called.

For instance:

```
<button id="elem">Click me</button>

<script>
  let obj = {
    handleEvent(event) {
      alert(event.type + " at " + event.currentTarget);
    }
  };

  elem.addEventListener('click', obj);
</script>
```

As we can see, when `addEventListener` receives an object as the handler, it calls `obj.handleEvent(event)` in case of an event.

We could also use objects of a custom class, like this:

```
<button id="elem">Click me</button>

<script>
  class Menu {
    handleEvent(event) {
      switch(event.type) {
        case 'mousedown':
          elem.innerHTML = "Mouse button pressed";
          break;
        case 'mouseup':
          elem.innerHTML += "...and released.";
          break;
      }
    }
  }

  let menu = new Menu();

  elem.addEventListener('mousedown', menu);
  elem.addEventListener('mouseup', menu);
</script>
```

Here the same object handles both events. Please note that we need to explicitly setup the events to listen using `addEventListener`. The `menu` object only gets `mousedown` and `mouseup` here, not any other types of events.

The method `handleEvent` does not have to do all the job by itself. It can call other event-specific methods instead, like this:

```
<button id="elem">Click me</button>

<script>
  class Menu {
    handleEvent(event) {
      // mousedown -> onMousedown
      let method = 'on' + event.type[0].toUpperCase() + event.type.slice(1);
      this[method](event);
    }

    onMousedown() {
      elem.innerHTML = "Mouse button pressed";
    }

    onMouseup() {
      elem.innerHTML += "...and released.";
    }
  }

  let menu = new Menu();
  elem.addEventListener('mousedown', menu);
  elem.addEventListener('mouseup', menu);
</script>
```

Now event handlers are clearly separated, that may be easier to support.

Summary

There are 3 ways to assign event handlers:

1. HTML attribute: `onclick="..."`.
2. DOM property: `elem.onclick = function`.
3. Methods: `elem.addEventListener(event, handler[, phase])` to add, `removeEventListener` to remove.

HTML attributes are used sparingly, because JavaScript in the middle of an HTML tag looks a little bit odd and alien. Also can't write lots of code in there.

DOM properties are ok to use, but we can't assign more than one handler of the particular event. In many cases that limitation is not pressing.

The last way is the most flexible, but it is also the longest to write. There are few events that only work with it, for instance `transitionend` and `DOMContentLoaded` (to be covered). Also `addEventListener` supports objects as event handlers. In that case the method `handleEvent` is called in case of the event.

No matter how you assign the handler – it gets an event object as the first argument. That object contains the details about what's happened.

We'll learn more about events in general and about different types of events in the next chapters.

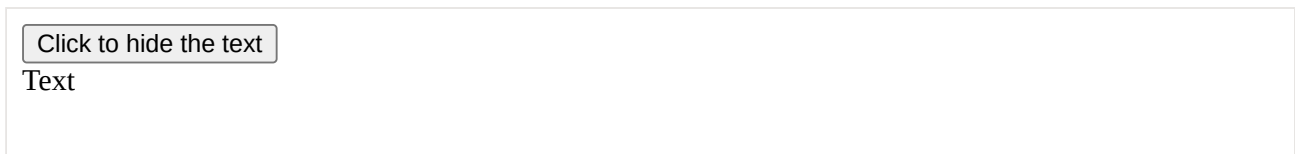
✓ Tasks

Hide on click

importance: 5

Add JavaScript to the `button` to make `<div id="text">` disappear when we click it.

The demo:



[Open a sandbox for the task.](#) ↗

[To solution](#)

Hide self

importance: 5

Create a button that hides itself on click.

[To solution](#)

Which handlers run?

importance: 5

There's a button in the variable. There are no handlers on it.

Which handlers run on click after the following code? Which alerts show up?

```
button.addEventListener("click", () => alert("1"));

button.removeEventListener("click", () => alert("1"));

button.onclick = () => alert(2);
```

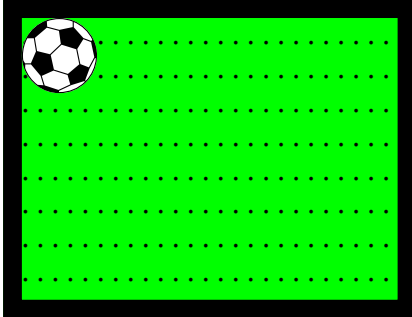
[To solution](#)

Move the ball across the field

importance: 5

Move the ball across the field to a click. Like this:

Click on a field to move the ball there.



Requirements:

- The ball center should come exactly under the pointer on click (if possible without crossing the field edge).
- CSS-animation is welcome.
- The ball must not cross field boundaries.
- When the page is scrolled, nothing should break.

Notes:

- The code should also work with different ball and field sizes, not be bound to any fixed values.
- Use properties `event.clientX/event.clientY` for click coordinates.

[Open a sandbox for the task.](#) [↗](#)

[To solution](#)

Create a sliding menu

importance: 5

Create a menu that opens/collapses on click:

► Sweeties (click me)!

P.S. HTML/CSS of the source document is to be modified.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Add a closing button

importance: 5

There's a list of messages.

Use JavaScript to add a closing button to the right-upper corner of each message.

The result should look like this:

Horse

[x]

The horse is one of two extant subspecies of *Equus ferus*. It is an odd-toed ungulate mammal belonging to the taxonomic family Equidae. The horse has evolved over the past 45 to 55 million years from a small multi-toed creature, *Eohippus*, into the large, single-toed animal of today.

Donkey

[x]

The donkey or ass (*Equus africanus asinus*) is a domesticated member of the horse family, Equidae. The wild ancestor of the donkey is the African wild ass, *E. africanus*. The donkey has been used as a working animal for at least 5000 years.

Cat

[x]

The domestic cat (Latin: *Felis catus*) is a small, typically furry, carnivorous mammal. They are often called house cats when kept as indoor pets or simply cats when there is no need to distinguish them from other felids and felines. Cats are often valued by humans for companionship and for their ability to hunt vermin.

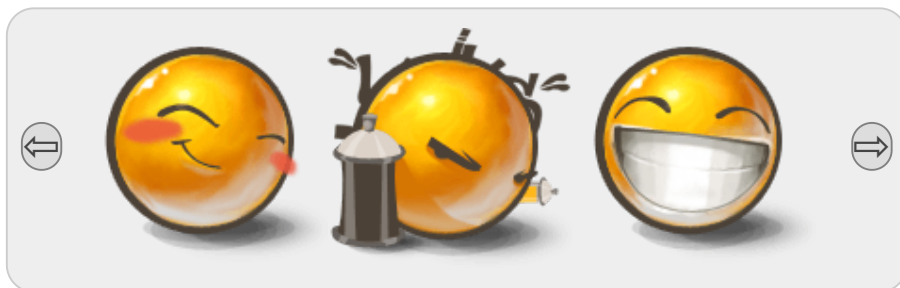
[Open a sandbox for the task.](#) ↗

[To solution](#)

Carousel

importance: 4

Create a “carousel” – a ribbon of images that can be scrolled by clicking on arrows.



Later we can add more features to it: infinite scrolling, dynamic loading etc.

P.S. For this task HTML/CSS structure is actually 90% of the solution.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Bubbling and capturing

Let's start with an example.

This handler is assigned to `<div>`, but also runs if you click any nested tag like `<em>` or `<code>`:

```
<div onclick="alert('The handler!')">
  <em>If you click on <code>EM</code>, the handler on <code>DIV</code> runs.</em>
</div>
```

If you click on EM, the handler on DIV runs.

Isn't it a bit strange? Why does the handler on `<div>` run if the actual click was on `<em>`?

Bubbling

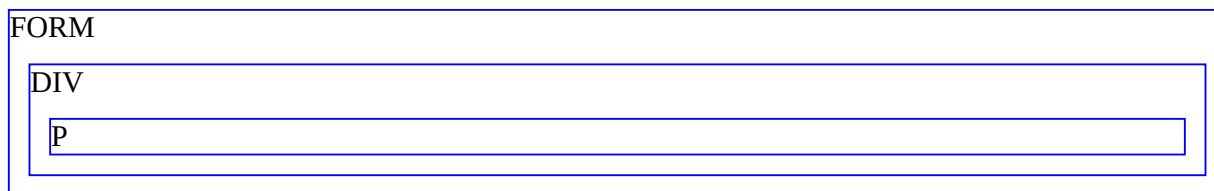
The bubbling principle is simple.

When an event happens on an element, it first runs the handlers on it, then on its parent, then all the way up on other ancestors.

Let's say we have 3 nested elements `FORM > DIV > P` with a handler on each of them:

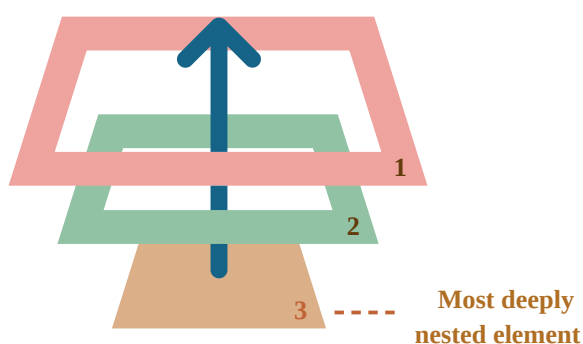
```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form onclick="alert('form')">FORM
  <div onclick="alert('div')">DIV
    <p onclick="alert('p')">P</p>
  </div>
</form>
```



A click on the inner `<p>` first runs `onclick`:

1. On that `<p>`.
2. Then on the outer `<div>`.
3. Then on the outer `<form>`.
4. And so on upwards till the `document` object.



So if we click on `<p>`, then we'll see 3 alerts: `p` → `div` → `form`.

The process is called “bubbling”, because events “bubble” from the inner element up through parents like a bubble in the water.

Almost all events bubble.

The key word in this phrase is “almost”.

For instance, a `focus` event does not bubble. There are other examples too, we'll meet them. But still it's an exception, rather than a rule, most events do bubble.

event.target

A handler on a parent element can always get the details about where it actually happened.

The most deeply nested element that caused the event is called a *target* element, accessible as `event.target`.

Note the differences from `this` (`= event.currentTarget`):

- `event.target` – is the “target” element that initiated the event, it doesn't change through the bubbling process.
- `this` – is the “current” element, the one that has a currently running handler on it.

For instance, if we have a single handler `form.onclick`, then it can “catch” all clicks inside the form. No matter where the click happened, it bubbles up to `<form>` and runs the handler.

In `form.onclick` handler:

- `this` (`= event.currentTarget`) is the `<form>` element, because the handler runs on it.
- `event.target` is the actual element inside the form that was clicked.

Check it out:

<https://plnkr.co/edit/vh3plE2ZelF0xKaI?p=preview> 

It's possible that `event.target` could equal `this` – it happens when the click is made directly on the `<form>` element.

Stopping bubbling

A bubbling event goes from the target element straight up. Normally it goes upwards till `<html>`, and then to `document` object, and some events even reach `window`, calling all handlers on the path.

But any handler may decide that the event has been fully processed and stop the bubbling.

The method for it is `event.stopPropagation()`.

For instance, here `body.onclick` doesn't work if you click on `<button>`:

```
<body onclick="alert(`the bubbling doesn't reach here`)">
  <button onclick="event.stopPropagation()">Click me</button>
</body>
```

Click me

`event.stopImmediatePropagation()`

If an element has multiple event handlers on a single event, then even if one of them stops the bubbling, the other ones still execute.

In other words, `event.stopPropagation()` stops the move upwards, but on the current element all other handlers will run.

To stop the bubbling and prevent handlers on the current element from running, there's a method `event.stopImmediatePropagation()`. After it no other handlers execute.

Don't stop bubbling without a need!

Bubbling is convenient. Don't stop it without a real need: obvious and architecturally well thought out.

Sometimes `event.stopPropagation()` creates hidden pitfalls that later may become problems.

For instance:

1. We create a nested menu. Each submenu handles clicks on its elements and calls `stopPropagation` so that the outer menu won't trigger.
2. Later we decide to catch clicks on the whole window, to track users' behavior (where people click). Some analytic systems do that. Usually the code uses `document.addEventListener('click'...)` to catch all clicks.
3. Our analytic won't work over the area where clicks are stopped by `stopPropagation`. Sadly, we've got a "dead zone".

There's usually no real need to prevent the bubbling. A task that seemingly requires that may be solved by other means. One of them is to use custom events, we'll cover them later. Also we can write our data into the `event` object in one handler and read it in another one, so we can pass to handlers on parents information about the processing below.

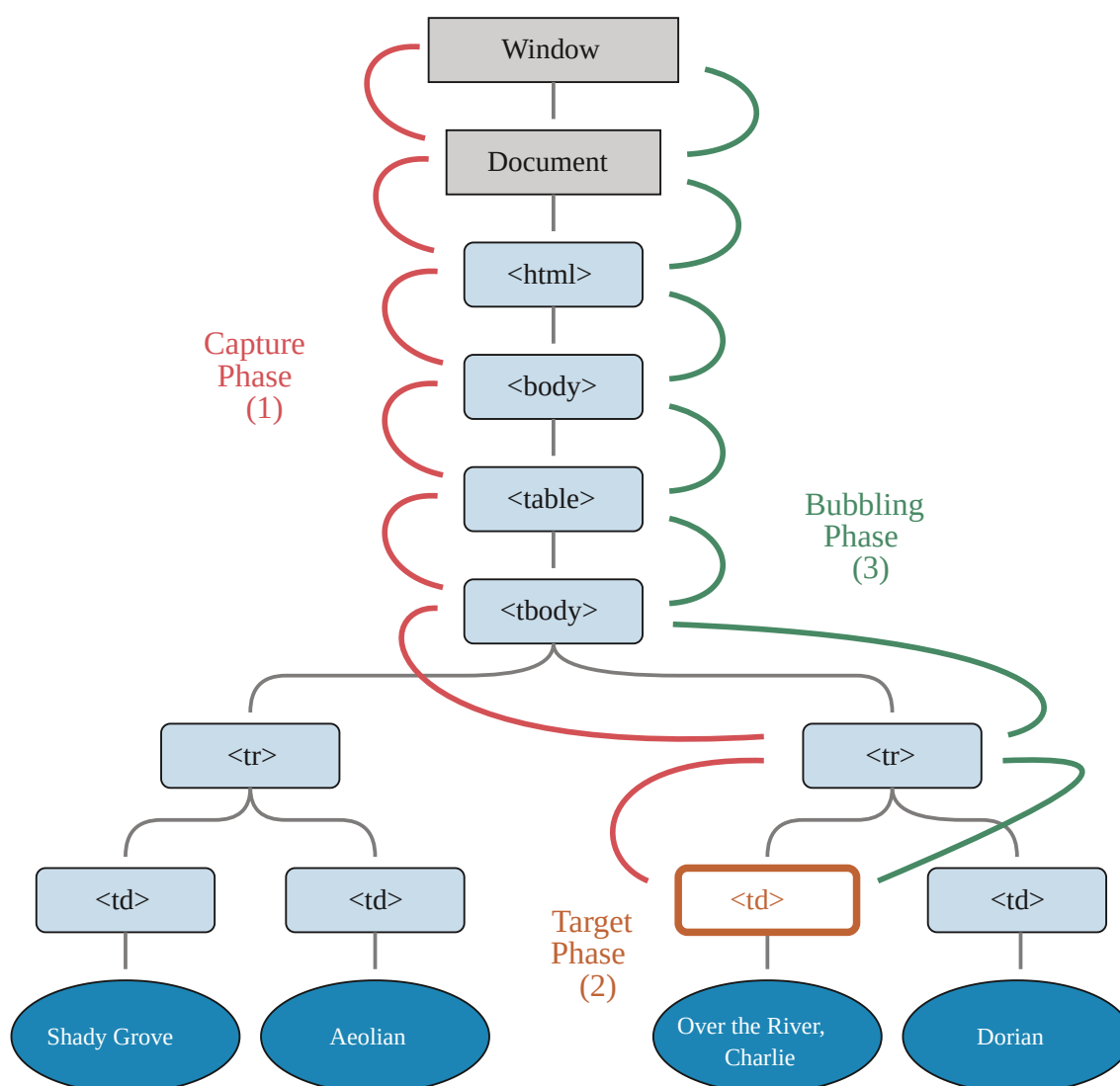
Capturing

There's another phase of event processing called "capturing". It is rarely used in real code, but sometimes can be useful.

The standard [DOM Events](#) describes 3 phases of event propagation:

1. Capturing phase – the event goes down to the element.
2. Target phase – the event reached the target element.
3. Bubbling phase – the event bubbles up from the element.

Here's the picture, taken from the specification, of the capturing (1), target (2) and bubbling (3) phases for a click event on a `<td>` inside a table:



That is: for a click on `<td>` the event first goes through the ancestors chain down to the element (capturing phase), then it reaches the target and triggers there (target phase), and then it goes up (bubbling phase), calling handlers on its way.

Until now, we only talked about bubbling, because the capturing phase is rarely used.

In fact, the capturing phase was invisible for us, because handlers added using `on<event>` -property or using HTML attributes or using two-argument `addEventListener(event, handler)` don't know anything about capturing, they only run on the 2nd and 3rd phases.

To catch an event on the capturing phase, we need to set the handler `capture` option to `true`:

```
elem.addEventListener(..., {capture: true})

// or, just "true" is an alias to {capture: true}
elem.addEventListener(..., true)
```

There are two possible values of the `capture` option:

- If it's `false` (default), then the handler is set on the bubbling phase.
- If it's `true`, then the handler is set on the capturing phase.

Note that while formally there are 3 phases, the 2nd phase ("target phase": the event reached the element) is not handled separately: handlers on both capturing and bubbling phases trigger at that phase.

Let's see both capturing and bubbling in action:

```
<style>
  body * {
    margin: 10px;
    border: 1px solid blue;
  }
</style>

<form>FORM
  <div>DIV
    <p>P</p>
  </div>
</form>

<script>
  for(let elem of document.querySelectorAll('*')) {
    elem.addEventListener("click", e => alert(`Capturing: ${elem.tagName}`), true);
    elem.addEventListener("click", e => alert(`Bubbling: ${elem.tagName}`));
  }
</script>
```

FORM

DIV

P

The code sets click handlers on every element in the document to see which ones are working.

If you click on `<p>`, then the sequence is:

1. `HTML` → `BODY` → `FORM` → `DIV` → `P` (capturing phase, the first listener):
2. `P` → `DIV` → `FORM` → `BODY` → `HTML` (bubbling phase, the second listener).

Please note, the `P` shows up twice, because we've set two listeners: capturing and bubbling. The target triggers at the end of the first and at the beginning of the second phase.

There's a property `event.eventPhase` that tells us the number of the phase on which the event was caught. But it's rarely used, because we usually know it in the handler.

i To remove the handler, `removeEventListener` needs the same phase

If we `addEventListener(..., true)`, then we should mention the same phase in `removeEventListener(..., true)` to correctly remove the handler.

i Listeners on the same element and same phase run in their set order

If we have multiple event handlers on the same phase, assigned to the same element with `addEventListener`, they run in the same order as they are created:

```
elem.addEventListener("click", e => alert(1)); // guaranteed to trigger first
elem.addEventListener("click", e => alert(2));
```

i The `event.stopPropagation()` during the capturing also prevents the bubbling

The `event.stopPropagation()` method and its sibling `event.stopImmediatePropagation()` can also be called on the capturing phase. Then not only the further capturing is stopped, but the bubbling as well.

In other words, normally the event goes first down (“capturing”) and then up (“bubbling”). But if `event.stopPropagation()` is called during the capturing phase, then the event travel stops, no bubbling will occur.

Summary

When an event happens – the most nested element where it happens gets labeled as the “target element” (`event.target`).

- Then the event moves down from the document root to `event.target`, calling handlers assigned with `addEventListener(..., true)` on the way (`true` is a shorthand for `{capture: true}`).
- Then handlers are called on the target element itself.
- Then the event bubbles up from `event.target` to the root, calling handlers assigned using `on<event>`, HTML attributes and `addEventListener` without the 3rd argument or with the 3rd argument `false/{capture: false}`.

Each handler can access `event` object properties:

- `event.target` – the deepest element that originated the event.
- `event.currentTarget` (`= this`) – the current element that handles the event (the one that has the handler on it)
- `event.eventPhase` – the current phase (capturing=1, target=2, bubbling=3).

Any event handler can stop the event by calling `event.stopPropagation()`, but that's not recommended, because we can't really be sure we won't need it above, maybe for completely different things.

The capturing phase is used very rarely, usually we handle events on bubbling. And there's a logical explanation for that.

In real world, when an accident happens, local authorities react first. They know best the area where it happened. Then higher-level authorities if needed.

The same for event handlers. The code that set the handler on a particular element knows maximum details about the element and what it does. A handler on a particular `<td>` may be suited for that exactly `<td>`, it knows everything about it, so it should get the chance first. Then its immediate parent also knows about the context, but a little bit less, and so on till the very top element that handles general concepts and runs the last one.

Bubbling and capturing lay the foundation for “event delegation” – an extremely powerful event handling pattern that we study in the next chapter.

Event delegation

Capturing and bubbling allow us to implement one of the most powerful event handling patterns called *event delegation*.

The idea is that if we have a lot of elements handled in a similar way, then instead of assigning a handler to each of them – we put a single handler on their common ancestor.

In the handler we get `event.target` to see where the event actually happened and handle it.

Let’s see an example – the [Ba-Gua diagram](#) reflecting the ancient Chinese philosophy.

Here it is:

Bagua Chart: Direction, Element, Color, Meaning		
Northwest Metal Silver Elders	North Water Blue Change	Northeast Earth Yellow Direction
West Metal Gold Youth	Center All Purple Harmony	East Wood Blue Future
Southwest Earth Brown Tranquility	South Fire Orange Fame	Southeast Wood Green Romance

The HTML is like this:

```
<table>
  <tr>
    <th colspan="3"><em>Bagua</em> Chart: Direction, Element, Color, Meaning</th>
  </tr>
  <tr>
    <td class="nw"><strong>Northwest</strong><br>Metal<br>Silver<br>Elders</td>
    <td class="n">...</td>
    <td class="ne">...</td>
  </tr>
  <tr>...2 more lines of this kind...</tr>
```

```
<tr>...2 more lines of this kind...</tr>
</table>
```

The table has 9 cells, but there could be 99 or 9999, doesn't matter.

Our task is to highlight a cell `<td>` on click.

Instead of assign an `onclick` handler to each `<td>` (can be many) – we'll setup the “catch-all” handler on `<table>` element.

It will use `event.target` to get the clicked element and highlight it.

The code:

```
let selectedTd;

table.onclick = function(event) {
  let target = event.target; // where was the click?

  if (target.tagName !== 'TD') return; // not on TD? Then we're not interested

  highlight(target); // highlight it
};

function highlight(td) {
  if (selectedTd) { // remove the existing highlight if any
    selectedTd.classList.remove('highlight');
  }
  selectedTd = td;
  selectedTd.classList.add('highlight'); // highlight the new td
}
```

Such a code doesn't care how many cells there are in the table. We can add/remove `<td>` dynamically at any time and the highlighting will still work.

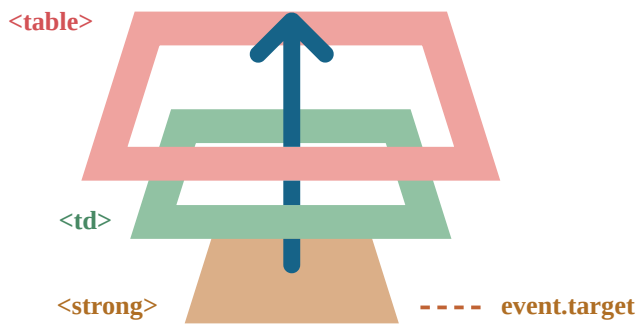
Still, there's a drawback.

The click may occur not on the `<td>`, but inside it.

In our case if we take a look inside the HTML, we can see nested tags inside `<td>`, like `<strong>`:

```
<td>
  <strong>Northwest</strong>
  ...
</td>
```

Naturally, if a click happens on that `<strong>` then it becomes the value of `event.target`.



In the handler `table.onclick` we should take such `event.target` and find out whether the click was inside `<td>` or not.

Here's the improved code:

```
table.onclick = function(event) {  
  let td = event.target.closest('td'); // (1)  
  
  if (!td) return; // (2)  
  
  if (!table.contains(td)) return; // (3)  
  
  highlight(td); // (4)  
};
```

Explanations:

1. The method `elem.closest(selector)` returns the nearest ancestor that matches the selector. In our case we look for `<td>` on the way up from the source element.
2. If `event.target` is not inside any `<td>`, then the call returns immediately, as there's nothing to do.
3. In case of nested tables, `event.target` may be a `<td>`, but lying outside of the current table. So we check if that's actually *our table's* `<td>`.
4. And, if it's so, then highlight it.

As the result, we have a fast, efficient highlighting code, that doesn't care about the total number of `<td>` in the table.

Delegation example: actions in markup

There are other uses for event delegation.

Let's say, we want to make a menu with buttons "Save", "Load", "Search" and so on. And there's an object with methods `save`, `load`, `search` ... How to match them?

The first idea may be to assign a separate handler to each button. But there's a more elegant solution. We can add a handler for the whole menu and `data-action` attributes for buttons that has the method to call:

```
<button data-action="save">Click to Save</button>
```

The handler reads the attribute and executes the method. Take a look at the working example:

```
<div id="menu">
  <button data-action="save">Save</button>
  <button data-action="load">Load</button>
  <button data-action="search">Search</button>
</div>

<script>
  class Menu {
    constructor(elem) {
      this._elem = elem;
      elem.onclick = this.onClick.bind(this); // (*)
    }

    save() {
      alert('saving');
    }

    load() {
      alert('loading');
    }

    search() {
      alert('searching');
    }

    onClick(event) {
      let action = event.target.dataset.action;
      if (action) {
        this[action]();
      }
    };
  }

  new Menu(menu);
</script>
```

Save Load Search

Please note that `this.onClick` is bound to `this` in `(*)`. That's important, because otherwise `this` inside it would reference the DOM element (`elem`), not the `Menu` object, and `this[action]` would not be what we need.

So, what advantages does delegation give us here?

- We don't need to write the code to assign a handler to each button. Just make a method and put it in the markup.
- The HTML structure is flexible, we can add/remove buttons at any time.

We could also use classes `.action-save`, `.action-load`, but an attribute `data-action` is better semantically. And we can use it in CSS rules too.

The “behavior” pattern

We can also use event delegation to add “behaviors” to elements *declaratively*, with special attributes and classes.

The pattern has two parts:

1. We add a custom attribute to an element that describes its behavior.
2. A document-wide handler tracks events, and if an event happens on an attributed element – performs the action.

Behavior: Counter

For instance, here the attribute `data-counter` adds a behavior: “increase value on click” to buttons:

```
Counter: <input type="button" value="1" data-counter>
One more counter: <input type="button" value="2" data-counter>

<script>
  document.addEventListener('click', function(event) {

    if (event.target.dataset.counter !== undefined) { // if the attribute exists.
      event.target.value++;
    }

  });
</script>
```

Counter: One more counter:

If we click a button – its value is increased. Not buttons, but the general approach is important here.

There can be as many attributes with `data-counter` as we want. We can add new ones to HTML at any moment. Using the event delegation we “extended” HTML, added an attribute that describes a new behavior.

⚠ For document-level handlers – always `addEventListener`

When we assign an event handler to the `document` object, we should always use `addEventListener`, not `document.on<event>`, because the latter will cause conflicts: new handlers overwrite old ones.

For real projects it’s normal that there are many handlers on `document` set by different parts of the code.

Behavior: Toggler

One more example of behavior. A click on an element with the attribute `data-toggle-id` will show/hide the element with the given `id`:

```
<button data-toggle-id="subscribe-mail">
  Show the subscription form
</button>

<form id="subscribe-mail" hidden>
  Your mail: <input type="email">
</form>

<script>
  document.addEventListener('click', function(event) {
    let id = event.target.dataset.toggleId;
    if (!id) return;

    let elem = document.getElementById(id);

    elem.hidden = !elem.hidden;
  });
</script>
```

Show the subscription form

Let’s note once again what we did. Now, to add toggling functionality to an element – there’s no need to know JavaScript, just use the attribute `data-toggle-id`.

That may become really convenient – no need to write JavaScript for every such element. Just use the behavior. The document-level handler makes it work for any element of the page.

We can combine multiple behaviors on a single element as well.

The “behavior” pattern can be an alternative to mini-fragments of JavaScript.

Summary

Event delegation is really cool! It's one of the most helpful patterns for DOM events. It's often used to add the same handling for many similar elements, but not only for that.

The algorithm:

1. Put a single handler on the container.
2. In the handler – check the source element `event.target`.
3. If the event happened inside an element that interests us, then handle the event.

Benefits:

- Simplifies initialization and saves memory: no need to add many handlers.
- Less code: when adding or removing elements, no need to add/remove handlers.
- DOM modifications: we can mass add/remove elements with `innerHTML` and the like.

The delegation has its limitations of course:

- First, the event must be bubbling. Some events do not bubble. Also, low-level handlers should not use `event.stopPropagation()`.
- Second, the delegation may add CPU load, because the container-level handler reacts on events in any place of the container, no matter whether they interest us or not. But usually the load is negligible, so we don't take it into account.

✔ Tasks

Hide messages with delegation

importance: 5

There's a list of messages with removal buttons `[x]`. Make the buttons work.

Like this:

Horse

[x]

The horse is one of two extant subspecies of *Equus ferus*. It is an odd-toed ungulate mammal belonging to the taxonomic family Equidae. The horse has evolved over the past 45 to 55 million years from a small multi-toed creature, *Eohippus*, into the large, single-toed animal of today.

Donkey

[x]

The donkey or ass (*Equus africanus asinus*) is a domesticated member of the horse family, Equidae. The wild ancestor of the donkey is the African wild ass, *E. africanus*. The donkey has been used as a working animal for at least 5000 years.

Cat

[x]

The domestic cat (Latin: *Felis catus*) is a small, typically furry, carnivorous mammal. They are often called house cats when kept as indoor pets or simply cats when there is no need to distinguish them from other felids and felines. Cats are often valued by humans for companionship and for their ability to hunt vermin.

P.S. Should be only one event listener on the container, use event delegation.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Tree menu

importance: 5

Create a tree that shows/hides node children on click:

- Animals
 - Mammals
 - Cows
 - Donkeys
 - Dogs
 - Tigers
 - Other
 - Snakes
 - Birds
 - Lizards
- Fishes
 - Aquarium
 - Guppy
 - Angelfish
 - Sea
 - Sea trout

Requirements:

- Only one event handler (use delegation)

- A click outside the node title (on an empty space) should not do anything.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Sortable table

importance: 4

Make the table sortable: clicks on `<th>` elements should sort it by corresponding column.

Each `<th>` has the type in the attribute, like this:

```
<table id="grid">
  <thead>
    <tr>
      <th data-type="number">Age</th>
      <th data-type="string">Name</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>5</td>
      <td>John</td>
    </tr>
    <tr>
      <td>10</td>
      <td>Ann</td>
    </tr>
    ...
  </tbody>
</table>
```

In the example above the first column has numbers, and the second one – strings. The sorting function should handle sort according to the type.

Only `"string"` and `"number"` types should be supported.

The working example:

Age	Name
5	John
2	Pete
12	Ann
9	Eugene
1	Ilya

P.S. The table can be big, with any number of rows and columns.

[Open a sandbox for the task.](#)

[To solution](#)

Tooltip behavior

importance: 5

Create JS-code for the tooltip behavior.

When a mouse comes over an element with `data-tooltip`, the tooltip should appear over it, and when it's gone then hide.

An example of annotated HTML:

```
<button data-tooltip="the tooltip is longer than the element">Short button</button>
<button data-tooltip="HTML<br>tooltip">One more button</button>
```

Should work like this:

LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa

LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa LaLaLa

Short button One more button

Scroll the page to make buttons appear on the top, check if the tooltips show up correctly.

In this task we assume that all elements with `data-tooltip` have only text inside. No nested tags (yet).

Details:

- The distance between the element and the tooltip should be `5px`.

- The tooltip should be centered relative to the element, if possible.
- The tooltip should not cross window edges. Normally it should be above the element, but if the element is at the page top and there's no space for the tooltip, then below it.
- The tooltip content is given in the `data-tooltip` attribute. It can be arbitrary HTML.

You'll need two events here:

- `mouseover` triggers when a pointer comes over an element.
- `mouseout` triggers when a pointer leaves an element.

Please use event delegation: set up two handlers on `document` to track all “overs” and “outs” from elements with `data-tooltip` and manage tooltips from there.

After the behavior is implemented, even people unfamiliar with JavaScript can add annotated elements.

P.S. Only one tooltip may show up at a time.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Browser default actions

Many events automatically lead to certain actions performed by the browser.

For instance:

- A click on a link – initiates navigation to its URL.
- A click on a form submit button – initiates its submission to the server.
- Pressing a mouse button over a text and moving it – selects the text.

If we handle an event in JavaScript, we may not want the corresponding browser action to happen, and want to implement another behavior instead.

Preventing browser actions

There are two ways to tell the browser we don't want it to act:

- The main way is to use the `event` object. There's a method `event.preventDefault()`.

- If the handler is assigned using `on<event>` (not by `addEventListener`), then returning `false` also works the same.

In this HTML, a click on a link doesn't lead to navigation; the browser doesn't do anything:

```
<a href="/" onclick="return false">Click here</a>  
or  
<a href="/" onclick="event.preventDefault()">here</a>
```

[Click here](#) or [here](#)

In the next example we'll use this technique to create a JavaScript-powered menu.

⚠ Returning `false` from a handler is an exception

The value returned by an event handler is usually ignored.

The only exception is `return false` from a handler assigned using `on<event>`.

In all other cases, `return` value is ignored. In particular, there's no sense in returning `true`.

Example: the menu

Consider a site menu, like this:

```
<ul id="menu" class="menu">  
  <li><a href="/html">HTML</a></li>  
  <li><a href="/javascript">JavaScript</a></li>  
  <li><a href="/css">CSS</a></li>  
</ul>
```

Here's how it looks with some CSS:

HTML

JavaScript

CSS

Menu items are implemented as HTML-links `<a>`, not buttons `<button>`. There are several reasons to do so, for instance:

- Many people like to use “right click” – “open in a new window”. If we use `<button>` or `<span>`, that doesn't work.
- Search engines follow `<a href="...">` links while indexing.

So we use `<a>` in the markup. But normally we intend to handle clicks in JavaScript. So we should prevent the default browser action.

Like here:

```
menu.onclick = function(event) {  
  if (event.target.nodeName !== 'A') return;  
  
  let href = event.target.getAttribute('href');  
  alert( href ); // ...can be loading from the server, UI generation etc  
  
  return false; // prevent browser action (don't go to the URL)  
};
```

If we omit `return false`, then after our code executes the browser will do its “default action” – navigating to the URL in `href`. And we don’t need that here, as we’re handling the click by ourselves.

By the way, using event delegation here makes our menu very flexible. We can add nested lists and style them using CSS to “slide down”.

i Follow-up events

Certain events flow one into another. If we prevent the first event, there will be no second.

For instance, `mousedown` on an `<input>` field leads to focusing in it, and the `focus` event. If we prevent the `mousedown` event, there’s no focus.

Try to click on the first `<input>` below – the `focus` event happens. But if you click the second one, there’s no focus.

```
<input value="Focus works" onfocus="this.value=''">  
<input onmousedown="return false" onfocus="this.value=''" value="Click me">
```

That’s because the browser action is canceled on `mousedown`. The focusing is still possible if we use another way to enter the input. For instance, the `Tab` key to switch from the 1st input into the 2nd. But not with the mouse click any more.

The “passive” handler option

The optional `passive: true` option of `addEventListener` signals the browser that the handler is not going to call `preventDefault()`.

Why might that be needed?

There are some events like `touchmove` on mobile devices (when the user moves their finger across the screen), that cause scrolling by default, but that scrolling can be prevented using `preventDefault()` in the handler.

So when the browser detects such event, it has first to process all handlers, and then if `preventDefault` is not called anywhere, it can proceed with scrolling. That may cause unnecessary delays and “jitters” in the UI.

The `passive: true` options tells the browser that the handler is not going to cancel scrolling. Then browser scrolls immediately providing a maximally fluent experience, and the event is handled by the way.

For some browsers (Firefox, Chrome), `passive` is `true` by default for `touchstart` and `touchmove` events.

event.defaultPrevented

The property `event.defaultPrevented` is `true` if the default action was prevented, and `false` otherwise.

There’s an interesting use case for it.

You remember in the chapter [Bubbling and capturing](#) we talked about `event.stopPropagation()` and why stopping bubbling is bad?

Sometimes we can use `event.defaultPrevented` instead, to signal other event handlers that the event was handled.

Let’s see a practical example.

By default the browser on `contextmenu` event (right mouse click) shows a context menu with standard options. We can prevent it and show our own, like this:

```
<button>Right-click shows browser context menu</button>

<button oncontextmenu="alert('Draw our menu'); return false">
  Right-click shows our context menu
</button>
```

Right-click shows browser context menu Right-click shows our context menu

Now, in addition to that context menu we’d like to implement document-wide context menu.

Upon right click, the closest context menu should show up.

```
<p>Right-click here for the document context menu</p>
<button id="elem">Right-click here for the button context menu</button>
```

```

<script>
  elem.oncontextmenu = function(event) {
    event.preventDefault();
    alert("Button context menu");
  };

  document.oncontextmenu = function(event) {
    event.preventDefault();
    alert("Document context menu");
  };
</script>

```

Right-click here for the document context menu

Right-click here for the button context menu

The problem is that when we click on `elem`, we get two menus: the button-level and (the event bubbles up) the document-level menu.

How to fix it? One of solutions is to think like: “When we handle right-click in the button handler, let’s stop its bubbling” and use `event.stopPropagation()`:

```

<p>Right-click for the document menu</p>
<button id="elem">Right-click for the button menu (fixed with event.stopPropagation())</button>

<script>
  elem.oncontextmenu = function(event) {
    event.preventDefault();
    event.stopPropagation();
    alert("Button context menu");
  };

  document.oncontextmenu = function(event) {
    event.preventDefault();
    alert("Document context menu");
  };
</script>

```

Right-click for the document menu

Right-click for the button menu (fixed with event.stopPropagation())

Now the button-level menu works as intended. But the price is high. We forever deny access to information about right-clicks for any outer code, including counters that gather statistics and so on. That’s quite unwise.

An alternative solution would be to check in the `document` handler if the default action was prevented? If it is so, then the event was handled, and we don’t need to react on it.

```

<p>Right-click for the document menu (added a check for event.defaultPrevented)<
<button id="elem">Right-click for the button menu</button>

<script>
  elem.oncontextmenu = function(event) {
    event.preventDefault();
    alert("Button context menu");
  };

  document.oncontextmenu = function(event) {
    if (event.defaultPrevented) return;

    event.preventDefault();
    alert("Document context menu");
  };
</script>

```

Right-click for the document menu (added a check for event.defaultPrevented)

Right-click for the button menu

Now everything also works correctly. If we have nested elements, and each of them has a context menu of its own, that would also work. Just make sure to check for `event.defaultPrevented` in each `contextmenu` handler.

i event.stopPropagation() and event.preventDefault()

As we can clearly see, `event.stopPropagation()` and `event.preventDefault()` (also known as `return false`) are two different things. They are not related to each other.

i Nested context menus architecture

There are also alternative ways to implement nested context menus. One of them is to have a single global object with a handler for `document.oncontextmenu`, and also methods that allow us to store other handlers in it.

The object will catch any right-click, look through stored handlers and run the appropriate one.

But then each piece of code that wants a context menu should know about that object and use its help instead of the own `contextmenu` handler.

Summary

There are many default browser actions:

- `mousedown` – starts the selection (move the mouse to select).

- `click` on `<input type="checkbox">` – checks/unchecks the `input`.
- `submit` – clicking an `<input type="submit">` or hitting `Enter` inside a form field causes this event to happen, and the browser submits the form after it.
- `keydown` – pressing a key may lead to adding a character into a field, or other actions.
- `contextmenu` – the event happens on a right-click, the action is to show the browser context menu.
- ...there are more...

All the default actions can be prevented if we want to handle the event exclusively by JavaScript.

To prevent a default action – use either `event.preventDefault()` or `return false`. The second method works only for handlers assigned with `on<event>`.

The `passive: true` option of `addEventListener` tells the browser that the action is not going to be prevented. That's useful for some mobile events, like `touchstart` and `touchmove`, to tell the browser that it should not wait for all handlers to finish before scrolling.

If the default action was prevented, the value of `event.defaultPrevented` becomes `true`, otherwise it's `false`.

Stay semantic, don't abuse

Technically, by preventing default actions and adding JavaScript we can customize the behavior of any elements. For instance, we can make a link `<a>` work like a button, and a button `<button>` behave as a link (redirect to another URL or so).

But we should generally keep the semantic meaning of HTML elements. For instance, `<a>` should perform navigation, not a button.

Besides being “just a good thing”, that makes your HTML better in terms of accessibility.

Also if we consider the example with `<a>`, then please note: a browser allows us to open such links in a new window (by right-clicking them and other means). And people like that. But if we make a button behave as a link using JavaScript and even look like a link using CSS, then `<a>`-specific browser features still won't work for it.

Tasks

Why "return false" doesn't work?

importance: 3

Why in the code below `return false` doesn't work at all?

```
<script>
  function handler() {
    alert( "... " );
    return false;
  }
</script>

<a href="https://w3.org" onclick="handler()">the browser will go to w3.org</a>
```

[the browser will go to w3.org](https://w3.org)

The browser follows the URL on click, but we don't want it.

How to fix?

To solution

Catch links in the element

importance: 5

Make all links inside the element with `id="contents"` ask the user if they really want to leave. And if they don't then don't follow.

Like this:

#contents

How about to read [Wikipedia](#) or visit [W3.org](#) and learn about modern standards?

Details:

- HTML inside the element may be loaded or regenerated dynamically at any time, so we can't find all links and put handlers on them. Use event delegation.
- The content may have nested tags. Inside links too, like `<a href=".."><i>...</i></a>`.

[Open a sandbox for the task.](#) ↗

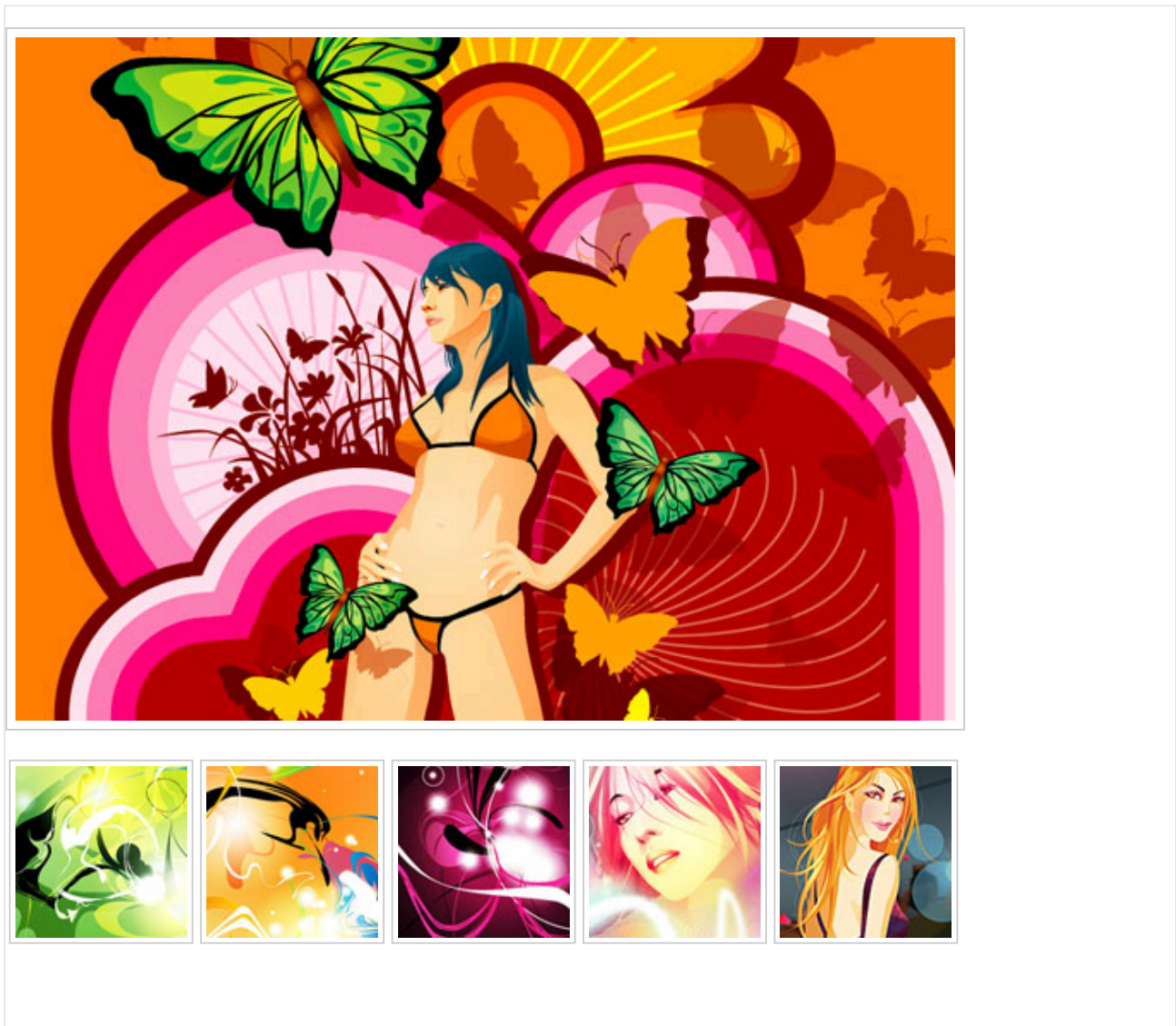
To solution

Image gallery

importance: 5

Create an image gallery where the main image changes by the click on a thumbnail.

Like this:



P.S. Use event delegation.

[Open a sandbox for the task.](#) ↗

[To solution](#)

Dispatching custom events

We can not only assign handlers, but also generate events from JavaScript.

Custom events can be used to create “graphical components”. For instance, a root element of our own JS-based menu may trigger events telling what happens with the menu: `open` (menu open), `select` (an item is selected) and so on. Another code may listen for the events and observe what’s happening with the menu.

We can generate not only completely new events, that we invent for our own purposes, but also built-in ones, such as `click`, `mousedown` etc. That may be helpful for automated testing.

Event constructor

Built-in event classes form a hierarchy, similar to DOM element classes. The root is the built-in `Event` class.

We can create `Event` objects like this:

```
let event = new Event(type[, options]);
```

Arguments:

- *type* – event type, a string like `"click"` or our own like `"my-event"`.
- *options* – the object with two optional properties:
 - `bubbles: true/false` – if `true`, then the event bubbles.
 - `cancelable: true/false` – if `true`, then the “default action” may be prevented. Later we’ll see what it means for custom events.

By default both are false: `{bubbles: false, cancelable: false}`.

dispatchEvent

After an event object is created, we should “run” it on an element using the call `elem.dispatchEvent(event)`.

Then handlers react on it as if it were a regular browser event. If the event was created with the `bubbles` flag, then it bubbles.

In the example below the `click` event is initiated in JavaScript. The handler works same way as if the button was clicked:

```
<button id="elem" onclick="alert('Click!');">Autoclick</button>

<script>
  let event = new Event("click");
  elem.dispatchEvent(event);
</script>
```

event.isTrusted

There is a way to tell a “real” user event from a script-generated one.

The property `event.isTrusted` is `true` for events that come from real user actions and `false` for script-generated events.

Bubbling example

We can create a bubbling event with the name `"hello"` and catch it on `document`.

All we need is to set `bubbles` to `true`:

```
<h1 id="elem">Hello from the script!</h1>

<script>
  // catch on document...
  document.addEventListener("hello", function(event) { // (1)
    alert("Hello from " + event.target.tagName); // Hello from H1
  });

  // ...dispatch on elem!
  let event = new Event("hello", {bubbles: true}); // (2)
  elem.dispatchEvent(event);

  // the handler on document will activate and display the message.

</script>
```

Notes:

1. We should use `addEventListener` for our custom events, because `on<event>` only exists for built-in events, `document.onhello` doesn't work.
2. Must set `bubbles: true`, otherwise the event won't bubble up.

The bubbling mechanics is the same for built-in (`click`) and custom (`hello`) events. There are also capturing and bubbling stages.

MouseEvent, KeyboardEvent and others

Here's a short list of classes for UI Events from the [UI Event specification](#) ↗:

- `UIEvent`
- `FocusEvent`
- `MouseEvent`
- `WheelEvent`

- `KeyboardEvent`
- ...

We should use them instead of `new Event` if we want to create such events. For instance, `new MouseEvent("click")`.

The right constructor allows to specify standard properties for that type of event.

Like `clientX/clientY` for a mouse event:

```
let event = new MouseEvent("click", {
  bubbles: true,
  cancelable: true,
  clientX: 100,
  clientY: 100
});
```

```
alert(event.clientX); // 100
```

Please note: the generic `Event` constructor does not allow that.

Let's try:

```
let event = new Event("click", {
  bubbles: true, // only bubbles and cancelable
  cancelable: true, // work in the Event constructor
  clientX: 100,
  clientY: 100
});
```

```
alert(event.clientX); // undefined, the unknown property is ignored!
```

Technically, we can work around that by assigning directly `event.clientX=100` after creation. So that's a matter of convenience and following the rules. Browser-generated events always have the right type.

The full list of properties for different UI events is in the specification, for instance, [MouseEvent](#) ↗.

Custom events

For our own, completely new events types like `"hello"` we should use `new CustomEvent`. Technically [CustomEvent](#) ↗ is the same as `Event`, with one exception.

In the second argument (object) we can add an additional property `detail` for any custom information that we want to pass with the event.

For instance:

```
<h1 id="elem">Hello for John!</h1>

<script>
  // additional details come with the event to the handler
  elem.addEventListener("hello", function(event) {
    alert(event.detail.name);
  });

  elem.dispatchEvent(new CustomEvent("hello", {
    detail: { name: "John" }
  }));
</script>
```

The `detail` property can have any data. Technically we could live without, because we can assign any properties into a regular `new Event` object after its creation. But `CustomEvent` provides the special `detail` field for it to evade conflicts with other event properties.

Besides, the event class describes “what kind of event” it is, and if the event is custom, then we should use `CustomEvent` just to be clear about what it is.

event.preventDefault()

Many browser events have a “default action”, such as navigating to a link, starting a selection, and so on.

For new, custom events, there are definitely no default browser actions, but a code that dispatches such event may have its own plans what to do after triggering the event.

By calling `event.preventDefault()`, an event handler may send a signal that those actions should be canceled.

In that case the call to `elem.dispatchEvent(event)` returns `false`. And the code that dispatched it knows that it shouldn't continue.

Let's see a practical example – a hiding rabbit (could be a closing menu or something else).

Below you can see a `#rabbit` and `hide()` function that dispatches `"hide"` event on it, to let all interested parties know that the rabbit is going to hide.

Any handler can listen for that event with `rabbit.addEventListener('hide', ...)` and, if needed, cancel the action using `event.preventDefault()`. Then the rabbit won't disappear:

```

<pre id="rabbit">
  |\  /|
  \_|_/
  /. .\
  =\Y_/_=
  {>o<}
</pre>
<button onclick="hide()">Hide()</button>

<script>
  function hide() {
    let event = new CustomEvent("hide", {
      cancelable: true // without that flag preventDefault doesn't work
    });
    if (!rabbit.dispatchEvent(event)) {
      alert('The action was prevented by a handler');
    } else {
      rabbit.hidden = true;
    }
  }

  rabbit.addEventListener('hide', function(event) {
    if (confirm("Call preventDefault?")) {
      event.preventDefault();
    }
  });
</script>

```

```

|\  /|
\_|_/
/. .\
=\Y_/_=
{>o<}

```

Hide()

Please note: the event must have the flag `cancelable: true`, otherwise the call `event.preventDefault()` is ignored.

Events-in-events are synchronous

Usually events are processed in a queue. That is: if the browser is processing `onclick` and a new event occurs, e.g. mouse moved, then its handling is queued up, corresponding `mousemove` handlers will be called after `onclick` processing is finished.

The notable exception is when one event is initiated from within another one, e.g. using `dispatchEvent`. Such events are processed immediately: the new event handlers are called, and then the current event handling is resumed.

For instance, in the code below the `menu-open` event is triggered during the `onclick`.

It's processed immediately, without waiting for `onclick` handler to end:

```
<button id="menu">Menu (click me)</button>

<script>
  menu.onclick = function() {
    alert(1);

    menu.dispatchEvent(new CustomEvent("menu-open", {
      bubbles: true
    }));

    alert(2);
  };

  // triggers between 1 and 2
  document.addEventListener('menu-open', () => alert('nested'));
</script>
```

Menu (click me)

The output order is: 1 → nested → 2.

Please note that the nested event `menu-open` is caught on the `document`. The propagation and handling of the nested event is finished before the processing gets back to the outer code (`onclick`).

That's not only about `dispatchEvent`, there are other cases. If an event handler calls methods that trigger other events – they are processed synchronously too, in a nested fashion.

Let's say we don't like it. We'd want `onclick` to be fully processed first, independently from `menu-open` or any other nested events.

Then we can either put the `dispatchEvent` (or another event-triggering call) at the end of `onclick` or, maybe better, wrap it in the zero-delay `setTimeout`:

```
<button id="menu">Menu (click me)</button>

<script>
  menu.onclick = function() {
    alert(1);

    setTimeout(() => menu.dispatchEvent(new CustomEvent("menu-open", {
      bubbles: true
    }))), 0);

    alert(2);
  };
</script>
```

```
document.addEventListener('menu-open', () => alert('nested'));  
</script>
```

Now `dispatchEvent` runs asynchronously after the current code execution is finished, including `menu.onclick`, so event handlers are totally separate.

The output order becomes: 1 → 2 → nested.

Summary

To generate an event from code, we first need to create an event object.

The generic `Event(name, options)` constructor accepts an arbitrary event name and the `options` object with two properties:

- `bubbles: true` if the event should bubble.
- `cancelable: true` if the `event.preventDefault()` should work.

Other constructors of native events like `MouseEvent`, `KeyboardEvent` and so on accept properties specific to that event type. For instance, `clientX` for mouse events.

For custom events we should use `CustomEvent` constructor. It has an additional option named `detail`, we should assign the event-specific data to it. Then all handlers can access it as `event.detail`.

Despite the technical possibility of generating browser events like `click` or `keydown`, we should use them with great care.

We shouldn't generate browser events as it's a hacky way to run handlers. That's bad architecture most of the time.

Native events might be generated:

- As a dirty hack to make 3rd-party libraries work the needed way, if they don't provide other means of interaction.
- For automated testing, to "click the button" in the script and see if the interface reacts correctly.

Custom events with our own names are often generated for architectural purposes, to signal what happens inside our menus, sliders, carousels etc.

UI Events

Here we cover most important user interface events and how to work with them.